Generative AI Terminologies: LLM, Agent, and MCP

LLM (Large Language Model):

An **LLM** is an advanced AI system trained to understand and generate human-like text.
It can read, write, and respond to questions in natural language.

Popular LLMs

- ChatGPT
- Google Gemini
- Claude
- DeepSeek

What LLMs Can Do

- Answer questions and explain topics
- Generate code or scripts
- Write emails, blogs, and documents
- Chat naturally with users

Example:

You can ask ChatGPT:

"Write a login test case in Java using Selenium."
It will generate the code for you — but it **won't run the test** itself.

Limitations of LLMs

LLMs are good at *thinking* and *writing*, but they can't *take real actions* in the outside world.

They **cannot**:

- Open a browser or click buttons
- Connect to a database
- Call APIs to get live data

They can only **generate** the instructions or code, not **execute** them.

Agent:

An **Agent** is a system that takes help from an LLM and performs real-world actions using external tools.

Think of it as an **assistant** that actually *does* what the LLM *says*.

Example:
If the LLM says “Run this Playwright test,”
the **Agent** will execute that test in a real browser.

So:

- The **LLM** thinks and plans.
- The **Agent** executes and performs.

MCP (Model Context Protocol):

MCP is a framework that connects LLMs to real-world tools like browsers, databases, and APIs.
It acts as a **bridge** that allows LLMs to send commands to tools and get back the results.

What MCP Can Do

1. **Browser Automation**
    - Open a website, fill forms, and click buttons
    - Example: Selenium MCP or Playwright MCP
2. **Database Operations**
    - Connect to MySQL or PostgreSQL and run queries
    - Example: MySQL MCP or PostgreSQL MCP
3. **API Requests**
    - Send or receive real-time API responses
    - Example: API MCP for calling web services

So, MCP helps the LLM and Agent work together to perform live, practical tasks.

Playwright MCP:

Playwright MCP connects LLMs to browsers using the **Playwright** automation tool (similar to Selenium).

Key Features

- Allows LLMs to control browsers (open sites, fill forms, click elements)
- Understands the web page structure using the **Accessibility Tree** (not images)
- Supports browser commands such as:
    - click()
    - fill('user@example.com')

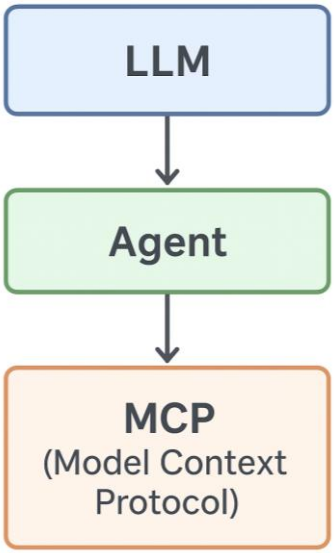
Example

You tell ChatGPT:

"Search for 'Selenium vs Playwright' and click the first result."

Behind the scenes:

- ChatGPT (LLM) sends this instruction to **Playwright MCP**.
- Playwright MCP opens a browser and performs the task automatically.



Summary

Term	In Simple Words	Example Scenario
LLM	The brain that understands and generates text.	GPT-5 explaining "What is API Testing?"
Agent	The assistant that uses the brain to perform actions.	Agent running a Playwright test based on ChatGPT's instructions.
MCP	The bridge that connects the brain and assistant to external tools.	ChatGPT using MCP to open files or run code in PyCharm.

conftest.py

```python
import pytest
import allure
from pathlib import Path
from playwright.sync_api import sync_playwright

# ============================================================================
# PYTEST + PLAYWRIGHT TEST CONFIGURATION FILE
# ============================================================================
# This file provides:
# 1. Command-line options (browser, base URL, video, screenshots, etc.)
# 2. Hooks to track test results
# 3. Fixtures for browser setup and teardown
# 4. Screenshot, video, and trace attachments to Allure reports
# ============================================================================


# ----------------------------------------------------------------------------
# STEP 1: ADD COMMAND LINE OPTIONS
# ----------------------------------------------------------------------------
def pytest_addoption(parser):
    """
    Adds command line options for test configuration.
    You can override these when running pytest or store defaults in pytest.ini.
    """
    parser.addoption("--browser", default="chromium", help="Browser: chromium, firefox, webkit")
    parser.addoption("--headed", action="store_true", help="Run in headed (visible) mode")
    parser.addoption("--base-url", default="https://tutorialsninja.com/demo/", help="Base URL
for tests")
    parser.addoption("--video", default="retain-on-failure", help="Record video: on, off,
retain-on-failure")
    parser.addoption("--screenshot", default="only-on-failure", help="Take screenshot: on, off,
only-on-failure")
    parser.addoption("--tracing", default="retain-on-failure", help="Tracing: on, off, retain-
on-failure")


# ----------------------------------------------------------------------------
# STEP 2: GET CONFIGURATION VALUE (CMDLINE OR pytest.ini)
# ----------------------------------------------------------------------------
def get_config_value(config, option_name):
    """
    Helper to read configuration values.
    Tries to get from command line first, otherwise from pytest.ini.
    Supports both string and boolean options.
    """
    # Try command-line first
    cmd_value = config.getoption(option_name)
    if cmd_value is not None:
        return cmd_value

    # Fallback to pytest.ini
    if option_name == "headed":
        ini_value = config.getini(option_name)
        return ini_value.lower() == "true" if isinstance(ini_value, str) else ini_value
    else:
        return config.getini(option_name)


# ----------------------------------------------------------------------------
# STEP 3: HOOK TO TRACK TEST RESULTS (PASS/FAIL)
# ----------------------------------------------------------------------------
@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    """
    Captures the test result (pass/fail/skip) after each test.
```

```
    This is used later to decide whether to take screenshots or save traces.
    """
    outcome = yield
    report = outcome.get_result()
    setattr(item, f"rep_{report.when}", report)


# ------------------------------------------------------------------------
# STEP 4: FIXTURE 1 - BROWSER CONTEXT SETUP
# ------------------------------------------------------------------------
@pytest.fixture(scope="function")
def browser_context(request):
    """
    Creates and manages the Playwright browser context.
    - Reads configuration (browser, headed mode, video settings)
    - Starts the Playwright browser
    - Enables video recording if configured
    - Cleans up automatically after each test
    """
    # Read configuration values
    browser_name = get_config_value(request.config, "browser")
    headed_flag = get_config_value(request.config, "headed")
    video_option = get_config_value(request.config, "video")

    print(f"🎯 Starting browser: {browser_name}")
    print(f"🎯 Headless mode: {not headed_flag} (headed={headed_flag})")

    # Start Playwright
    playwright = sync_playwright().start()

    # Launch the specified browser
    if browser_name.lower() == "chromium":
        browser = playwright.chromium.launch(headless=not headed_flag)
    elif browser_name.lower() == "firefox":
        browser = playwright.firefox.launch(headless=not headed_flag)
    elif browser_name.lower() == "webkit":
        browser = playwright.webkit.launch(headless=not headed_flag)
    else:
        raise ValueError(f"❌ Unsupported browser: {browser_name}")

    # Create a browser context (optionally with video recording)
    if video_option in ["on", "retain-on-failure"]:
        context = browser.new_context(record_video_dir="reports/videos")
    else:
        context = browser.new_context()

    # Yield the context for use in tests
    yield context

    # Clean up after the test
    print("🧹 Closing browser context and stopping Playwright...")
    context.close()
    browser.close()
    playwright.stop()


# ------------------------------------------------------------------------
# STEP 5: FIXTURE 2 - PAGE CREATION AND TEST ARTIFACT MANAGEMENT
# ------------------------------------------------------------------------
@pytest.fixture(scope="function")
def page(request, browser_context):
    """
    Creates a new browser page for each test.
    - Navigates to the base URL
    - Starts tracing (if enabled)
    - Captures screenshots, traces, and videos for failed tests
    - Attaches all artifacts to Allure report
    """
    # Read test configuration
```

```python
base_url = get_config_value(request.config, "base_url")
screenshot_option = get_config_value(request.config, "screenshot")
tracing_option = get_config_value(request.config, "tracing")
video_option = get_config_value(request.config, "video")

print(f"🌐 Navigating to: {base_url}")

# Start tracing if enabled
if tracing_option in ["on", "retain-on-failure"]:
    print("📼 Tracing enabled - capturing screenshots and actions")
    browser_context.tracing.start(screenshots=True, snapshots=True, sources=True)

# Create and navigate to base URL
page = browser_context.new_page()
page.goto(base_url)

# Yield the page to the test
yield page

# ------------------------------------------------------------------------
# After the test: manage artifacts (screenshots, videos, traces)
# ------------------------------------------------------------------------
test_name = request.node.name
test_failed = hasattr(request.node, "rep_call") and request.node.rep_call.failed

print(f"📊 Test '{test_name}' result: {'❌ FAILED' if test_failed else '☑️ PASSED'}")

# Save and attach trace
if tracing_option in ["on", "retain-on-failure"]:
    trace_path = f"reports/traces/{test_name}_trace.zip"
    browser_context.tracing.stop(path=trace_path)
    print(f"💾 Trace saved: {trace_path}")

    # Attach trace to Allure report if test failed
    # Currently ZIP file is not supporting to attach in allure reports
    # if test_failed:
    #     allure.attach.file(
    #         trace_path,
    #         name=f"{test_name}_trace",
    #         attachment_type=allure.attachment_type.ZIP
    #     )
    #     print("📎 Trace attached to Allure report")

# Take screenshot if test failed
if test_failed and screenshot_option in ["on", "only-on-failure"]:
    screenshot_path = f"reports/screenshots/{test_name}.png"
    page.screenshot(path=screenshot_path)
    print(f"📸 Screenshot saved: {screenshot_path}")

    # Attach to Allure report
    allure.attach.file(
        screenshot_path,
        name=f"{test_name}_screenshot",
        attachment_type=allure.attachment_type.PNG
    )
    print("📎 Screenshot attached to Allure report")

# Attach video if available and test failed
if test_failed and video_option in ["on", "retain-on-failure"]:
    video_path = page.video.path() if page.video else None
    if video_path and Path(video_path).exists():
        allure.attach.file(
            video_path,
            name=f"{test_name}_video",
            attachment_type=allure.attachment_type.WEBM
        )
        print("🤖 Video attached to Allure report")
```

Code Explanation

This code is designed to **set up an automation testing environment** using **Pytest + Playwright**, with built-in support for:

✅ Browser control
✅ Screenshot, video, and trace recording
✅ Reporting integration via **Allure**

1️⃣ Command-line Options (pytest_addoption)

You can run tests with different configurations like:

pytest --browser=firefox --headed --video=on

These options can also come from your pytest.ini file.

2️⃣ Config Value Helper (get_config_value)

This function ensures that your framework checks:

1. Command-line arguments first (highest priority)
2. Then falls back to pytest.ini defaults

So you can either run pytest directly or customize pytest.ini easily.

3️⃣ Test Report Hook (pytest_runtest_makereport)

Tracks the **result** of each test:

- Pass
- Fail
- Skip

Later, this info decides whether to take screenshots or save traces.

4️⃣ Browser Context Fixture (browser_context)

Manages the **Playwright browser**:

- Launches Chromium, Firefox, or WebKit
- Starts in headless or headed mode
- Records video (if enabled)
- Cleans up automatically after tests

Yielding the context allows tests to use it temporarily — once the test is done, cleanup happens automatically.

5️⃣ Page Fixture (page)

Responsible for:

- Opening a **new browser tab (page)**
- Navigating to the **base URL**
- Starting **tracing** (recording actions + screenshots)
- Capturing **screenshots, videos, and traces** when a test fails
- Attaching all artifacts to the **Allure report**

This fixture ensures **every test starts cleanly** and leaves behind helpful debugging evidence if something fails.

📦 Artifacts Saved

Type	Path	Trigger
Screenshot	reports/screenshots/	Only when test fails
Video	reports/videos/	Based on --video setting
Trace	reports/traces/	Based on --tracing setting

⚡ In Short

This setup provides:

- ✅ Clean and reusable browser management
- ✅ Support for all browsers (Chromium, Firefox, WebKit)
- ✅ Configurable video, screenshot, and tracing
- ✅ Allure-ready reporting integration
- ✅ Easy to extend and understand for beginners

pytest.ini

```
# ========================================
# pytest.ini - Pytest Configuration File
# ========================================

[pytest]

addopts =
    # ------------------------------
    # General Options
    # ------------------------------
    -v
    --browser=chromium
    --headed
    #--base-url=http://localhost/opencart/upload/
    #--base-url=https://tutorialsninja.com/demo/
    --base-url=https://naveenautomationlabs.com/opencart

    # ------------------------------
    # Captures and Reports (Debugging)
    # ------------------------------
    --video=retain-on-failure
    #--video=on
    --screenshot=only-on-failure
    #--screenshot=on
    --tracing=retain-on-failure
    #--tracing=on
    --html=reports/myreport.html --self-contained-html --capture=tee-sys
    --alluredir=reports/allure-results

    # ------------------------------
    # Parallel Execution
    # ------------------------------
    #-n=1
    # --numprocesses=2

    # ------------------------------
    # Test Grouping (Markers)
    # ------------------------------
    # Use one of the below commands when running pytest:
    # -m "sanity"
    # -m "regression"
    # -m "sanity or regression"
    # -m "sanity and regression"
    # -m "sanity and not regression"
    # -m "regression and not sanity"

    -m "sanity or regression"

    # ------------------------------
    # Flaky Test Handling
    # ------------------------------
    #--reruns 2
    #--reruns-delay 2


# ------------------------------
# Custom Markers
# ------------------------------
markers =
    sanity                      # Marks a sanity test (basic quick check)
    regression                  # Marks a regression test (full end-to-end validation)
```

Playwright + Python /Pytest automation framework

1) Create New Project in PyCharm IDE.

2) Install Required Packages

pytest
playwright
pytest-xdist
pytest-html
allure-pytest
pytest-rerunfailures
openpyxl
Faker
python-slugify

Option 1: single command to install all the required plugins:

pip install pytest playwright pytest-xdist pytest-html allure-pytest pytest-rerunfailures openpyxl Faker python-slugify

Option 2: Create **requirements.txt** file under the root directory of the project with all the plugins.

Ex:

playwright
pytest-xdist
pytest-html
allure-pytest
pytest-rerunfailures
openpyxl
Faker
python-slugify

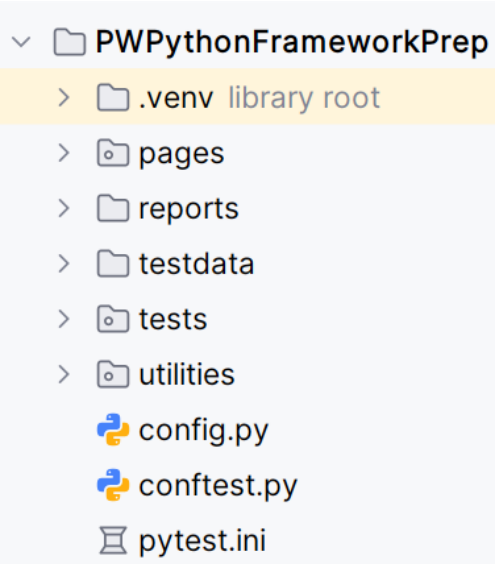
Once you have this file in your project root, install all dependencies in one go by running the below command from terminal.

pip install -r requirements.txt

Install playwright browsers

playwright install

3) Create Folder Structure



4) Create Page Object Classes (under pages/)

Example files:

home_page.py
registration_page.py
login_page.py
logout_page.py
my_account_page.py
product_page.py
search_results_page.py
shopping_cart_page.py

5) Create Test Data Files (under testdata/)

Example:

logindata.json
logindata.csv
logindata.xlsx

Used for data-driven testing.

6) Create Utility Files (under utilities/)

- random_data_util.py → generate random emails, names, etc. using Faker.
- data_reader_util.py → read test data from JSON/CSV/Excel files.

7) Create config.py

Store reusable data:

Example:

```
class Config:
    email="pavanol@abc.com"
    password="test@123"

    invalid_email="pavanol123@abc.com"
    invalid_password="test@123xyz"
    product_name = "MacBook"
    product_quantity = "1"
    total_price = "$602.00"
```

8) Create pytest.ini

Common CLI options:

-v : verbose - Run tests in verbose mode (shows detailed test names and results)
--browser : Specify browser type (chromium, firefox, or webkit)
--headed : Run browser in headed mode (shows browser UI)
--base-url : Define the base URL for all tests
--video : Record video only if the test fails
--screenshot : Capture screenshot only on failure
--tracing : Save Playwright trace file only if the test fails
--html : Generate HTML report
--self-contained-html : Embed all report assets (CSS, JS, images) inside the HTML
--capture=tee-sys : Display console logs while also capturing them for reports
--alluredir : Generate Allure-compatible results for advanced reporting
-n=1 or --numprocesses :Run tests parallel
--reruns : Retry failed tests up to 2 times
--reruns-delay : Wait time before retrying failed tests

9) Create conftest.py

- Handle browser setup/teardown.
- Manage screenshots, video, and trace recording.
- Integrate HTML and Allure reporting.

10) Create Test Files (under tests/)

Example tests:

```
test_user_registration.py
test_login.py
```

test_login_data_driven.py
test_product_search.py
test_user_logout.py

Use page objects inside tests.
Group tests with **@pytest.mark.sanity** or **@pytest.mark.regression**.

11) Run Tests

Browsers: Chromium, Firefox, webkit
Grouping: sanity, regression
Re-run flaky tests
Run tests parallel
Run tests headed/headless

Note: If you have noticed errors like below while running the tests.

ValueError: option name {'--base-url'} already added

That means the command-line option --base-url is being registered twice — once by Playwright's pytest plugin (pytest-playwright), and again by your custom conftest.py.

So, you need to run the below command through terminal to un-install plugin.

pip uninstall pytest-base-url

12) Generate and View the Allure Report

Run time:

allure serve reports/allure-results

Permanent report:

allure generate reports/allure-results -o reports/allure-report --clean

Playwright + Python /Pytest automation framework

1) Create New Project in PyCharm IDE.

2) Install Required Packages

pytest
playwright
pytest-xdist
pytest-html
allure-pytest
pytest-rerunfailures
openpyxl
Faker
python-slugify

Option 1: single command to install all the required plugins:

pip install pytest playwright pytest-xdist pytest-html allure-pytest pytest-rerunfailures openpyxl Faker python-slugify

Option 2: Create **requirements.txt** file under the root directory of the project with all the plugins.

Ex:

playwright
pytest-xdist
pytest-html
allure-pytest
pytest-rerunfailures
openpyxl
Faker
python-slugify

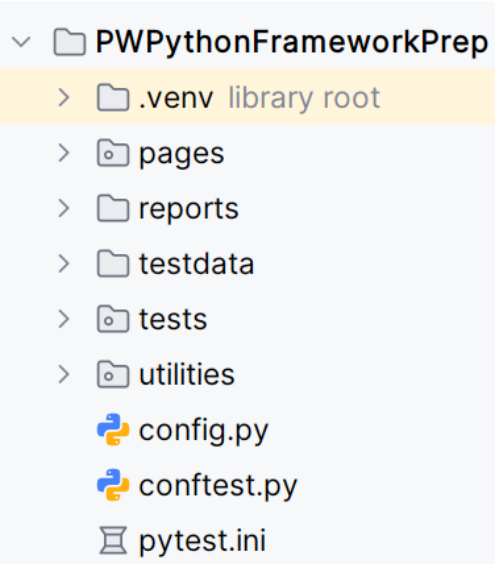
Once you have this file in your project root, install all dependencies in one go by running the below command from terminal.

pip install -r requirements.txt

Install playwright browsers

playwright install

3) Create Folder Structure



4) Create Page Object Classes (under pages/)

Example files:

home_page.py
registration_page.py
login_page.py
logout_page.py
my_account_page.py
product_page.py
search_results_page.py
shopping_cart_page.py

5) Create Test Data Files (under testdata/)

Example:

logindata.json
logindata.csv
logindata.xlsx

Used for data-driven testing.

6) Create Utility Files (under utilities/)

- random_data_util.py → generate random emails, names, etc. using Faker.
- data_reader_util.py → read test data from JSON/CSV/Excel files.

7) Create config.py

Store reusable data:

Example:

```
class Config:
    email="pavanol@abc.com"
    password="test@123"

    invalid_email="pavanol123@abc.com"
    invalid_password="test@123xyz"
    product_name = "MacBook"
    product_quantity = "1"
    total_price = "$602.00"
```

8) Create pytest.ini

Common CLI options:

-v : verbose - Run tests in verbose mode (shows detailed test names and results)
--browser : Specify browser type (chromium, firefox, or webkit)
--headed : Run browser in headed mode (shows browser UI)
--base-url : Define the base URL for all tests
--video : Record video only if the test fails
--screenshot : Capture screenshot only on failure
--tracing : Save Playwright trace file only if the test fails
--html : Generate HTML report
--self-contained-html : Embed all report assets (CSS, JS, images) inside the HTML
--capture=tee-sys : Display console logs while also capturing them for reports
--alluredir : Generate Allure-compatible results for advanced reporting
-n=1 or --numprocesses :Run tests parallel
--reruns : Retry failed tests up to 2 times
--reruns-delay : Wait time before retrying failed tests

9) Create conftest.py

- Handle browser setup/teardown.
- Manage screenshots, video, and trace recording.
- Integrate HTML and Allure reporting.

10) Create Test Files (under tests/)

Example tests:

```
test_user_registration.py
test_login.py
```

test_login_data_driven.py
test_product_search.py
test_user_logout.py

Use page objects inside tests.
Group tests with **@pytest.mark.sanity** or **@pytest.mark.regression**.

11) Run Tests

Browsers: Chromium, Firefox, webkit
Grouping: sanity, regression
Re-run flaky tests
Run tests parallel
Run tests headed/headless

Note: If you have noticed errors like below while running the tests.

ValueError: option name {'--base-url'} already added

That means the command-line option --base-url is being registered twice — once by Playwright's pytest plugin (pytest-playwright), and again by your custom conftest.py.

So, you need to run the below command through terminal to un-install plugin.

pip uninstall pytest-base-url

12) Generate and View the Allure Report

Run time:

allure serve reports/allure-results

Permanent report:

allure generate reports/allure-results -o reports/allure-report --clean

Playwright + Python /Pytest automation framework

1) Create New Project in PyCharm IDE.

2) Install Required Packages

pytest
playwright
pytest-xdist
pytest-html
allure-pytest
pytest-rerunfailures
openpyxl
Faker
python-slugify

Option 1: single command to install all the required plugins:

pip install pytest playwright pytest-xdist pytest-html allure-pytest pytest-rerunfailures openpyxl Faker python-slugify

Option 2: Create **requirements.txt** file under the root directory of the project with all the plugins.

Ex:

playwright
pytest-xdist
pytest-html
allure-pytest
pytest-rerunfailures
openpyxl
Faker
python-slugify

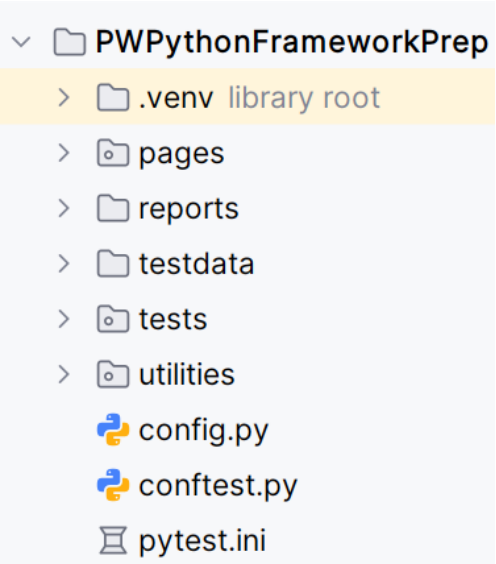
Once you have this file in your project root, install all dependencies in one go by running the below command from terminal.

pip install -r requirements.txt

Install playwright browsers

playwright install

3) Create Folder Structure



4) Create Page Object Classes (under pages/)

Example files:

home_page.py
registration_page.py
login_page.py
logout_page.py
my_account_page.py
product_page.py
search_results_page.py
shopping_cart_page.py

5) Create Test Data Files (under testdata/)

Example:

logindata.json
logindata.csv
logindata.xlsx

Used for data-driven testing.

6) Create Utility Files (under utilities/)

- random_data_util.py → generate random emails, names, etc. using Faker.
- data_reader_util.py → read test data from JSON/CSV/Excel files.

7) Create config.py

Store reusable data:

Example:

```
class Config:
    email="pavanol@abc.com"
    password="test@123"

    invalid_email="pavanol123@abc.com"
    invalid_password="test@123xyz"
    product_name = "MacBook"
    product_quantity = "1"
    total_price = "$602.00"
```

8) Create pytest.ini

Common CLI options:

-v : verbose - Run tests in verbose mode (shows detailed test names and results)
--browser : Specify browser type (chromium, firefox, or webkit)
--headed : Run browser in headed mode (shows browser UI)
--base-url : Define the base URL for all tests
--video : Record video only if the test fails
--screenshot : Capture screenshot only on failure
--tracing : Save Playwright trace file only if the test fails
--html : Generate HTML report
--self-contained-html : Embed all report assets (CSS, JS, images) inside the HTML
--capture=tee-sys : Display console logs while also capturing them for reports
--alluredir : Generate Allure-compatible results for advanced reporting
-n=1 or --numprocesses :Run tests parallel
--reruns : Retry failed tests up to 2 times
--reruns-delay : Wait time before retrying failed tests

9) Create conftest.py

- Handle browser setup/teardown.
- Manage screenshots, video, and trace recording.
- Integrate HTML and Allure reporting.

10) Create Test Files (under tests/)

Example tests:

```
test_user_registration.py
test_login.py
```

test_login_data_driven.py
test_product_search.py
test_user_logout.py

Use page objects inside tests.
Group tests with **@pytest.mark.sanity** or **@pytest.mark.regression**.

11) Run Tests

Browsers: Chromium, Firefox, webkit
Grouping: sanity, regression
Re-run flaky tests
Run tests parallel
Run tests headed/headless

Note: If you have noticed errors like below while running the tests.

ValueError: option name {'--base-url'} already added

That means the command-line option --base-url is being registered twice — once by Playwright's pytest plugin (pytest-playwright), and again by your custom conftest.py.

So, you need to run the below command through terminal to un-install plugin.

pip uninstall pytest-base-url

12) Generate and View the Allure Report

Run time:

allure serve reports/allure-results

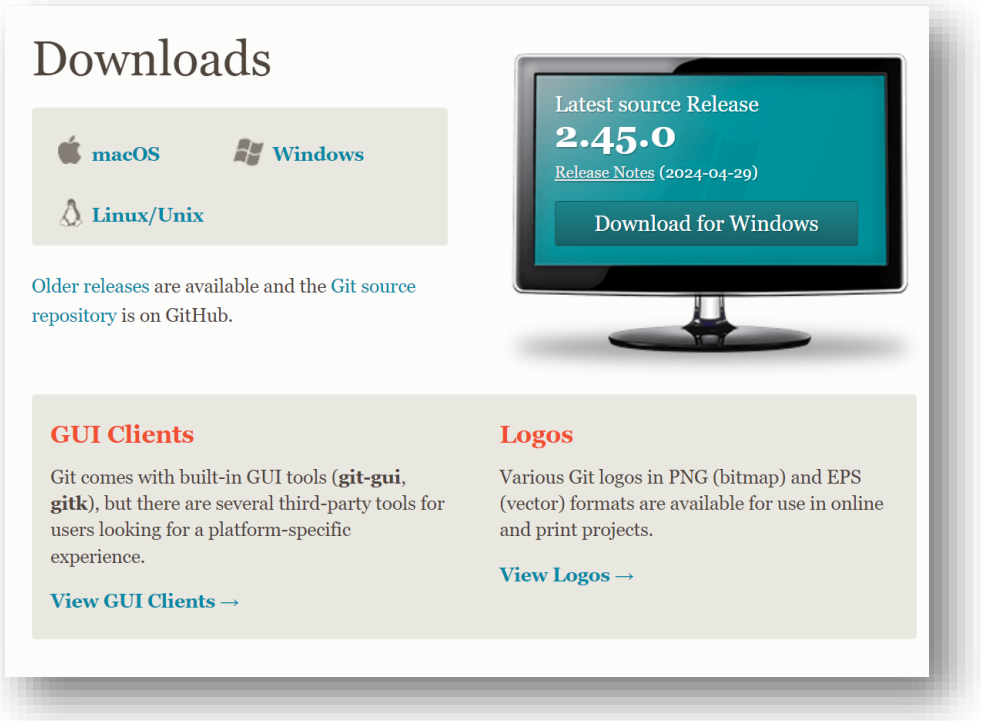
Permanent report:

allure generate reports/allure-results -o reports/allure-report --clean

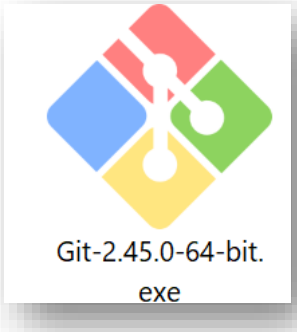
GIT Installation

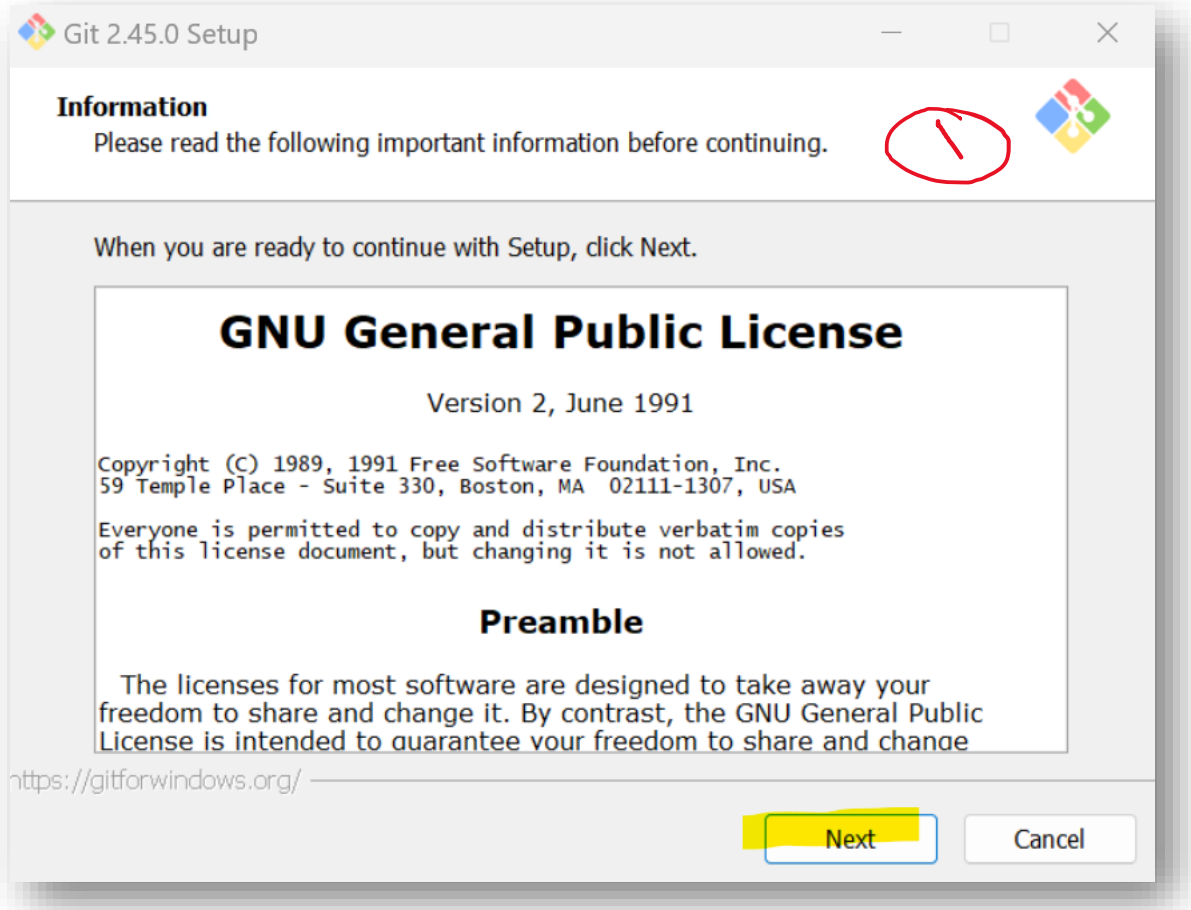
Download Git:

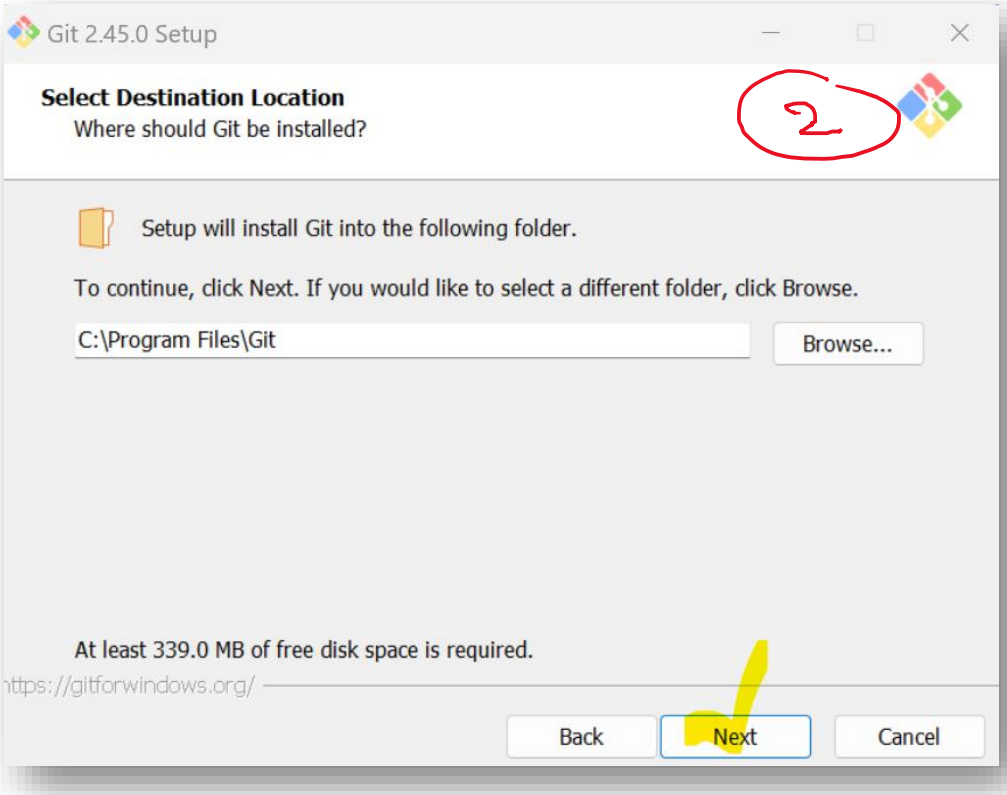
https://git-scm.com/downloads

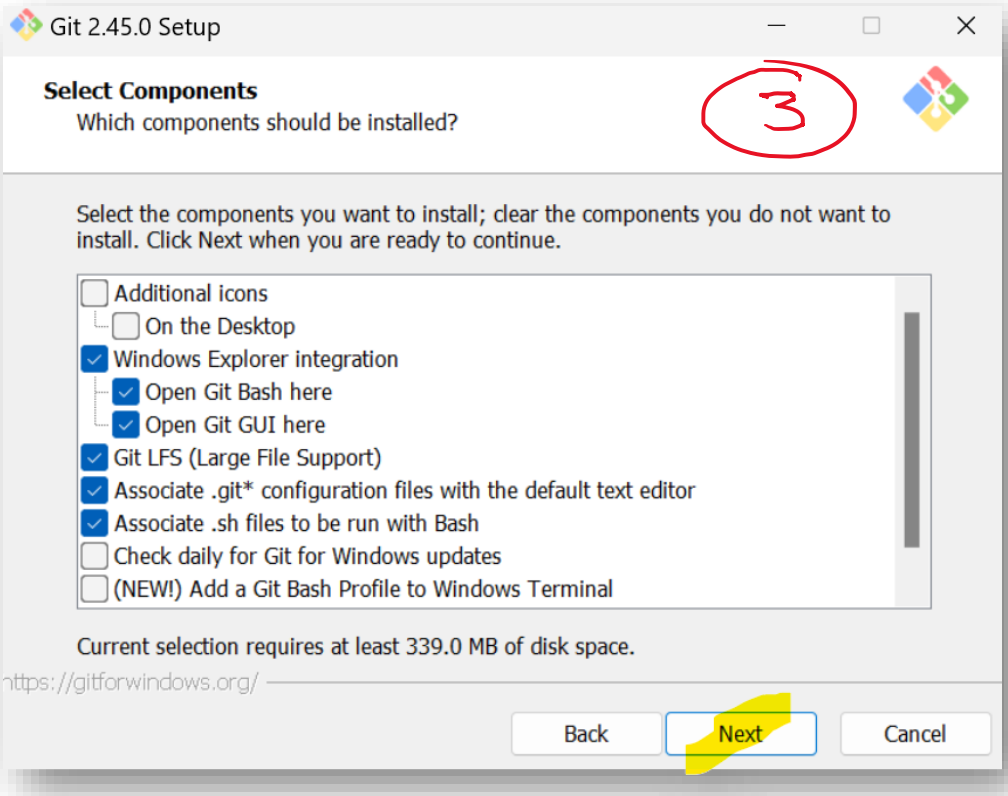


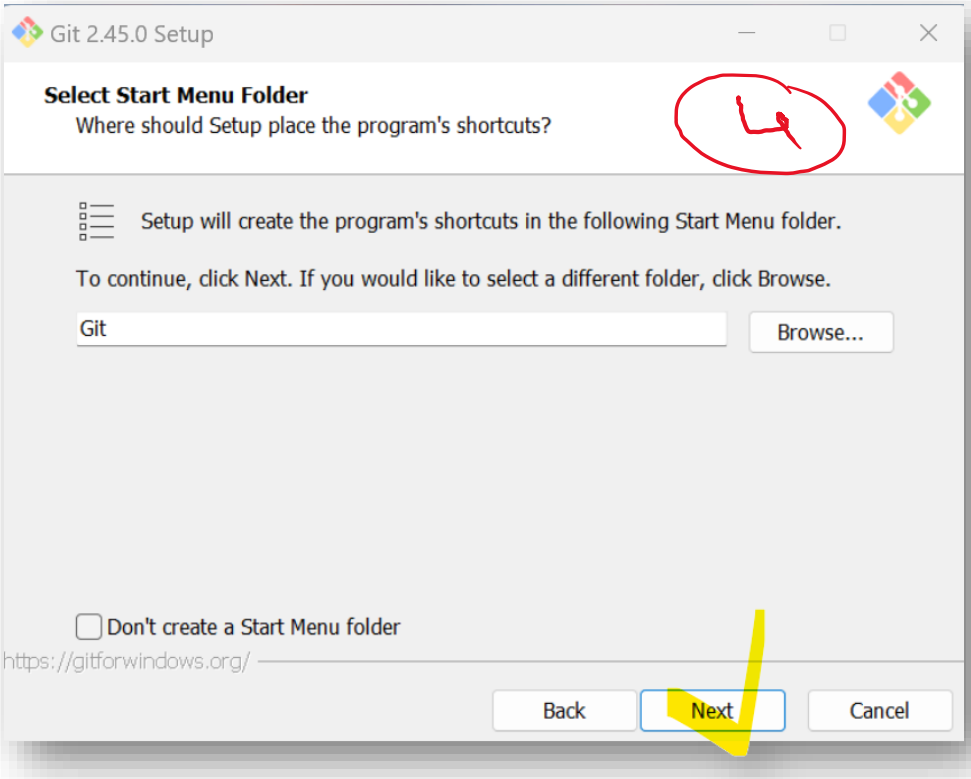
Installer File (Git-xxx.exe file)

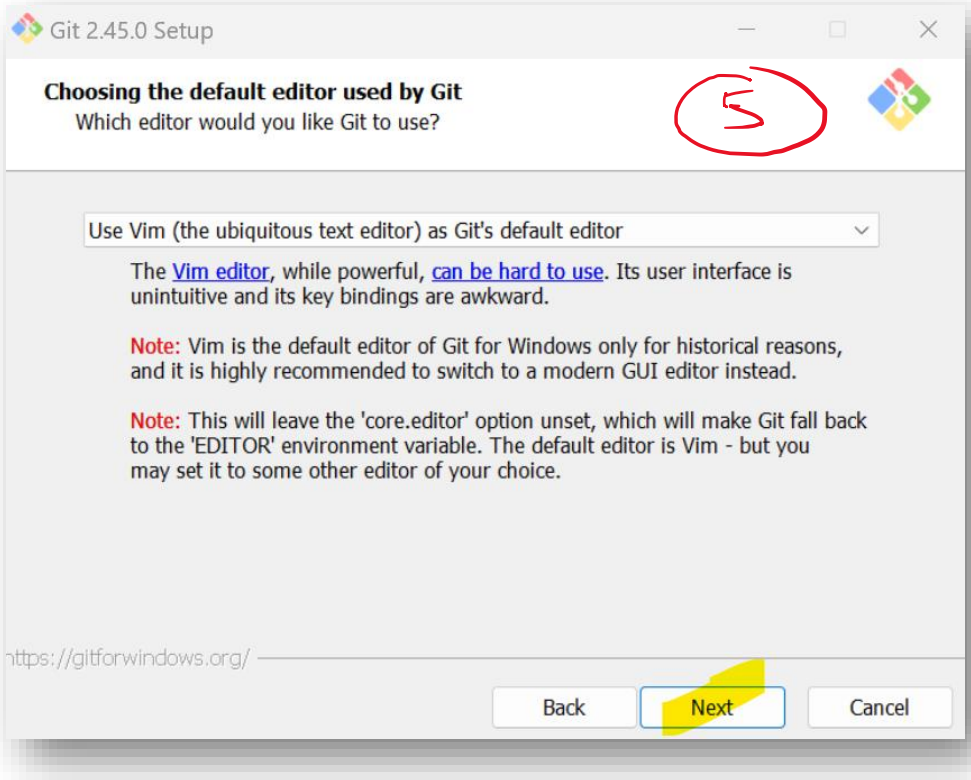


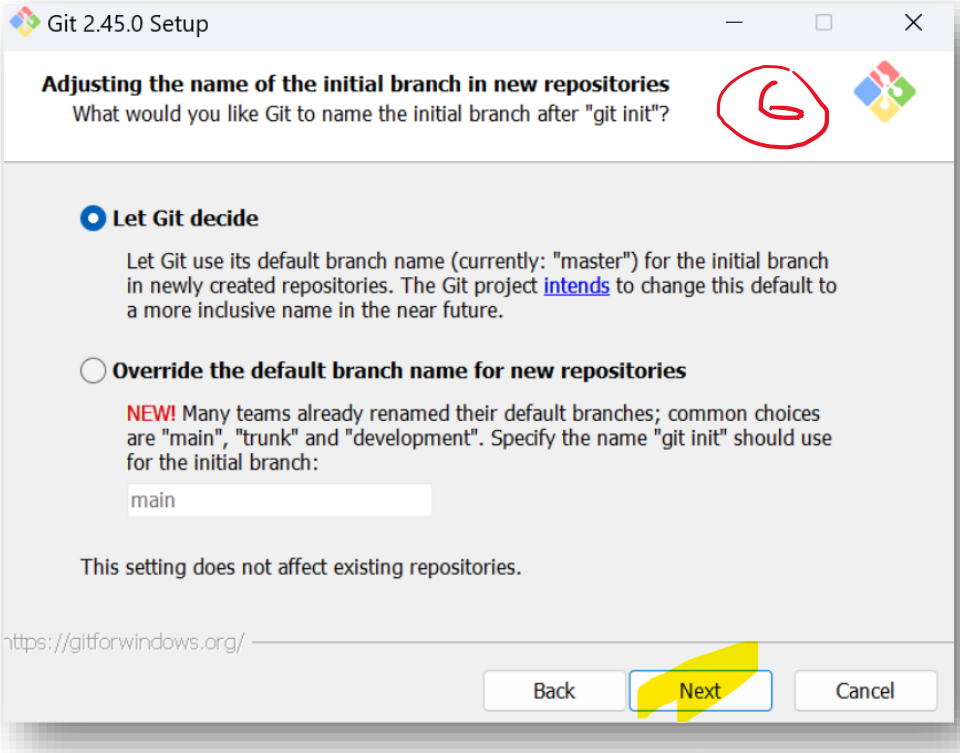


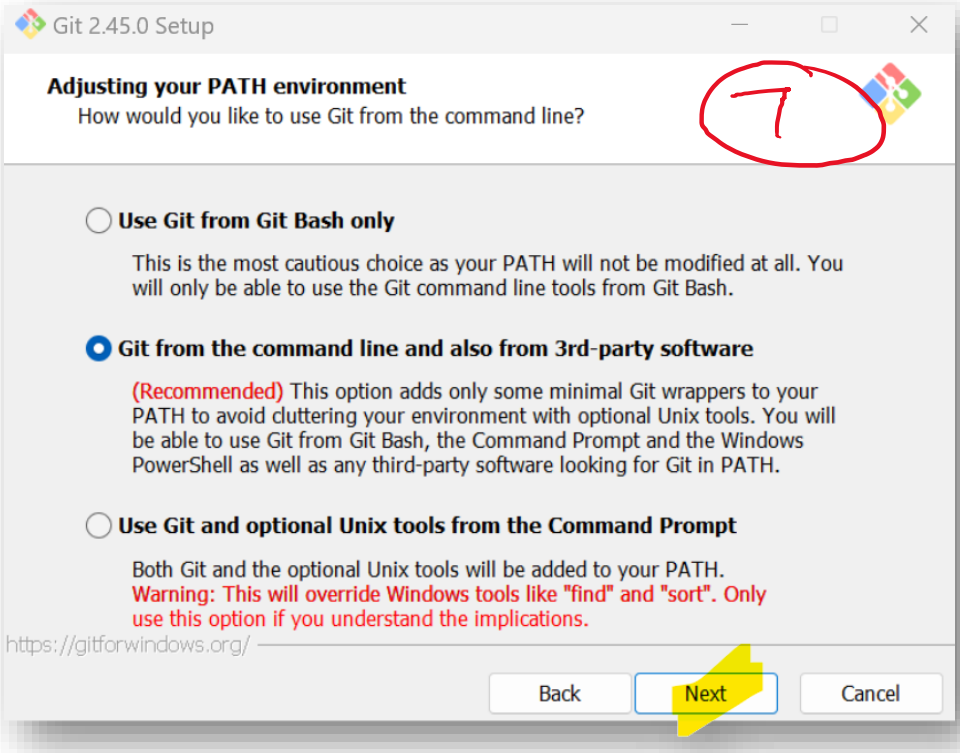

Git 2.45.0 Setup
Select Destination Location
Where should Git be installed?
2
Setup will install Git into the following folder.
To continue, click Next. If you would like to select a different folder, click Browse.
C:\Program Files\Git
Browse...
At least 339.0 MB of free disk space is required.
https://gitforwindows.org/
Back
Next
Cancel

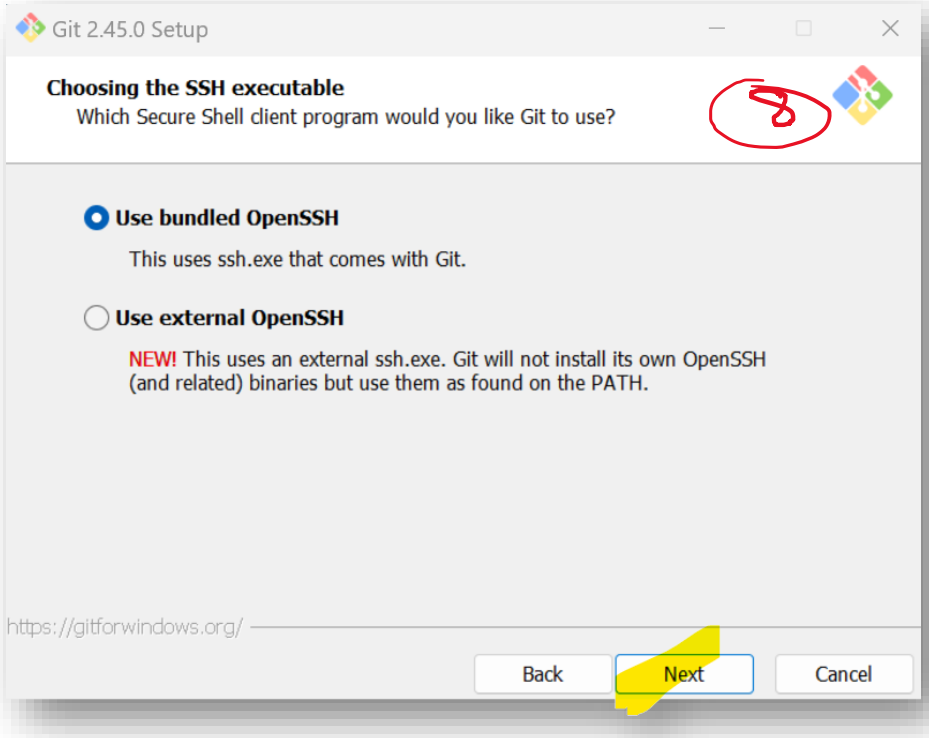

Git 2.45.0 Setup
Select Components
Which components should be installed?
3
Select the components you want to install; clear the components you do not want to install. Click Next when you are ready to continue.
Additional icons
On the Desktop
Windows Explorer integration
Open Git Bash here
Open Git GUI here
Git LFS (Large File Support)
Associate .git* configuration files with the default text editor
Associate .sh files to be run with Bash
Check daily for Git for Windows updates
(NEW!) Add a Git Bash Profile to Windows Terminal
Current selection requires at least 339.0 MB of disk space.
https://gitforwindows.org/
Back
Next
Cancel

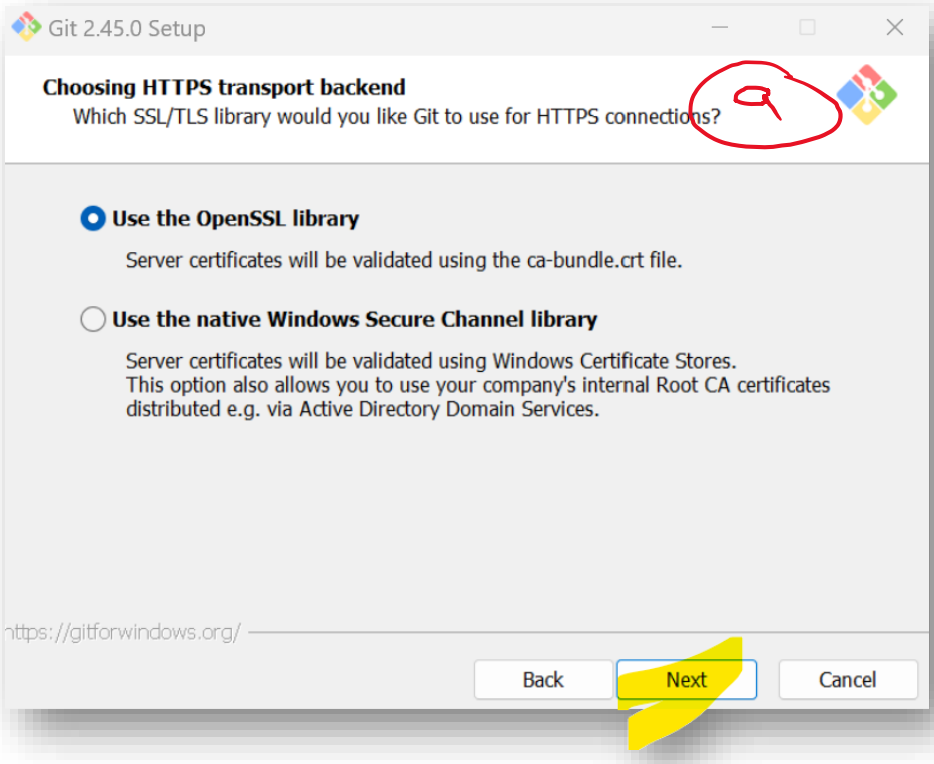

Git 2.45.0 Setup
Select Start Menu Folder
Where should Setup place the program's shortcuts?
4
Setup will create the program's shortcuts in the following Start Menu folder.
To continue, click Next. If you would like to select a different folder, click Browse.
Git
Browse...
Don't create a Start Menu folder
https://gitforwindows.org/
Back
Next
Cancel

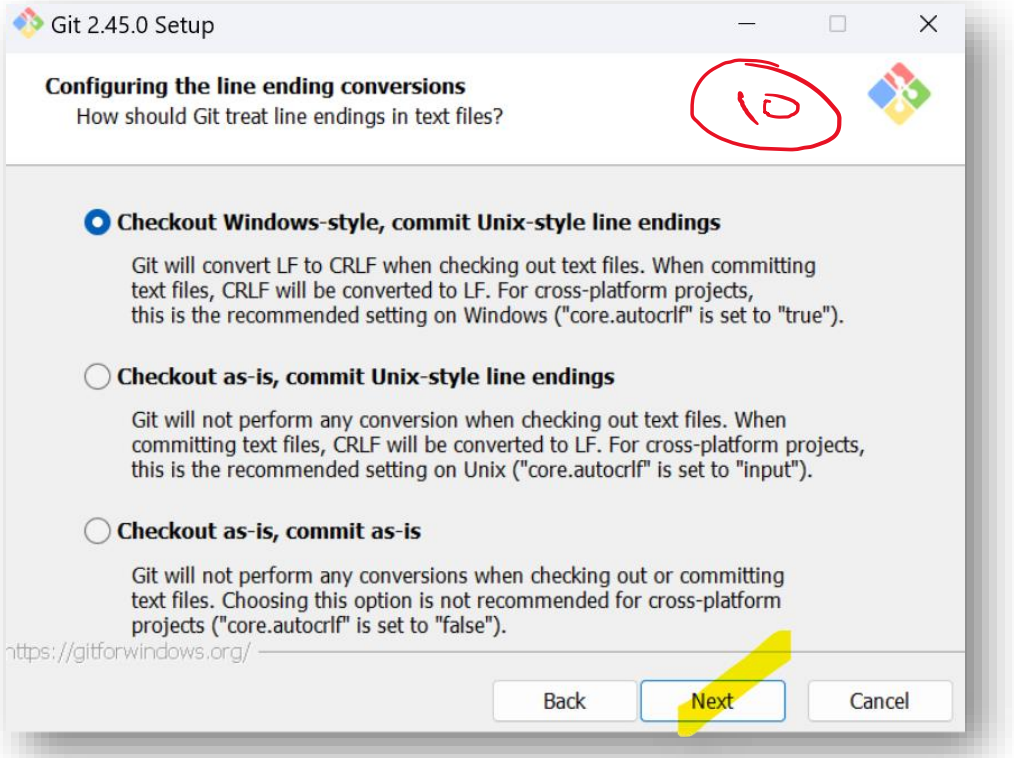

Git 2.45.0 Setup
Choosing the default editor used by Git
Which editor would you like Git to use?
5
Use Vim (the ubiquitous text editor) as Git's default editor
The Vim editor, while powerful, can be hard to use. Its user interface is unintuitive and its key bindings are awkward.
Note: Vim is the default editor of Git for Windows only for historical reasons, and it is highly recommended to switch to a modern GUI editor instead.
Note: This will leave the 'core.editor' option unset, which will make Git fall back to the 'EDITOR' environment variable. The default editor is Vim - but you may set it to some other editor of your choice.
https://gitforwindows.org/
Back
Next
Cancel

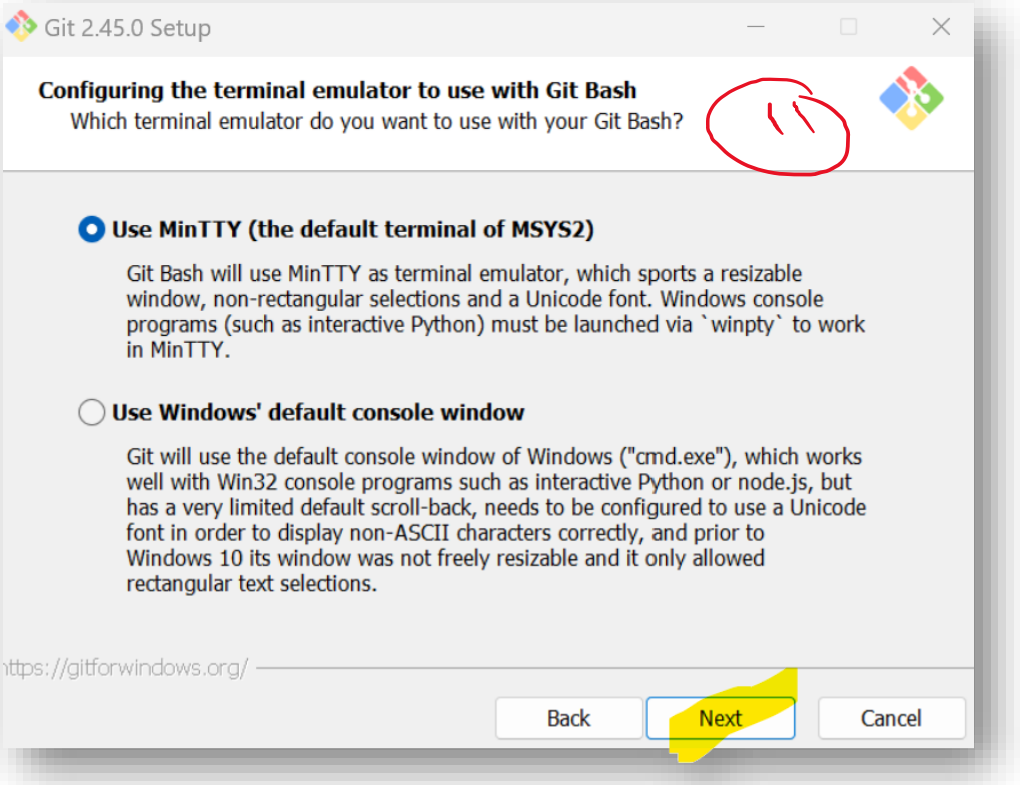

Git 2.45.0 Setup
Adjusting the name of the initial branch in new repositories
What would you like Git to name the initial branch after "git init"?
6
Let Git decide
Let Git use its default branch name (currently: "master") for the initial branch in newly created repositories. The Git project intends to change this default to a more inclusive name in the near future.
Override the default branch name for new repositories
NEW! Many teams already renamed their default branches; common choices are "main", "trunk" and "development". Specify the name "git init" should use for the initial branch:
main
This setting does not affect existing repositories.
https://gitforwindows.org/
Back
Next
Cancel

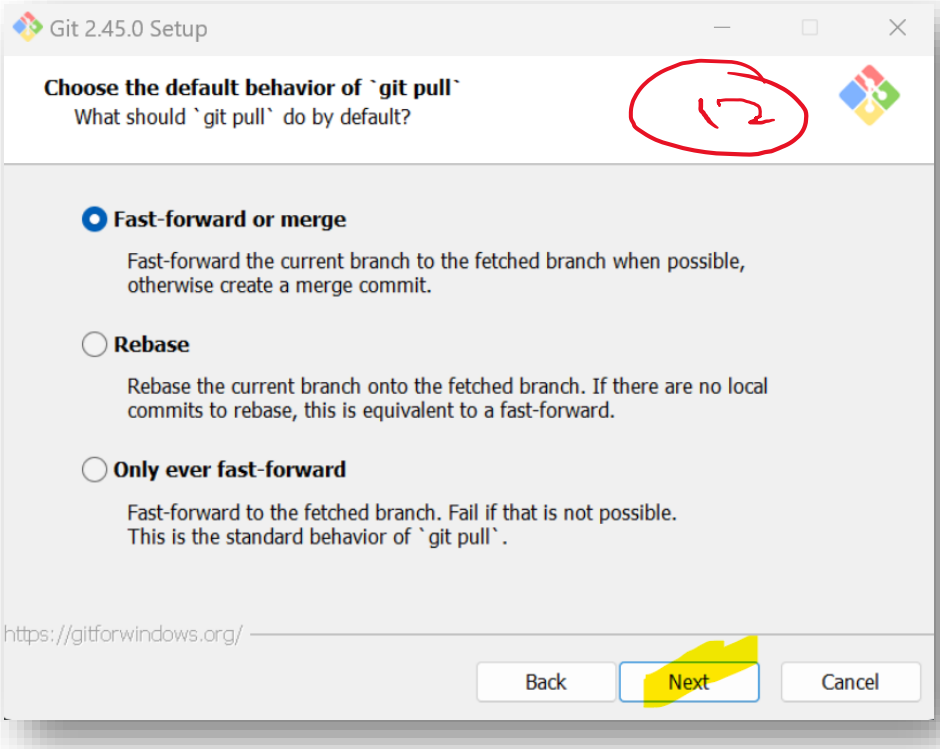

Git 2.45.0 Setup
Adjusting your PATH environment
How would you like to use Git from the command line?
7
Use Git from Git Bash only
This is the most cautious choice as your PATH will not be modified at all. You will only be able to use the Git command line tools from Git Bash.
Git from the command line and also from 3rd-party software
(Recommended) This option adds only some minimal Git wrappers to your PATH to avoid cluttering your environment with optional Unix tools. You will be able to use Git from Git Bash, the Command Prompt and the Windows PowerShell as well as any third-party software looking for Git in PATH.
Use Git and optional Unix tools from the Command Prompt
Both Git and the optional Unix tools will be added to your PATH.
Warning: This will override Windows tools like "find" and "sort". Only use this option if you understand the implications.
https://gitforwindows.org/
Back
Next
Cancel

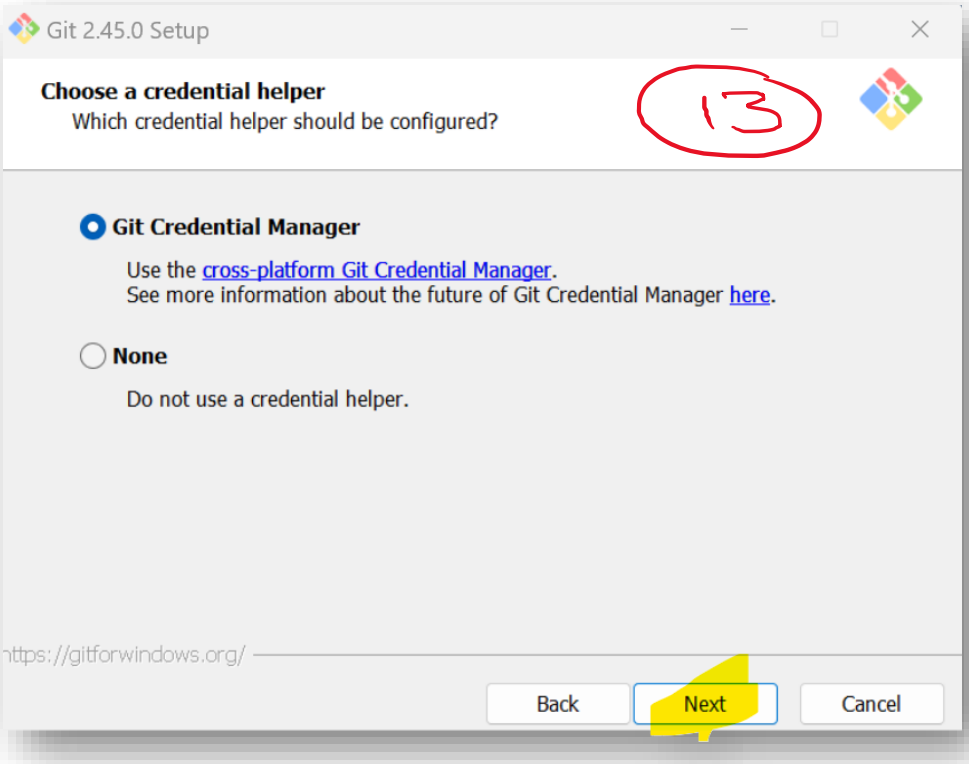

Git 2.45.0 Setup
Choosing the SSH executable
Which Secure Shell client program would you like Git to use?
Use bundled OpenSSH
This uses ssh.exe that comes with Git.
Use external OpenSSH
NEW! This uses an external ssh.exe. Git will not install its own OpenSSH (and related) binaries but use them as found on the PATH.
https://gitforwindows.org/
Back
Next
Cancel

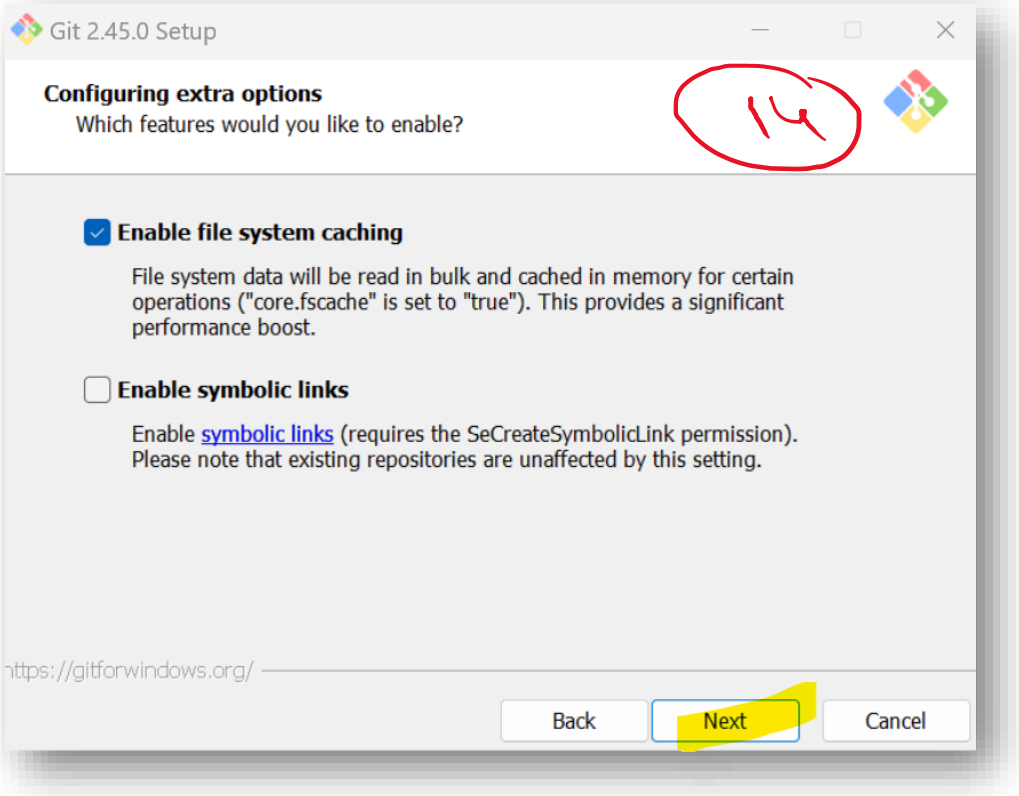

Git 2.45.0 Setup
Choosing HTTPS transport backend
Which SSL/TLS library would you like Git to use for HTTPS connections?
Use the OpenSSL library
Server certificates will be validated using the ca-bundle.crt file.
Use the native Windows Secure Channel library
Server certificates will be validated using Windows Certificate Stores.
This option also allows you to use your company's internal Root CA certificates distributed e.g. via Active Directory Domain Services.
https://gitforwindows.org/
Back
Next
Cancel

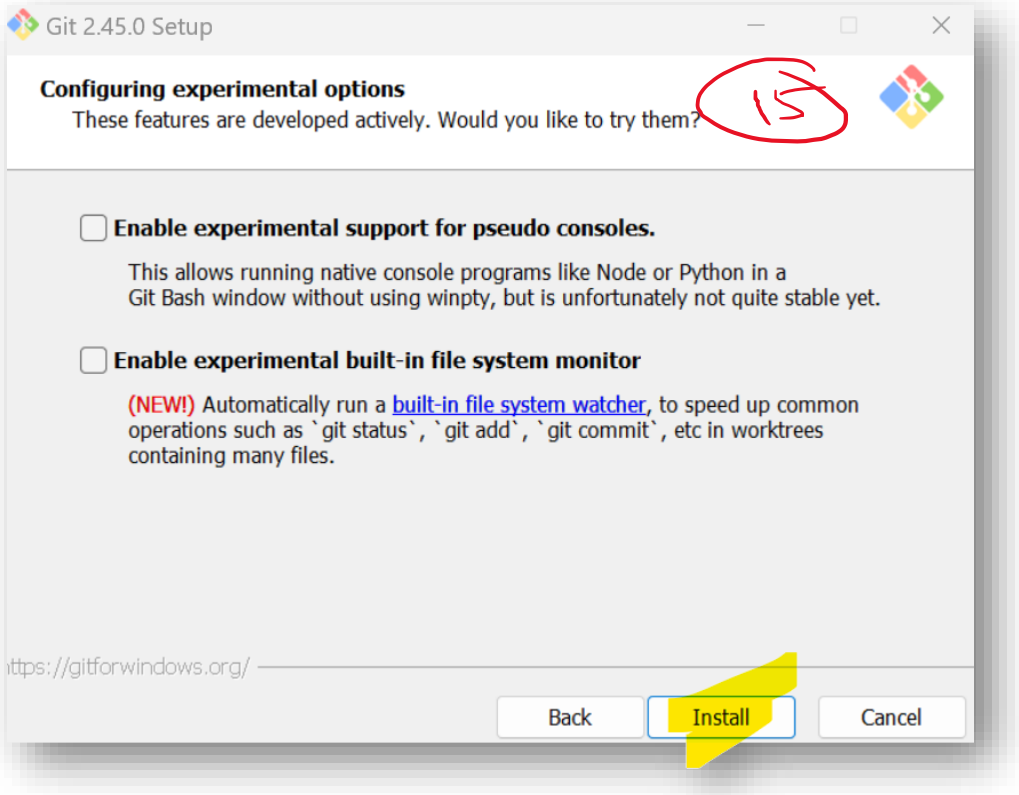

Git 2.45.0 Setup
Configuring the line ending conversions
How should Git treat line endings in text files?
Checkout Windows-style, commit Unix-style line endings
Git will convert LF to CRLF when checking out text files. When committing text files, CRLF will be converted to LF. For cross-platform projects, this is the recommended setting on Windows ("core.autocrlf" is set to "true").
Checkout as-is, commit Unix-style line endings
Git will not perform any conversion when checking out text files. When committing text files, CRLF will be converted to LF. For cross-platform projects, this is the recommended setting on Unix ("core.autocrlf" is set to "input").
Checkout as-is, commit as-is
Git will not perform any conversions when checking out or committing text files. Choosing this option is not recommended for cross-platform projects ("core.autocrlf" is set to "false").
https://gitforwindows.org/
Back
Next
Cancel

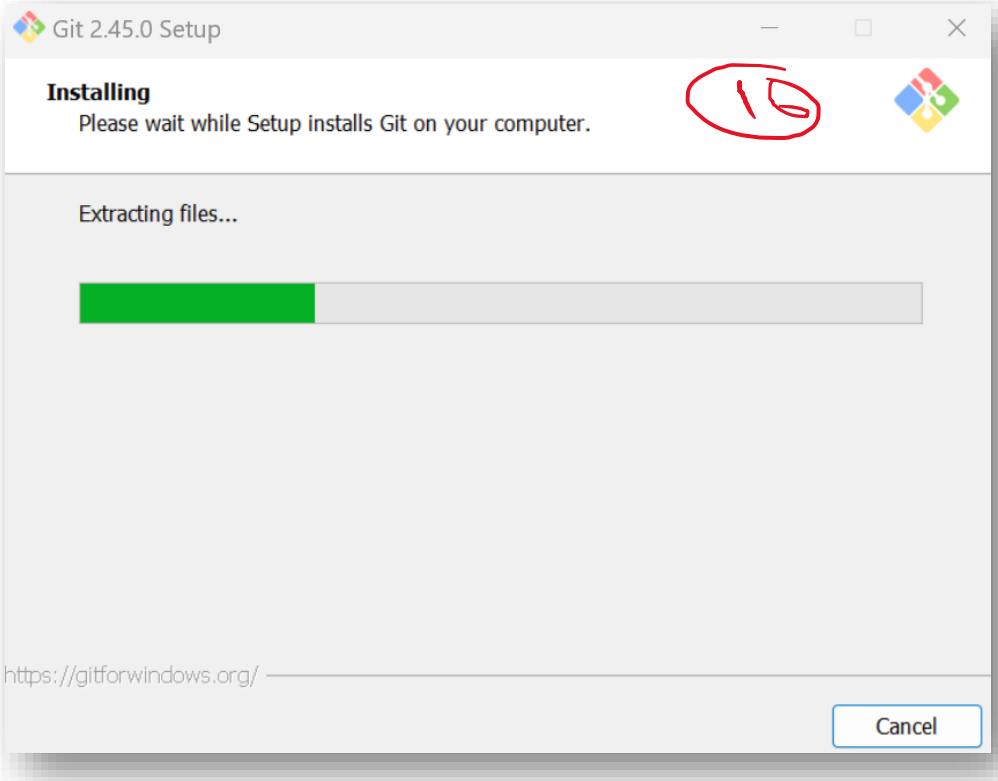

Git 2.45.0 Setup
Configuring the terminal emulator to use with Git Bash
Which terminal emulator do you want to use with your Git Bash?
Use MinTTY (the default terminal of MSYS2)
Git Bash will use MinTTY as terminal emulator, which sports a resizable window, non-rectangular selections and a Unicode font. Windows console programs (such as interactive Python) must be launched via `winpty` to work in MinTTY.
Use Windows' default console window
Git will use the default console window of Windows ("cmd.exe"), which works well with Win32 console programs such as interactive Python or node.js, but has a very limited default scroll-back, needs to be configured to use a Unicode font in order to display non-ASCII characters correctly, and prior to Windows 10 its window was not freely resizable and it only allowed rectangular text selections.
https://gitforwindows.org/
Back
Next
Cancel

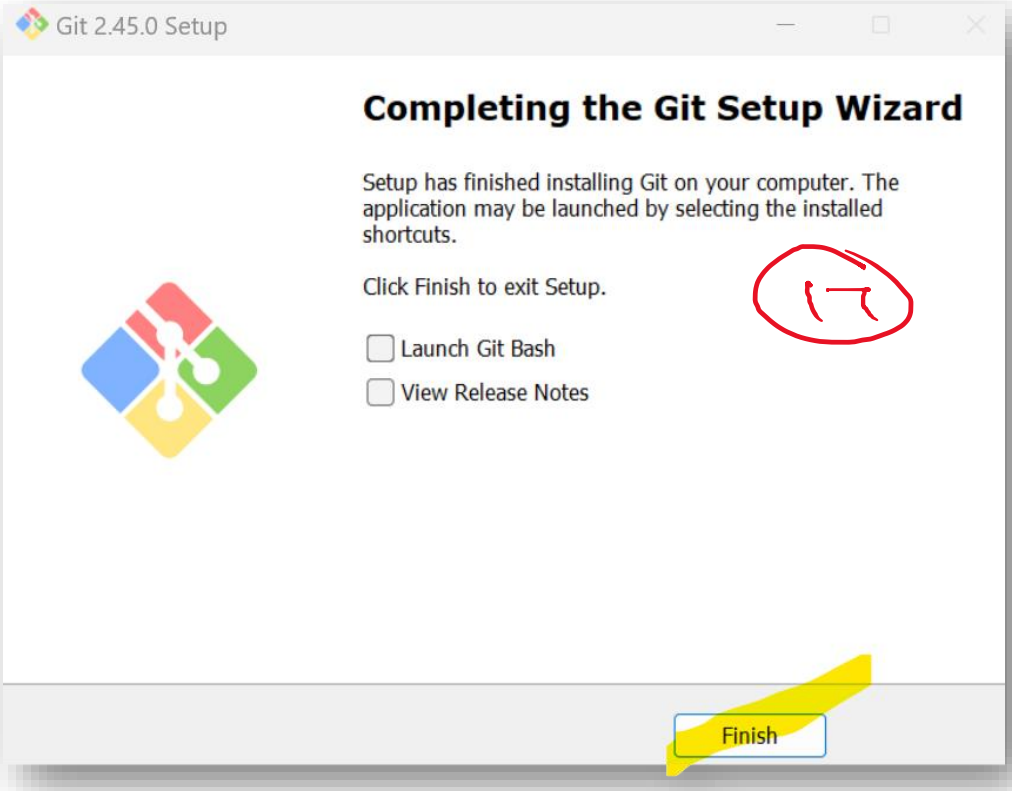

Git 2.45.0 Setup
Choose the default behavior of `git pull`
What should `git pull` do by default?
12
Fast-forward or merge
Fast-forward the current branch to the fetched branch when possible, otherwise create a merge commit.
Rebase
Rebase the current branch onto the fetched branch. If there are no local commits to rebase, this is equivalent to a fast-forward.
Only ever fast-forward
Fast-forward to the fetched branch. Fail if that is not possible. This is the standard behavior of `git pull`.
https://gitforwindows.org/
Back
Next
Cancel


Git 2.45.0 Setup
Choose a credential helper
Which credential helper should be configured?
13
Git Credential Manager
Use the cross-platform Git Credential Manager.
See more information about the future of Git Credential Manager here.
None
Do not use a credential helper.
https://gitforwindows.org/
Back
Next
Cancel

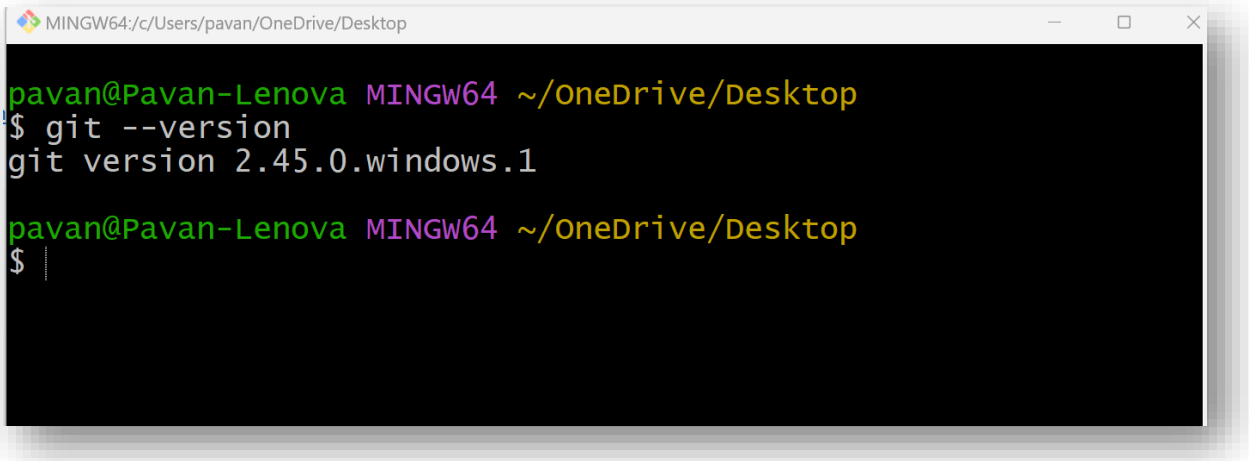

Git 2.45.0 Setup
Configuring extra options
Which features would you like to enable?
14
Enable file system caching
File system data will be read in bulk and cached in memory for certain operations ("core.fscache" is set to "true"). This provides a significant performance boost.
Enable symbolic links
Enable symbolic links (requires the SeCreateSymbolicLink permission). Please note that existing repositories are unaffected by this setting.
https://gitforwindows.org/
Back
Next
Cancel


Git 2.45.0 Setup
Configuring experimental options
These features are developed actively. Would you like to try them?
15
Enable experimental support for pseudo consoles.
This allows running native console programs like Node or Python in a Git Bash window without using winpty, but is unfortunately not quite stable yet.
Enable experimental built-in file system monitor
(NEW!) Automatically run a built-in file system watcher, to speed up common operations such as `git status`, `git add`, `git commit`, etc in worktrees containing many files.
https://gitforwindows.org/
Back
Install
Cancel


Git 2.45.0 Setup
Installing
Please wait while Setup installs Git on your computer.
Extracting files...
https://gitforwindows.org/
Cancel


Git 2.45.0 Setup
Completing the Git Setup Wizard
Setup has finished installing Git on your computer. The application may be launched by selecting the installed shortcuts.
Click Finish to exit Setup.
Launch Git Bash
View Release Notes
Finish

Git command prompt



Git & GitHub

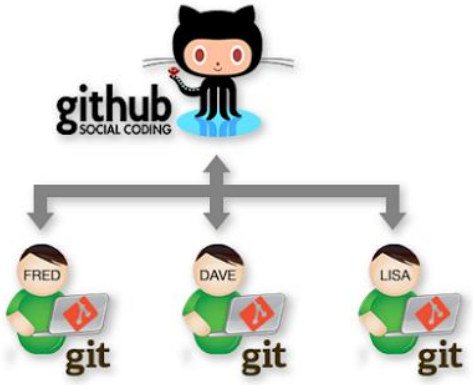
Git

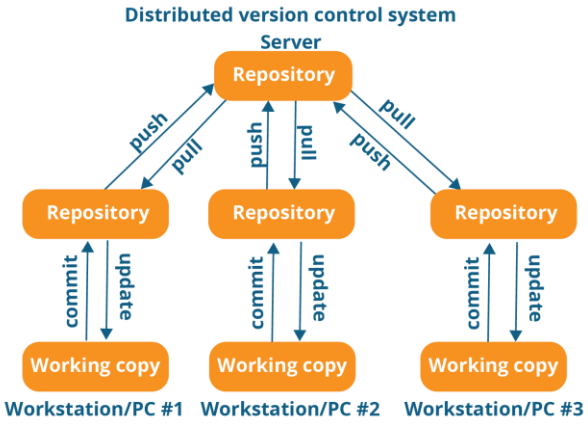
Git is a distributed version control system that helps developers track changes in code and collaborate on projects.

GitHub

GitHub is a cloud-based platform for hosting Git repositories, enabling collaboration and version control.

Git & GitHub Workflow





Steps for Git workflow:

1. **Initialize a repository.**
2. **Configure user information.**
3. **Stage files for a commit.**
4. **Commit changes locally.**
5. **Link to a remote repository.**
6. **Push changes to the remote repository.**
7. **Pull latest changed from remote repository.**
8. **Delete local repository (Optional)**

1. Create a New Local Git Repository

To initialize a new Git repository in a directory, navigate to the desired folder and execute the following command:

```
git init
```

2. Configure User Information (One-Time Setup)

Set your username and email for Git. This is typically a one-time setup for identifying the author of the commits. Replace "your name" and "your email" with your actual details:

```
git config --global user.name "your name"
git config --global user.email "your email"
```

3. Add Files or Folders to the Staging Area

To stage changes for the next commit, use the git add command. You can stage specific files, all files, or files matching certain patterns:

- **Add all files and folders to staging:**

```
git add -A
```

- **Add a specific file:**

```
git add filename
```

- **Add all files with a specific extension (e.g., Java files):**

```
git add *.py
```

- **Add all files within a folder:**

```
git add foldername
git add .
```

4. Commit Changes to the Local Repository

After staging the files, commit them to your local repository with a descriptive commit message:

```
git commit -m "commit message"
```

5. Connect Local Repository to a Remote Repository (One-Time Setup)

Link your local repository to a remote repository using the git remote add command. Replace the example URL with the actual remote repository URL:

```
git remote add origin " https://github.com/pavanoltraining/myproject.git"
```

6. Push Changes to the Remote Repository

To upload your committed changes to the remote repository, use the git push command. Specify the remote name (origin) and the branch name (master for the main branch):

```
git push origin master
```

Note: Need to pass token which is generated in GitHub.

7. Pull the latest Changes from the Remote Repository

To get latest changes from the remote repository to local repository , you need to use git pull command.

```
git pull origin master
```

8. To Delete local repository

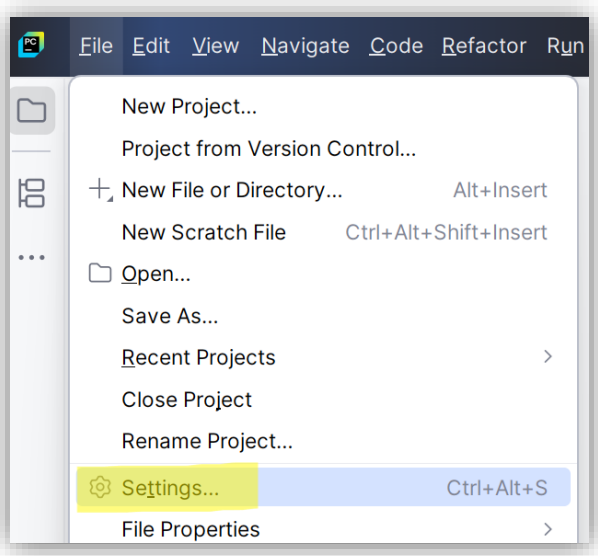
```
rm -rf
```

Git Advanced Tutorials:

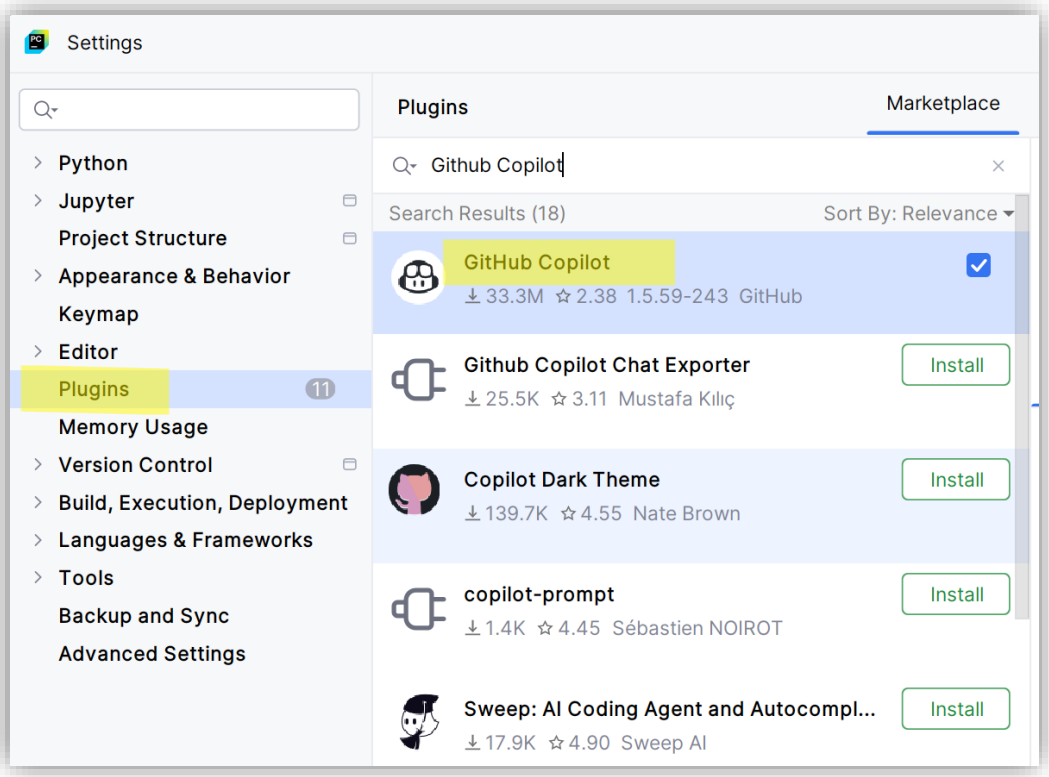
https://tinyurl.com/2ktjbscr

GitHub Copilot Installation on PyCharm IDE

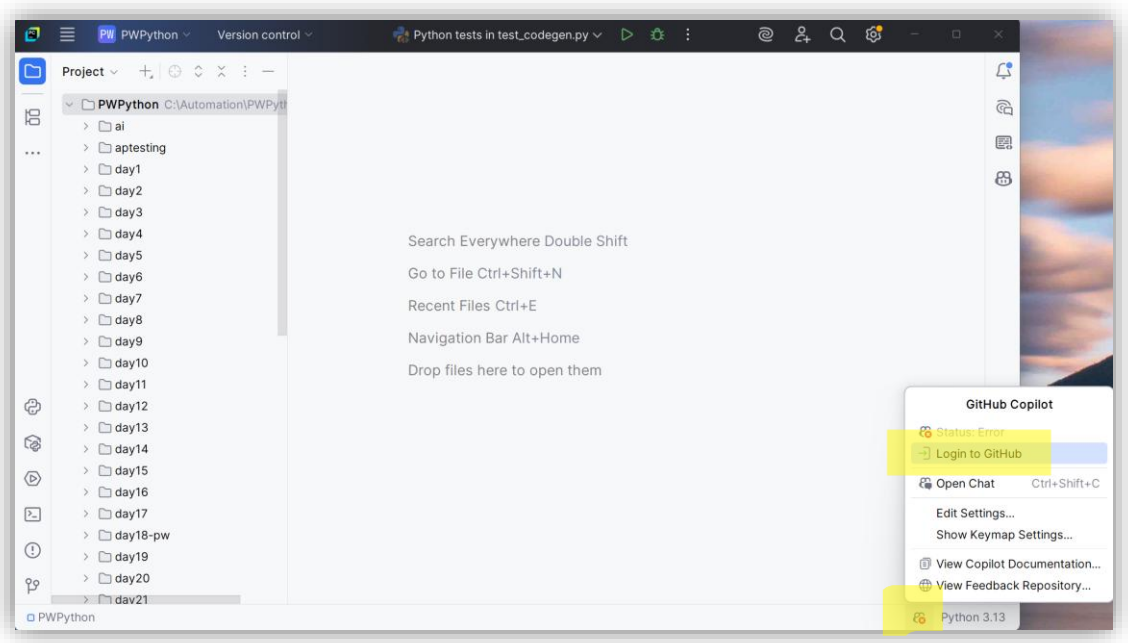
- Navigate to **File → Settings**

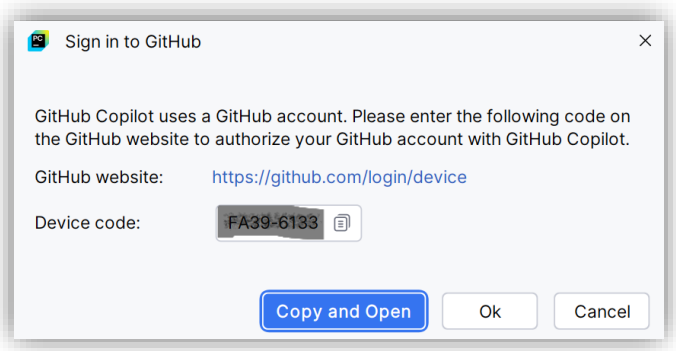


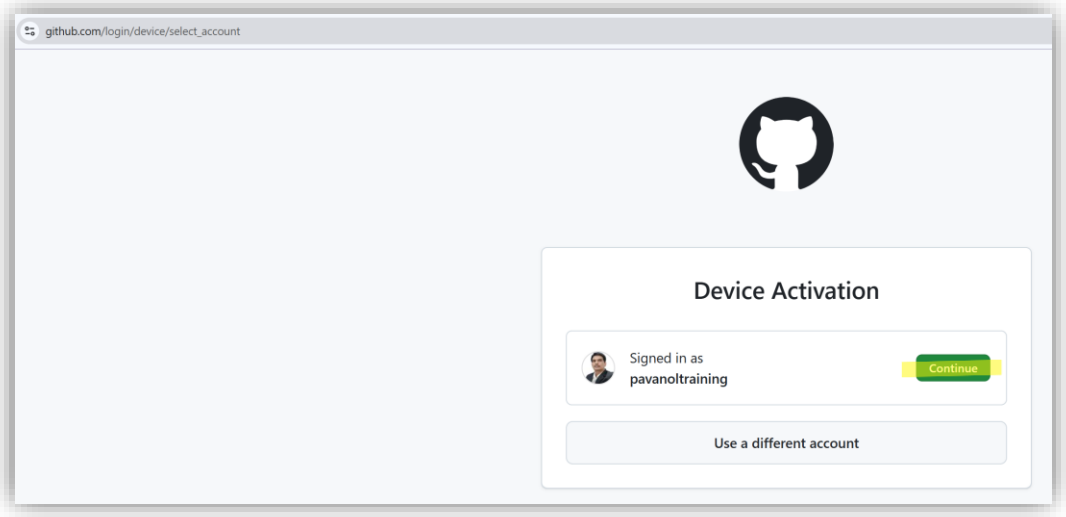
- Go to **Plugins** → Search for GitHub Copilot in the Market place then install.
- Restart PyCharm IDE.

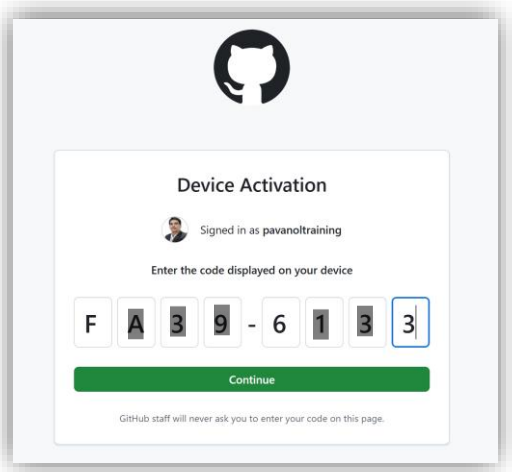


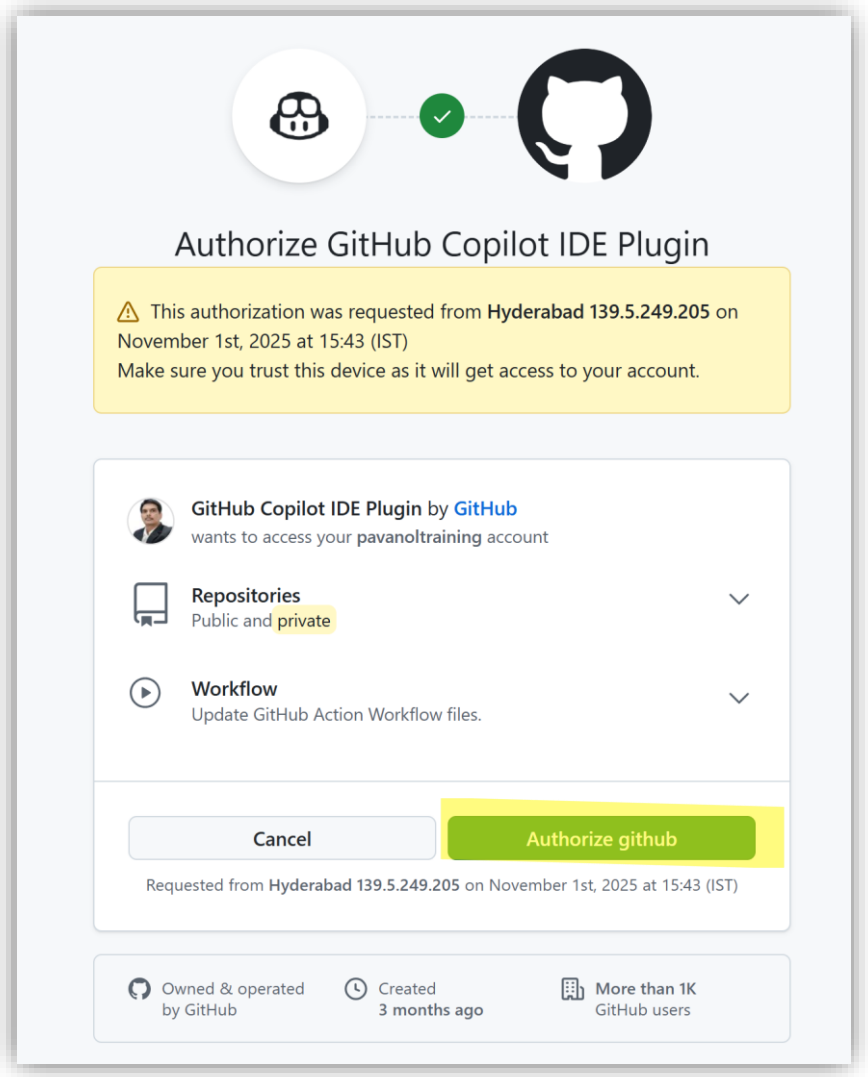
Login-to GitHub Copilot from PyCharm IDE

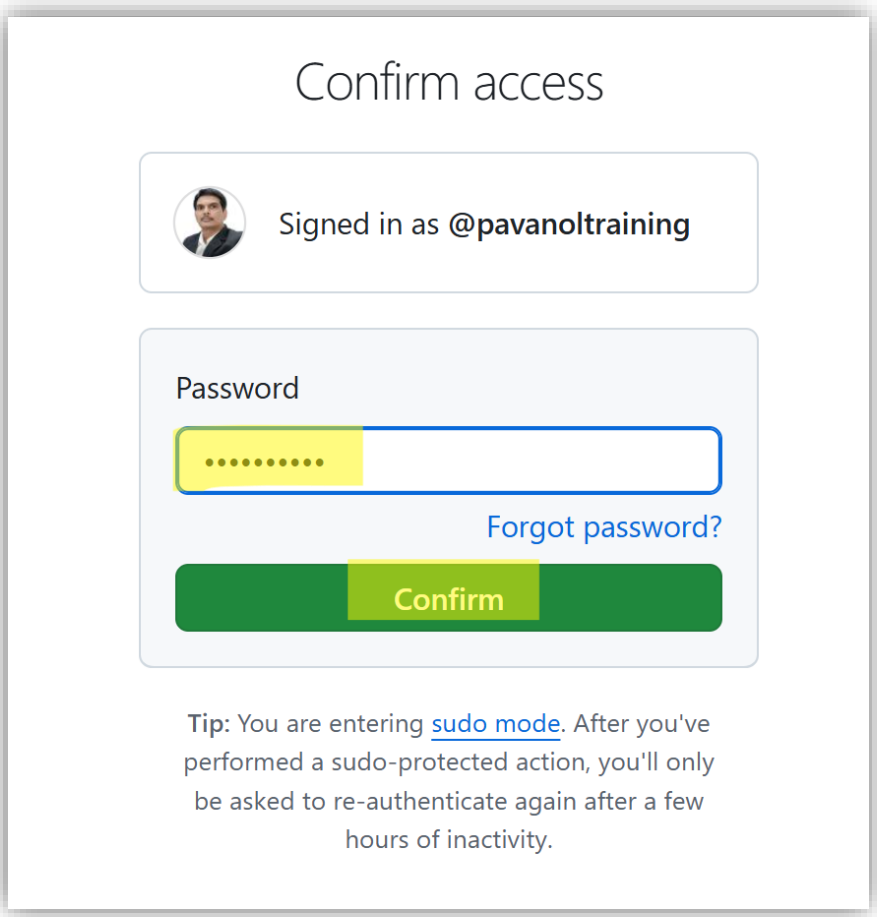


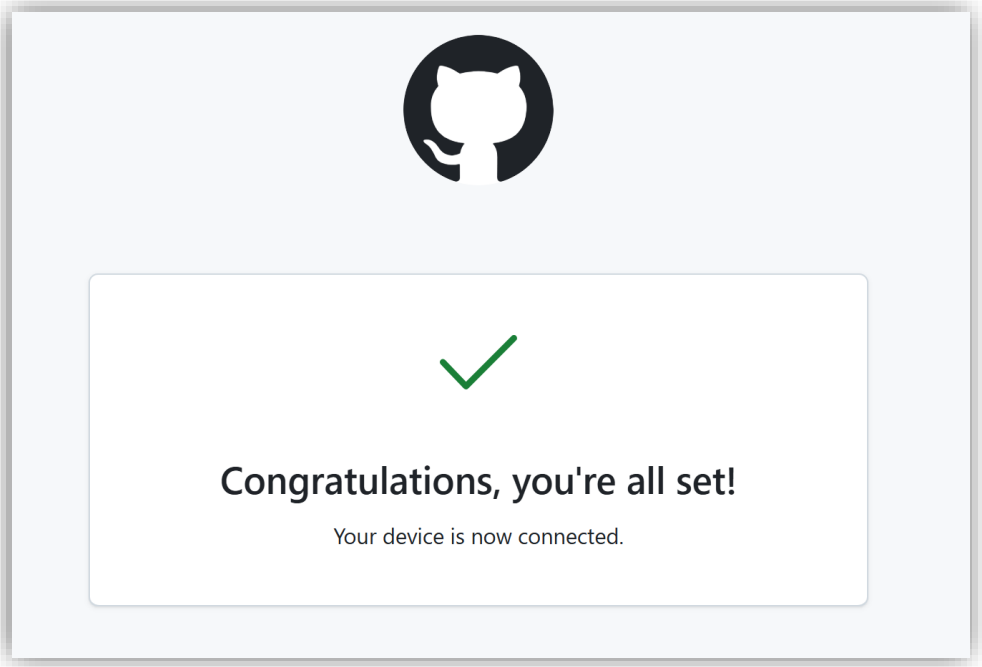


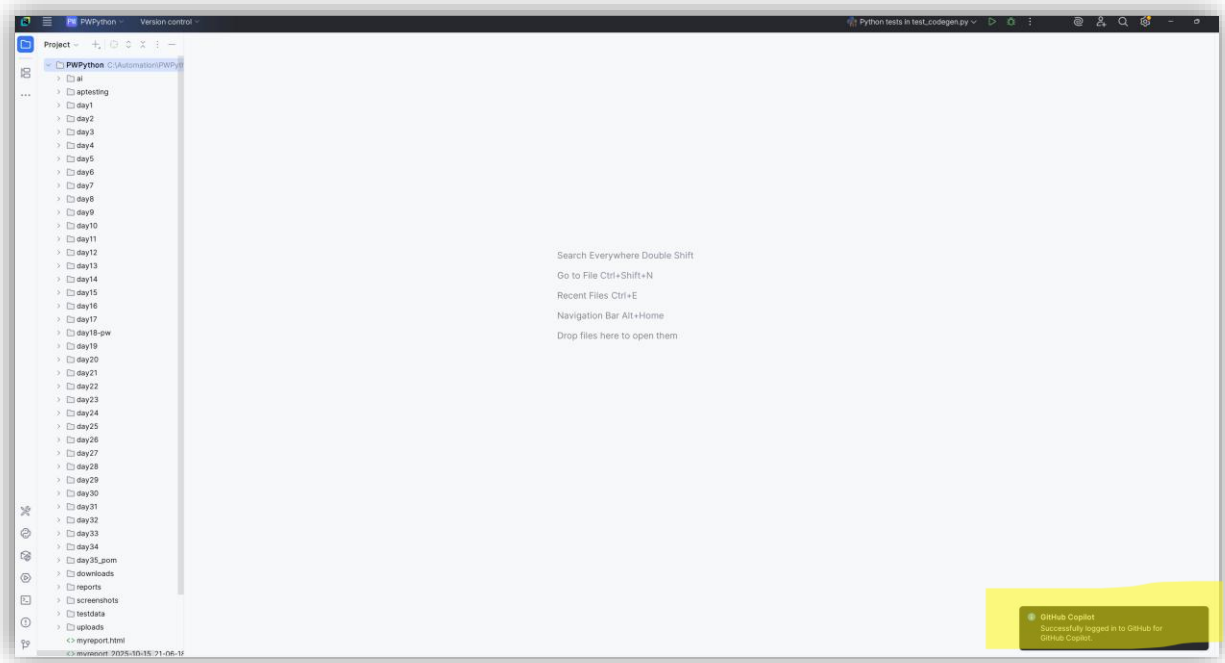


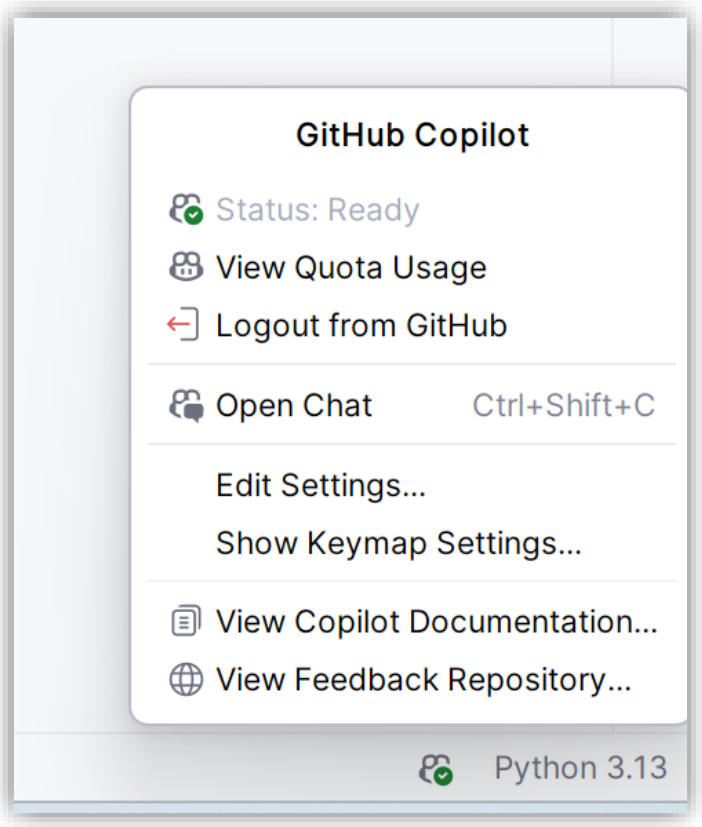

Device Activation
Signed in as pavanoltraining
Enter the code displayed on your device
F A 3 9 - 6 1 3 3
Continue
GitHub staff will never ask you to enter your code on this page.


Authorize GitHub Copilot IDE Plugin
This authorization was requested from Hyderabad 139.5.249.205 on November 1st, 2025 at 15:43 (IST)
Make sure you trust this device as it will get access to your account.
GitHub Copilot IDE Plugin by GitHub
wants to access your pavanoltraining account
Repositories
Public and private
Workflow
Update GitHub Action Workflow files.
Cancel
Authorize github
Requested from Hyderabad 139.5.249.205 on November 1st, 2025 at 15:43 (IST)
Owned & operated by GitHub
Created 3 months ago
More than 1K GitHub users


Confirm access
Signed in as @pavanoltraining
Password
Forgot password?
Confirm
Tip: You are entering sudo mode. After you've performed a sudo-protected action, you'll only be asked to re-authenticate again after a few hours of inactivity.


Congratulations, you're all set!
Your device is now connected.


Search Everywhere Double Shift
Go to File Ctrl+Shift+N
Recent Files Ctrl+E
Navigation Bar Alt+Home
Drop files here to open them
GitHub Copilot
Successfully logged in to GitHub for GitHub Copilot


GitHub Copilot
Status: Ready
View Quota Usage
Logout from GitHub
Open Chat Ctrl+Shift+C
Edit Settings...
Show Keymap Settings...
View Copilot Documentation...
View Feedback Repository...
Python 3.13

How to use GitHub Copilot in PyCharm IDE

1. **Code Completion**
2. **Q&A Comment Blocks**
3. **"Explain This" Feature**
4. **"Simplify This" Feature**
5. **Chat Window**
6. **Generate Documentation**
7. **Unpin Chat Window**

1. Code Completion

GitHub Copilot automatically suggests the next lines of code as you type.

Example 1 – Creating a Playwright Test
Start typing the following code:

```
from playwright.sync_api import Playwright, sync_playwright


def test_login_functionality(playwright: Playwright):
    browser = playwright.chromium.launch(headless=False)
    context = browser.new_context()
    page = context.new_page()
    # Navigate to login page
```

Copilot will automatically suggest the next steps, such as typing the username, entering the password, clicking the login button, and verifying the login success message.

2. Q&A Comment Blocks

You can ask Copilot questions by writing them inside comments. Copilot then generates relevant code suggestions as answers.

Example 1 – Question-style Comment

```
# How do I launch a browser in Playwright using Python?
```

Copilot may respond with:

```
from playwright.sync_api import sync_playwright


with sync_playwright() as p:
```

```
    browser = p.chromium.launch(headless=False)
    page = browser.new_page()
    page.goto("https://example.com")
    browser.close()
```

Example 2 – Another Q&A Comment

How to handle dropdowns in Playwright Python?

Copilot may respond with:

```
page.select_option("select#country", "India")
```

3. "Explain This" Feature

You can ask Copilot to explain a selected code block for better understanding.

Example:

```
from playwright.sync_api import Page, expect


def test_verifyPageUrl(page: Page):
    page.goto("http://www.automationpractice.pl/index.php")  # Navigate to URL


    myurl = page.url
    print("Url of the application:", myurl)


    expect(page).to_have_url("http://www.automationpractice.pl/index.php")  # Validate URL
```

Select the code and use **Copilot → Explain This** to get a plain-language explanation of what each part does.

4. "Fix This" Feature

Copilot can detect and correct code errors when you select a buggy block and choose **Fix This**.

Example:

```
from playwright.sync_api import Page, expect


def test_verifyPageUrl(page: Page):
    page.goto("http://www.automationpractice.pl/index.php")
```

```
myurl = page.url()  # Incorrect usage
print("Url of the application:", myurl)


expect(page).to_have_url("http://www.automationpractice.pl/index.php")
```

Copilot will automatically fix the issue by removing parentheses (since page.url is a property, not a function).

5. "Simplify This" Feature

Copilot can refactor or shorten your code while keeping its behavior the same.

Example – Before Simplification:

```
from playwright.sync_api import Page, expect


def test_verifyPageUrl(page: Page):
    page.goto("http://www.automationpractice.pl/index.php")
    myurl = page.url
    print("Url of the application:", myurl)
    expect(page).to_have_url("http://www.automationpractice.pl/index.php")
```

Select the code block → Right-click → **Copilot → Simplify This**

After Simplification:

```
from playwright.sync_api import Page, expect


def test_verify_page_url(page: Page):
    page.goto("http://www.automationpractice.pl/index.php")
    print("Url of the application:", page.url)
    expect(page).to_have_url("http://www.automationpractice.pl/index.php")
```

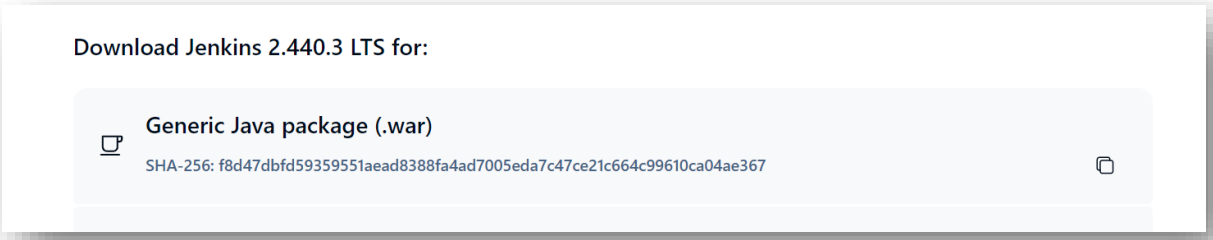
6. "Generate Documentation" Feature

This feature automatically creates documentation or docstrings for your selected code, explaining the function's purpose, parameters, and logic.

Jenkins Setup using war file

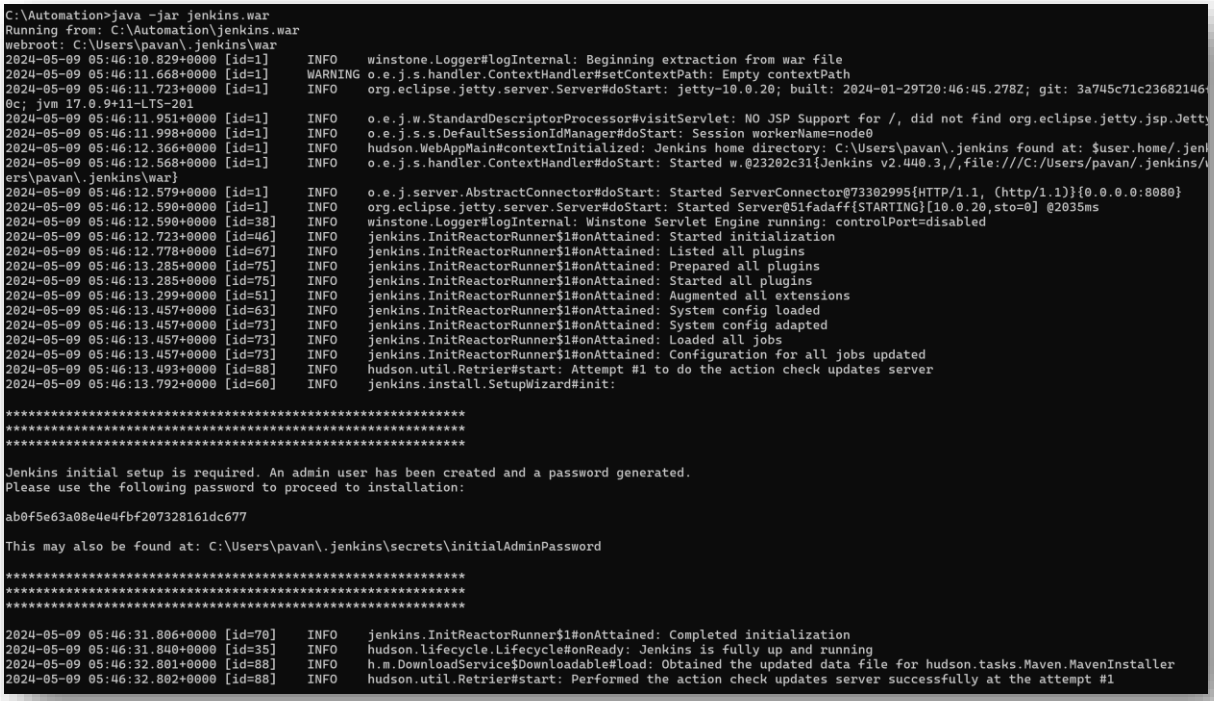
Step1: Download Jenkins (.war file)

https://www.jenkins.io/download/

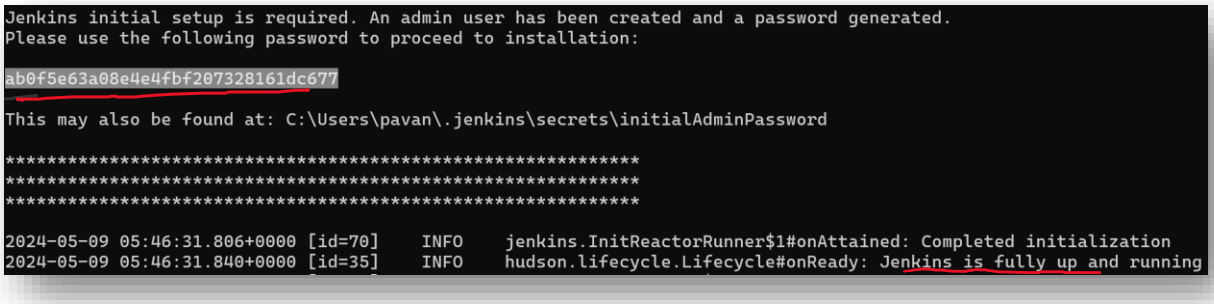


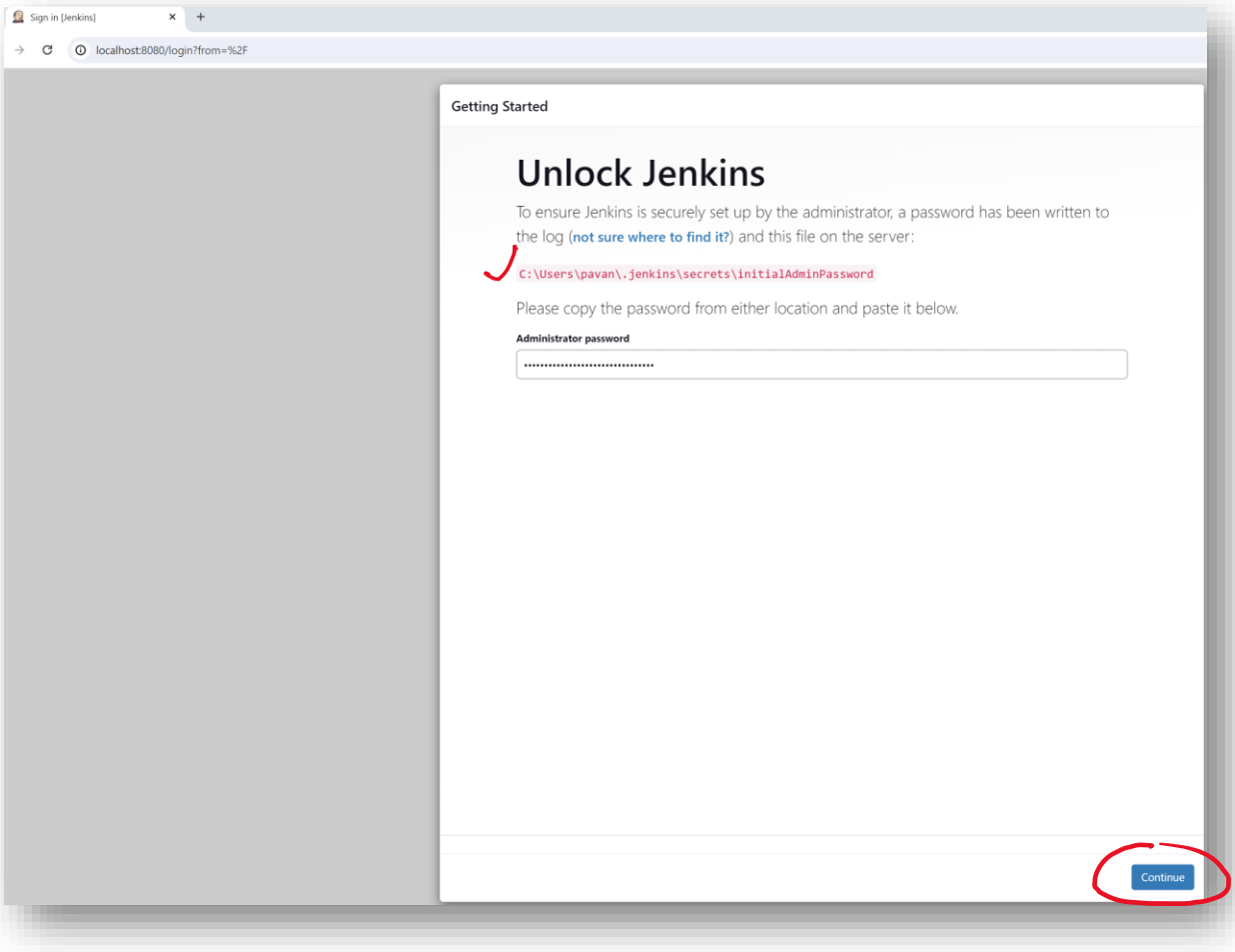
Step2: Run below command in the windows command prompt (Don't close command prompt until your work is done

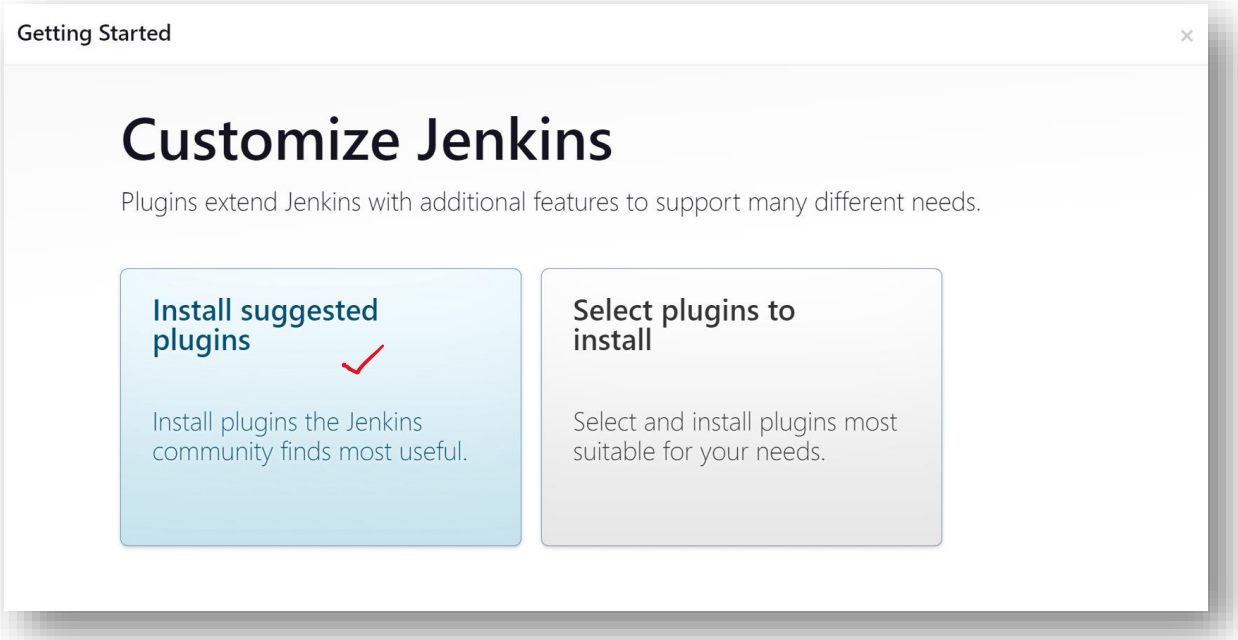
java -jar Jenkins.war

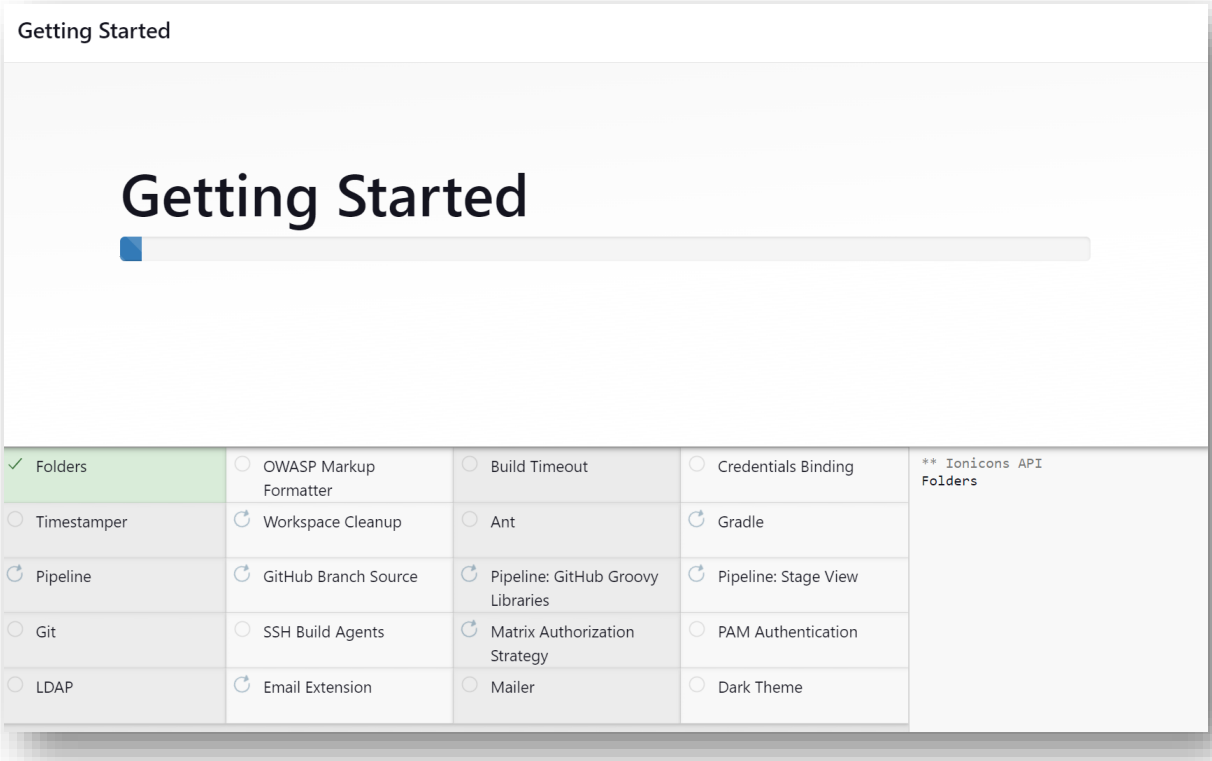


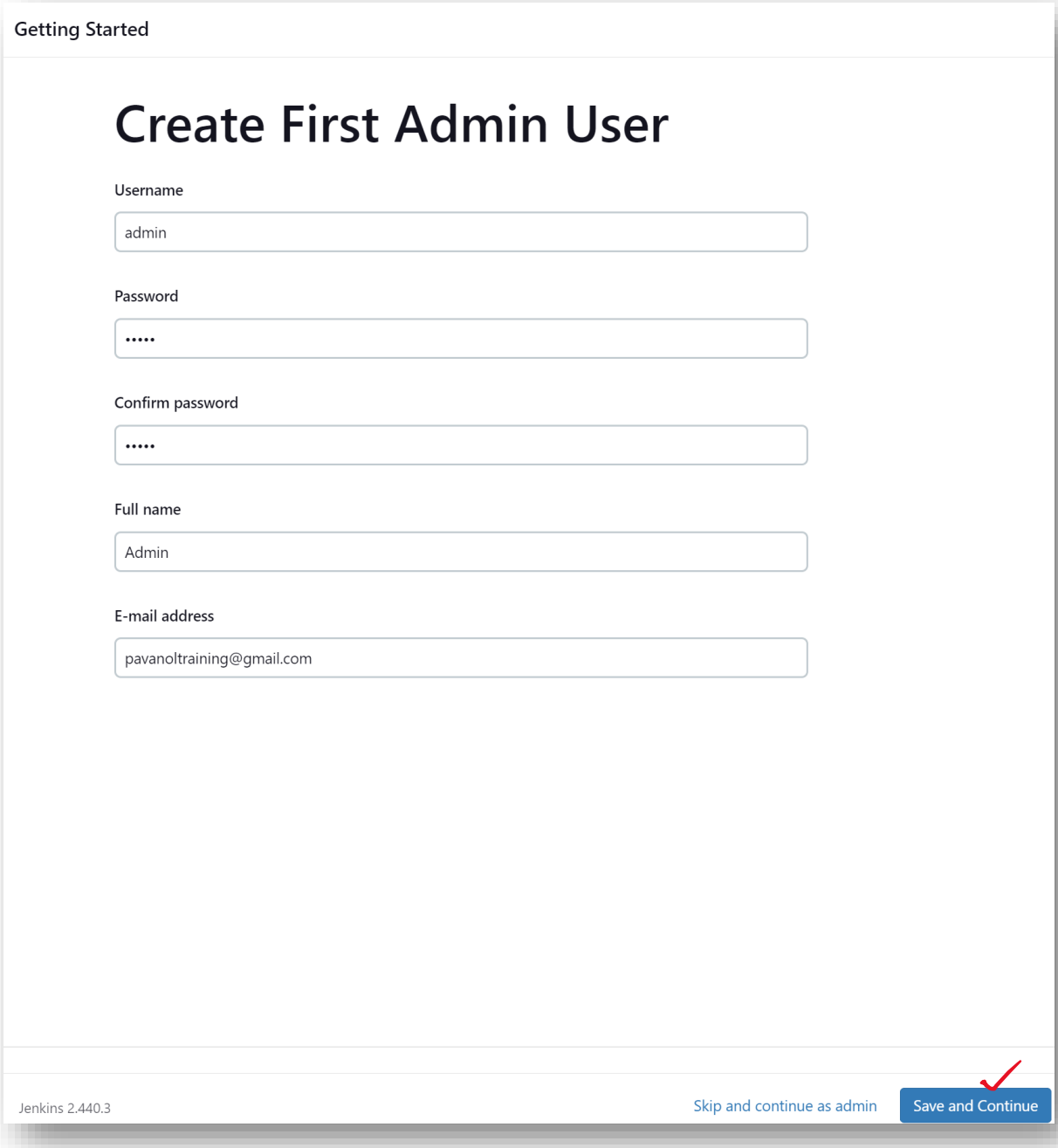
Copy admin password in the note pad (Later we use it).

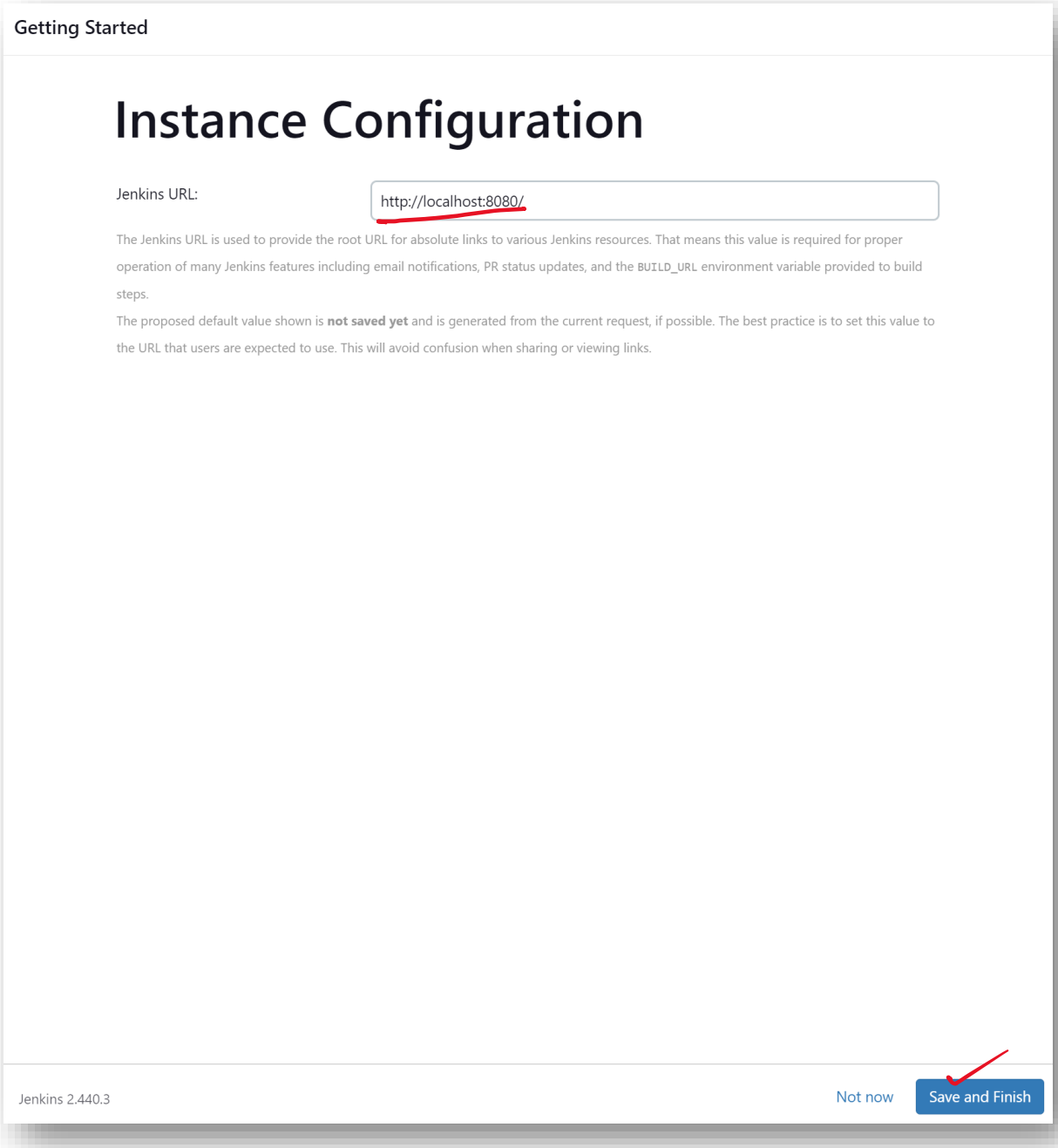


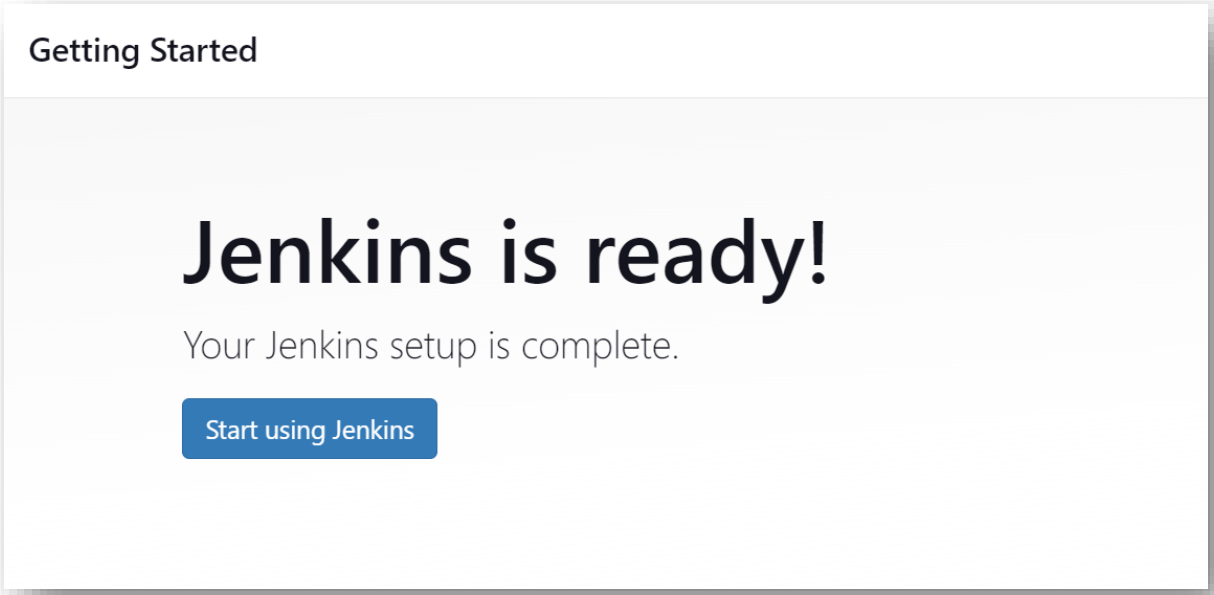

Sign in [Jenkins]
localhost:8080/login?from=%2F
Getting Started
Unlock Jenkins
To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:
C:\Users\pavan\.jenkins\secrets\initialAdminPassword
Please copy the password from either location and paste it below.
Administrator password
Continue


Getting Started
Customize Jenkins
Plugins extend Jenkins with additional features to support many different needs.
Install suggested plugins
Install plugins the Jenkins community finds most useful.
Select plugins to install
Select and install plugins most suitable for your needs.

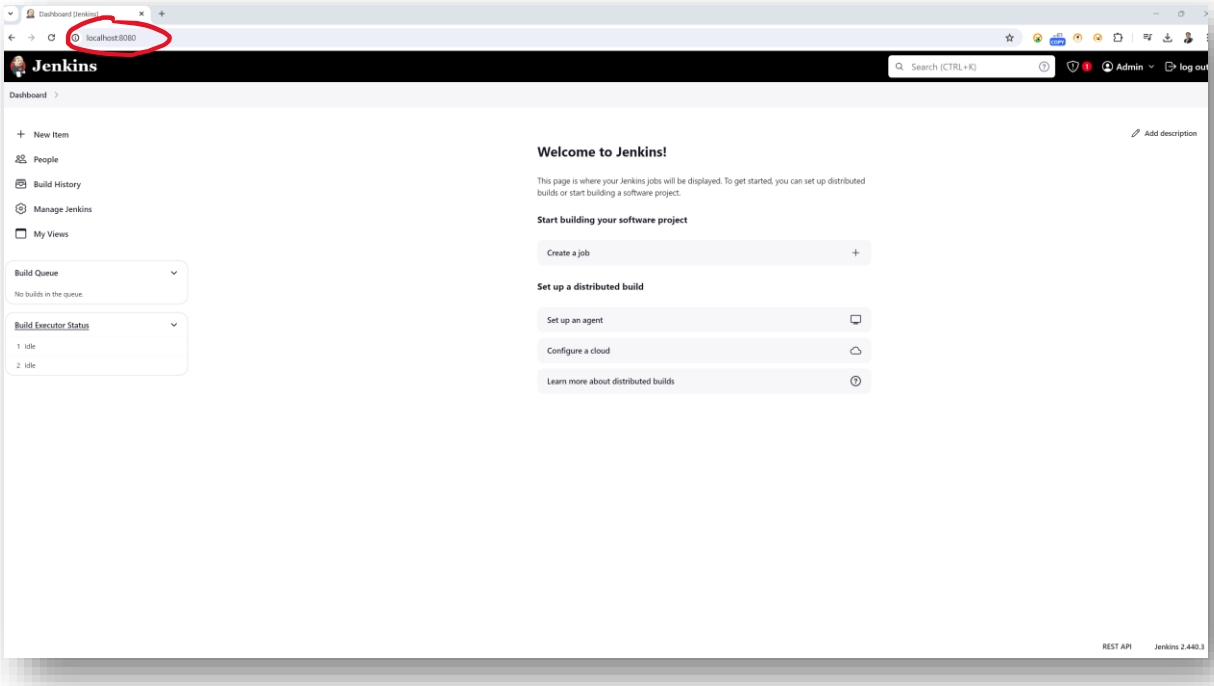

Getting Started
Getting Started
Folders
OWASP Markup Formatter
Build Timeout
Credentials Binding
** Ionicons API
Folders
Timestamper
Workspace Cleanup
Ant
Gradle
Pipeline
GitHub Branch Source
Pipeline: GitHub Groovy Libraries
Pipeline: Stage View
Git
SSH Build Agents
Matrix Authorization Strategy
PAM Authentication
LDAP
Email Extension
Mailer
Dark Theme


Getting Started
Create First Admin User
Username
admin
Password
Confirm password
Full name
Admin
E-mail address
pavanoltraining@gmail.com
Jenkins 2.440.3
Skip and continue as admin
Save and Continue


Getting Started
Instance Configuration
Jenkins URL:
http://localhost:8080/
The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the BUILD_URL environment variable provided to build steps.
The proposed default value shown is not saved yet and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.
Jenkins 2.440.3
Not now
Save and Finish



Here you can see Jenkins dashboard.



Playwright python Project Integration with Jenkins

How to Run a Local Playwright Project in Jenkins:

Step 1: Login to Jenkins

Open your Jenkins instance in a browser and log in using your credentials.

Step 2: Create a New Jenkins Job

- Click on **"New Item"**.
- Enter a name for your job.
- Select **"Freestyle project"**.
- Click **OK** to proceed.

Step 3: Configure the Job

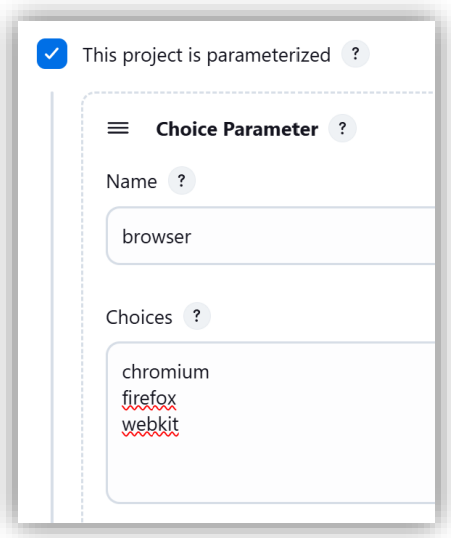
Enable Parameters

- Check the box **"This project is parameterized"**.
- Click **Add Parameter → Choice Parameter**.
    - **Name:** browser
    - **Choices:**

        chromium

        firefox

        webkit



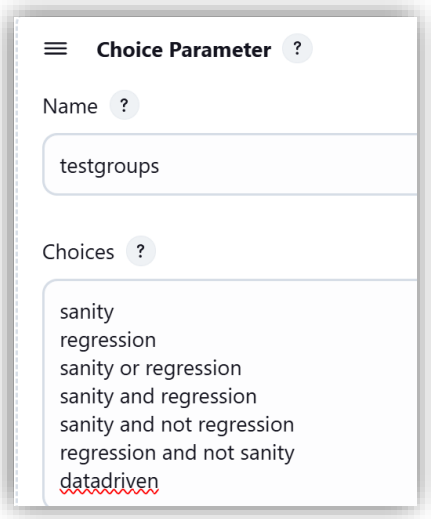
- Click **Add Parameter → Choice Parameter**.
    - **Name:** testgroups
    - **Choices:**

        sanity

        regression

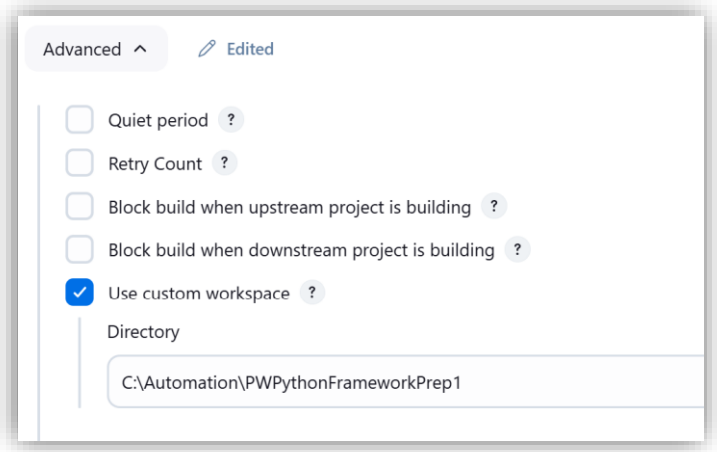
        sanity or regression

        sanity and regression



Set Custom Workspace

- Check **“Use custom workspace”**.
- Enter the full path to your local Playwright project directory.



Add Build Step

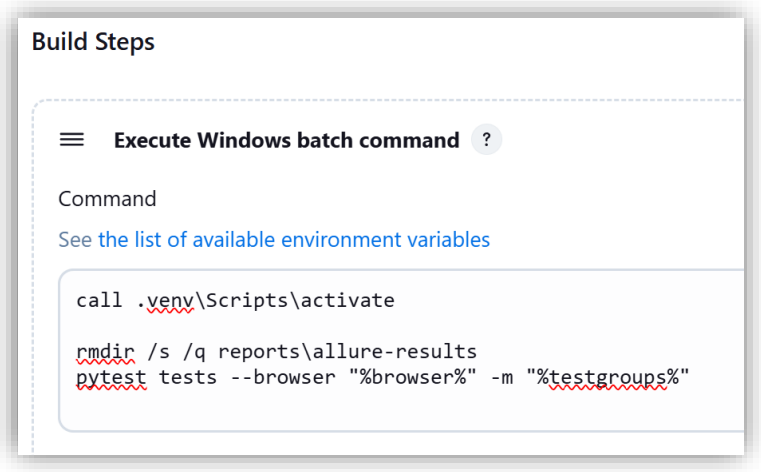
- Click on **“Add build step” → “Execute Windows batch command”** *(if on Windows)*.

- **Command:**

```
call .venv\Scripts\activate
rmdir /s /q reports\allure-results
pytest tests --browser "%browser%" -m "%testgroups%"
```

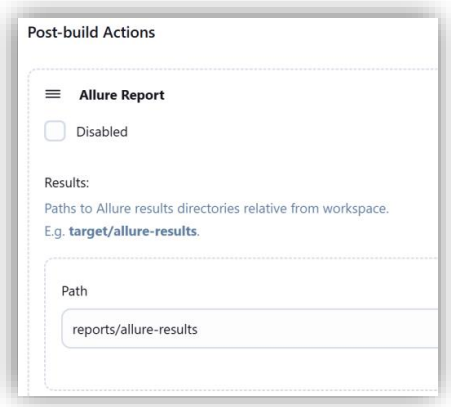
Note: If you're on **Mac/Linux**, select **"Execute shell"** and use:

```
call .venv\Scripts\activate
rmdir /s /q reports/allure-results
pytest tests\ --browser "$browser" -m "$testgroups"
```



Step 4: Post-build Actions (Allure Reports)

- Scroll to **"Post-build Actions"**.
- Select **"Allure Report"**.
- In **"Results path"**, enter:
- allure-results



Step 5: Save the Configuration

- Click **Apply** and then **Save** the job.

How to Run GitHub Playwright project in Jenkins

Step 1: Login to Jenkins

Open your Jenkins instance in a browser and log in using your credentials.

Step 2: Create a New Jenkins Job

- Click on **"New Item"**.
- Enter a name for your job.
- Select **"Freestyle project"**.
- Click **OK** to proceed.

Step 3: Configure the Job

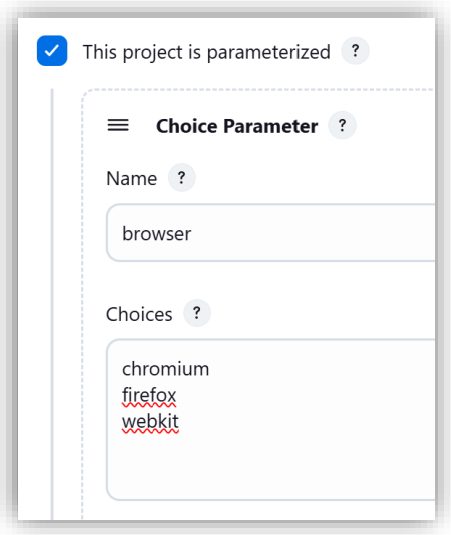
Enable Parameters

- Check the box **"This project is parameterized"**.
- Click **Add Parameter → Choice Parameter**.
    - **Name:** browser
    - **Choices:**

        chromium

        firefox

        webkit



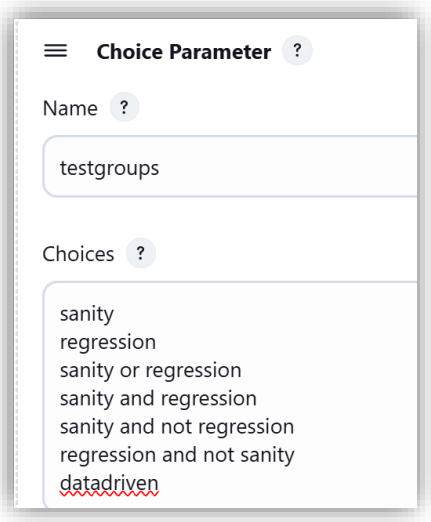
- Click **Add Parameter → Choice Parameter**.
    - **Name:** testgroups
    - **Choices:**

sanity

regression

sanity or regression

sanity and regression



Set GitHub Repository URL

- Go to '**Source Code Management'** section.
- Check **"Git"**.
- Provide GitHub repository URL.

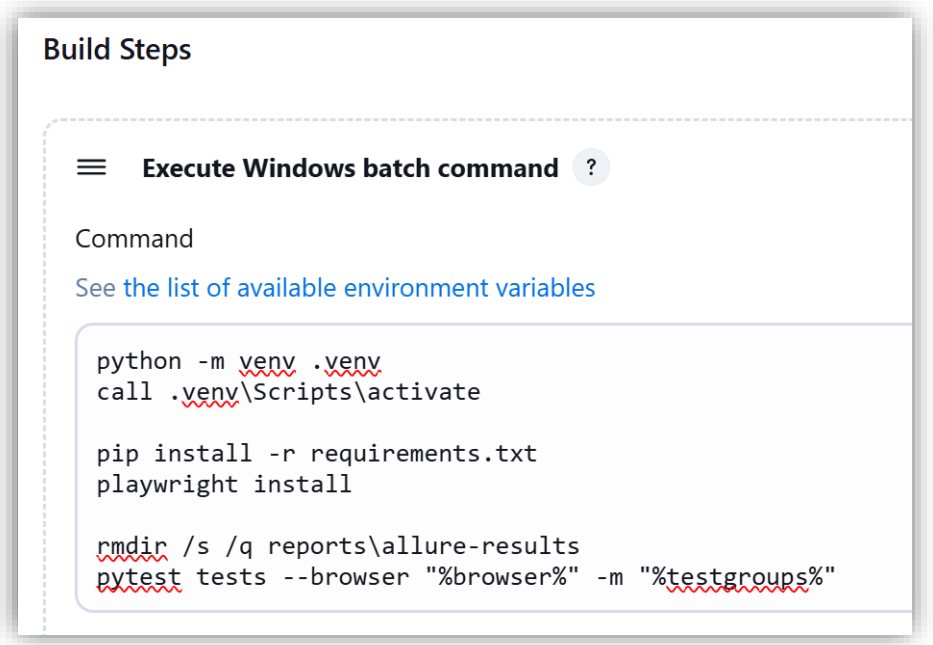
Add Build Step

- Click on **"Add build step" → "Execute Windows batch command"** *(if on Windows)*.
    - **Command:**

```
python -m venv .venv
call .venv\Scripts\activate

pip install -r requirements.txt
playwright install

rmdir /s /q reports\allure-results
pytest tests --browser "%browser%" -m "%testgroups%"
```

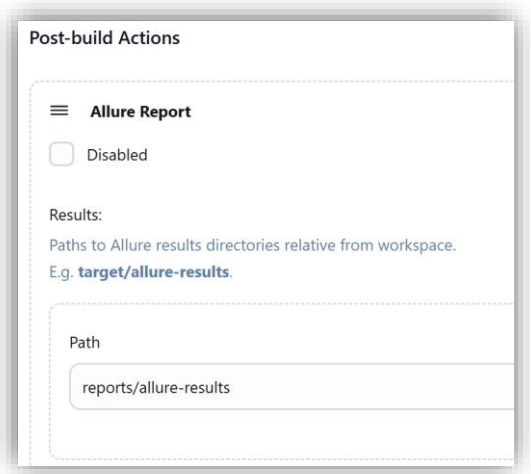
Note: If you're on **Mac/Linux**, select **"Execute shell"** and use:

```
python -m venv .venv
call .venv\Scripts\activate
pip install -r requirements.txt
playwright install

rmdir /s /q reports\allure-results
pytest tests\ --browser "$browser" -m "$testgroups"
```



Step 4: Post-build Actions (Allure Reports)

- Scroll to **"Post-build Actions"**.
- Select **"Allure Report"**.
- In **"Results path"**, enter:
- allure-results



Step 5: Save the Configuration

- Click **Apply** and then **Save** the job.

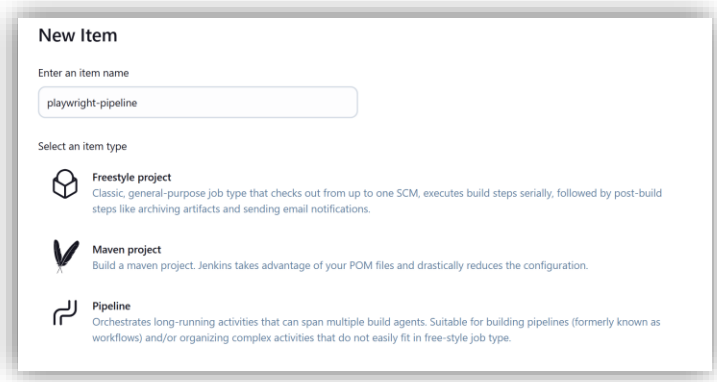
Run Github Playwright project in Jenkins with Pipeline

Step 1: Login to Jenkins

Open your Jenkins instance in a browser and log in using your credentials.

Step 2: Create a New Jenkins Job

- Click on **“New Item”**.
- Enter a name for your job.
- Select **“Pipeline project”**. (If you could not see this , then you need to install Pipeline plugin from manage Jenkins→plugins).
- Click **OK** to proceed.



Step 3: Configure the Job

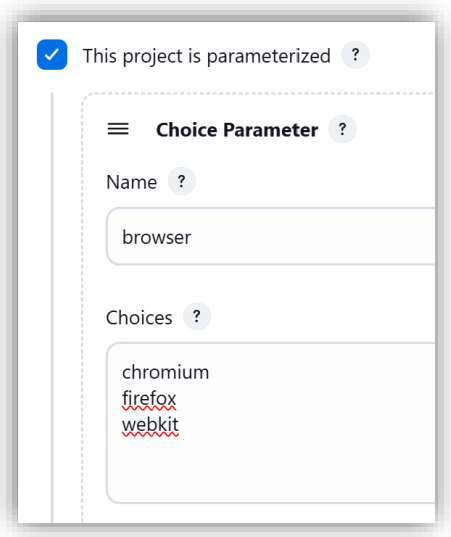
Enable Parameters

- Check the box **“This project is parameterized”**.
- Click **Add Parameter → Choice Parameter**.
    - **Name:** browser
    - **Choices:**

        chromium

        firefox

        webkit



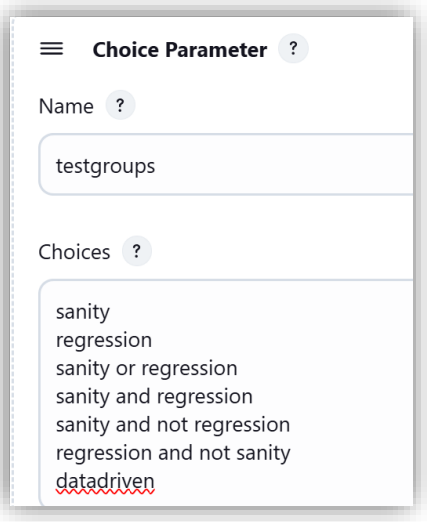
- Click **Add Parameter → Choice Parameter**.
    - **Name:** testgroups
    - **Choices:**

        sanity

        regression

        sanity or regression

        sanity and regression



Goto Pipeline section and add Pipeline script.

Pipeline Script (Windows):

```
pipeline {
    agent any


    environment {
        VENV_DIR = '.venv'
        REPORT_DIR = 'reports'
```

```
    }

    stages {

        // Step 1: Checkout Code from GitHub
        stage('Checkout Code') {
            steps {
                git 'https://github.com/pavanoltraining/PWPythonFrameworkPrep1.git'
                echo "Code checked out successfully."
            }
        }

        // Step 2: Setup Python Virtual Environment
        stage('Setup Python Environment') {
            steps {
                script {
                    if (!fileExists("${VENV_DIR}\\Scripts\\activate")) {
                        echo "Creating virtual environment..."
                        bat "python -m venv ${VENV_DIR}"
                    } else {
                        echo "Virtual environment already exists."
                    }
                }
            }
        }

        // Step 3: Install Required Packages
        stage('Install Dependencies') {
            steps {
                bat """
                    call ${VENV_DIR}\\Scripts\\activate
                    python -m pip install --upgrade pip
                    pip install -r requirements.txt
                    playwright install
                """
            }
        }

        // Step 4: Clean Old Test Reports
        stage('Clean Old Reports') {
            steps {
                bat """
                    if exist ${REPORT_DIR} (
                        rmdir /s /q ${REPORT_DIR}
                    )
                """
            }
        }

        // Step 5: Run Pytest Tests and Generate Allure Results
        stage('Run Tests') {
            steps {
                bat """
                    call ${VENV_DIR}\\Scripts\\activate
                    pytest tests --browser "%browser%" -m "%testgroups%" --
alluredir=${REPORT_DIR}\\allure-results
                """
            }
        }

        // Step 6: Publish Allure Report in Jenkins
        stage('Publish Allure Report') {
            steps {
                allure includeProperties: false, jdk: '', results: [[path:
"${REPORT_DIR}\\allure-results"]]
            }
        }
    }

    post {
```

```
        always {
            echo "Cleaning up workspace after build..."
        }
        success {
            echo "Tests executed successfully!"
        }
        failure {
            echo "Test execution failed!"
        }
    }
}
```

Pipeline Script (Max/Lnux):

```
pipeline {
    agent any

    // Optional parameters (you can remove if not needed)
    parameters {
        string(name: 'browser', defaultValue: 'chromium', description: 'Browser to run tests
on')
        string(name: 'testgroups', defaultValue: '', description: 'Test group or marker name')
    }

    environment {
        VENV_DIR = '.venv'
        REPORT_DIR = 'reports'
    }

    stages {

        // Step 1: Checkout Code from GitHub
        stage('Checkout Code') {
            steps {
                git 'https://github.com/pavanoltraining/PWPythonFrameworkPrep1.git'
                echo "Code checked out successfully."
            }
        }

        // Step 2: Setup Python Virtual Environment
        stage('Setup Python Environment') {
            steps {
                script {
                    if (!fileExists("${VENV_DIR}/bin/activate")) {
                        echo "Creating virtual environment..."
                        sh "python3 -m venv ${VENV_DIR}"
                    } else {
                        echo "Virtual environment already exists."
                    }
                }
            }
        }

        // Step 3: Install Required Packages
        stage('Install Dependencies') {
            steps {
                sh """
                    source ${VENV_DIR}/bin/activate
                    python3 -m pip install --upgrade pip
                    pip install -r requirements.txt
                    playwright install
                """
            }
        }

        // Step 4: Clean Old Test Reports
        stage('Clean Old Reports') {
            steps {
```

```
                sh """
                    if [ -d "${REPORT_DIR}" ]; then
                        rm -rf ${REPORT_DIR}
                    fi
                """
            }
        }

        // Step 5: Run Pytest Tests and Generate Allure Results
        stage('Run Tests') {
            steps {
                sh """
                    source ${VENV_DIR}/bin/activate
                    pytest tests --browser "${browser}" -m "${testgroups}" --
alluredir=${REPORT_DIR}/allure-results
                """
            }
        }

        // Step 6: Publish Allure Report in Jenkins
        stage('Publish Allure Report') {
            steps {
                allure includeProperties: false, jdk: '', results: [[path:
"${REPORT_DIR}/allure-results"]]
            }
        }
    }

    post {
        always {
            echo "Cleaning up workspace after build..."
        }
        success {
            echo "Tests executed successfully!"
        }
        failure {
            echo "Test execution failed!"
        }
    }
}
```

Step 4: Save the Configuration

- Click **Apply** and then **Save** the job.

Pipeline stages:

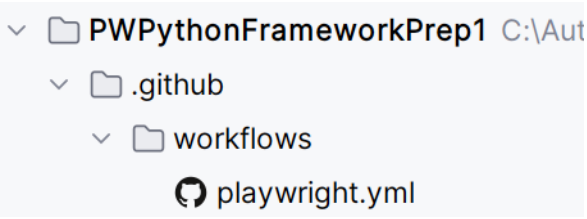
1. Checkout Code
2. Setup Python Environment
3. Install Dependencies
4. Clean Old Reports
5. Run Tests
6. Publish Allure Report

GitHub Actions:

On push/pull_request

Tests will run on push or pull request on branches main/master. The workflow will install all dependencies, install Playwright and then run the tests.

File location:

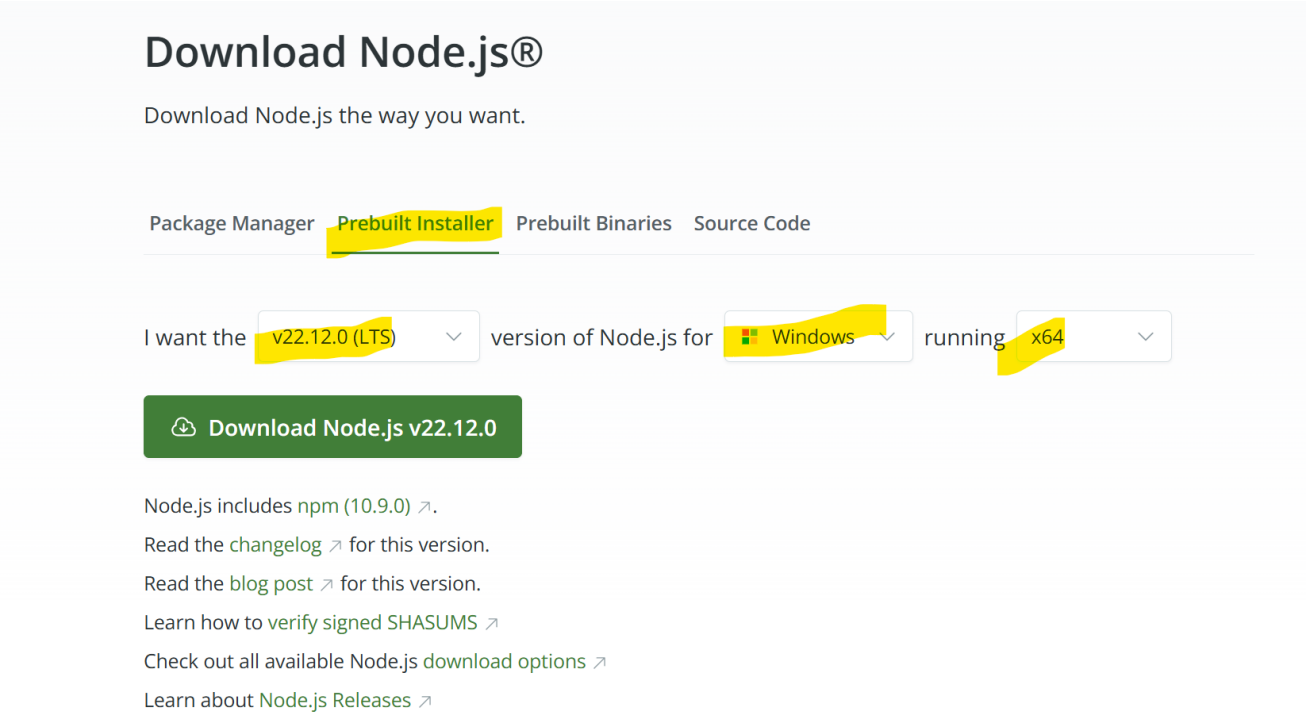


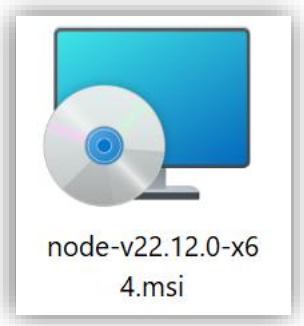
.github/workflows/playwright.yml

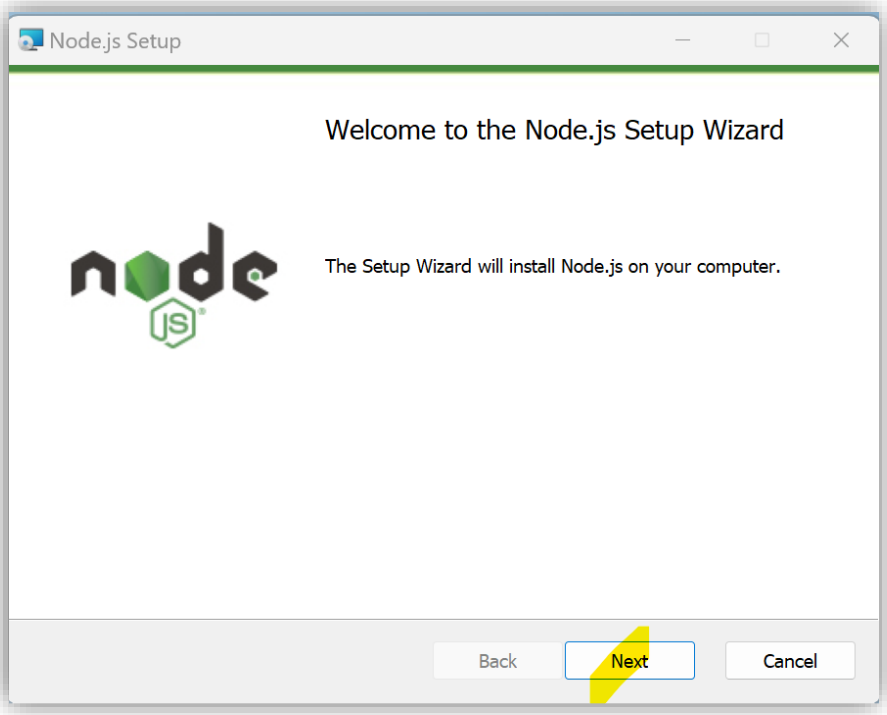
```
name: Playwright Tests
on:
  push:
    branches: [ master ]
  pull_request:
    branches: [ master ]
jobs:
  playwright:
    name: 'Playwright Tests'
    runs-on: ubuntu-latest
    container:
      image: mcr.microsoft.com/playwright/python:v1.55.0-noble
      options: --user 1001
    steps:
      - uses: actions/checkout@v5
      - name: Set up Python
        uses: actions/setup-python@v6
        with:
          python-version: '3.13'
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
      - name: Run your tests
        run: pytest
```

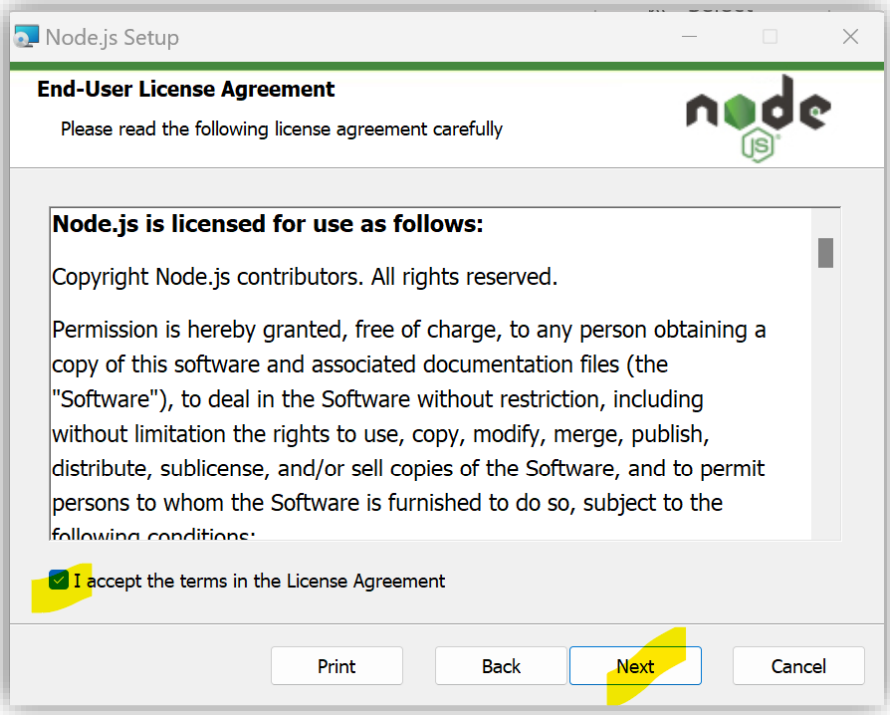
NodeJS Installation

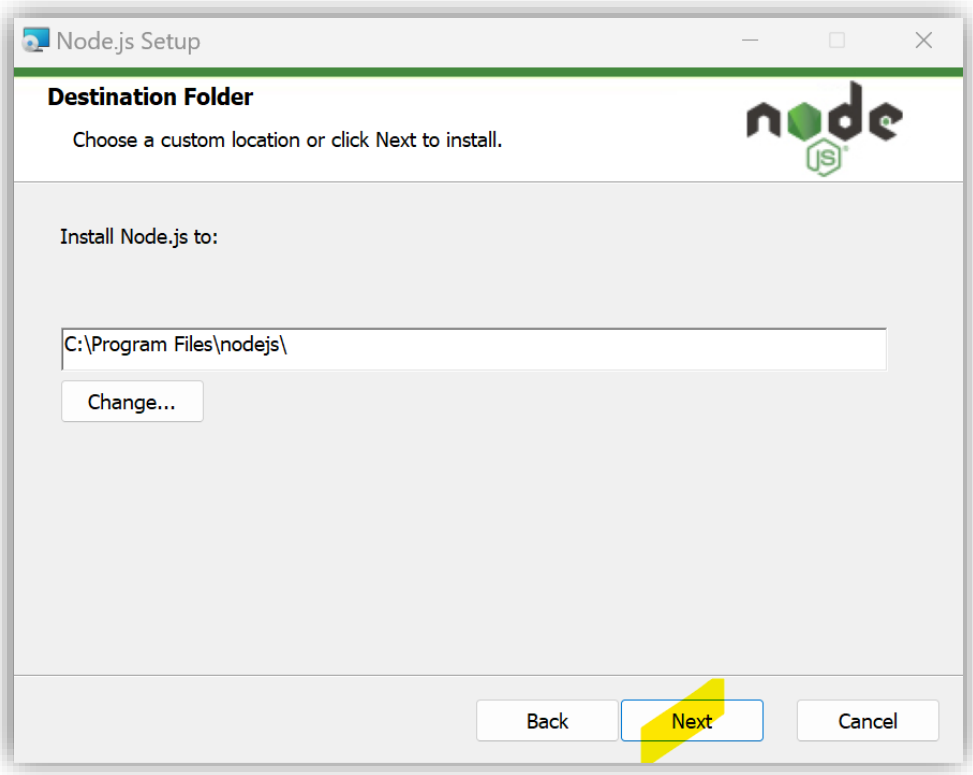
Download link: https://nodejs.org/en/download/

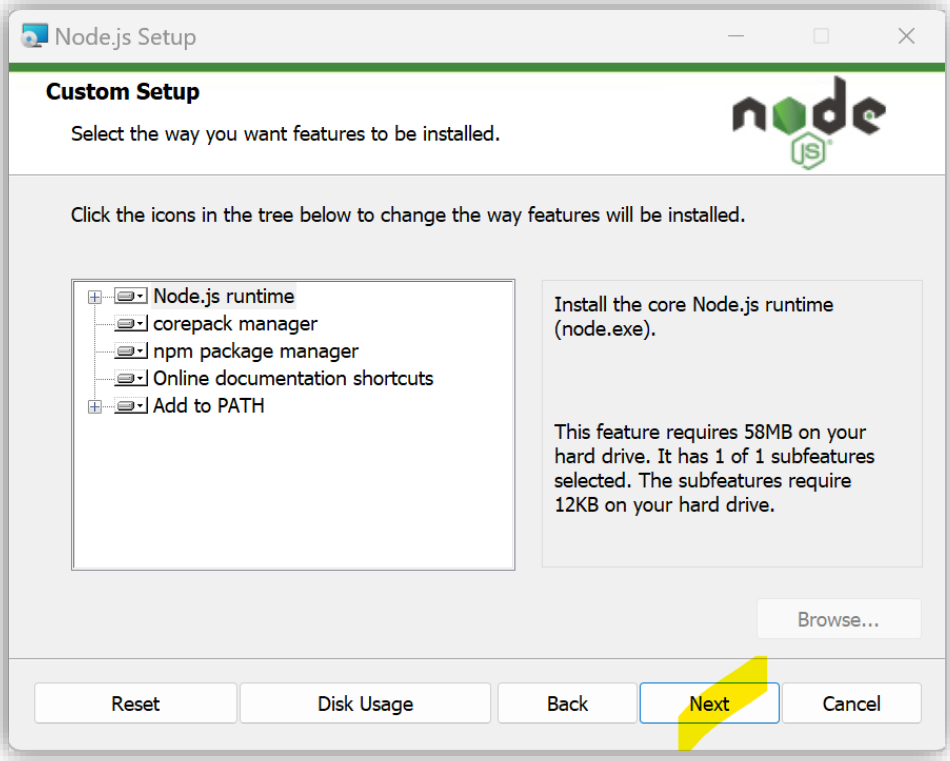


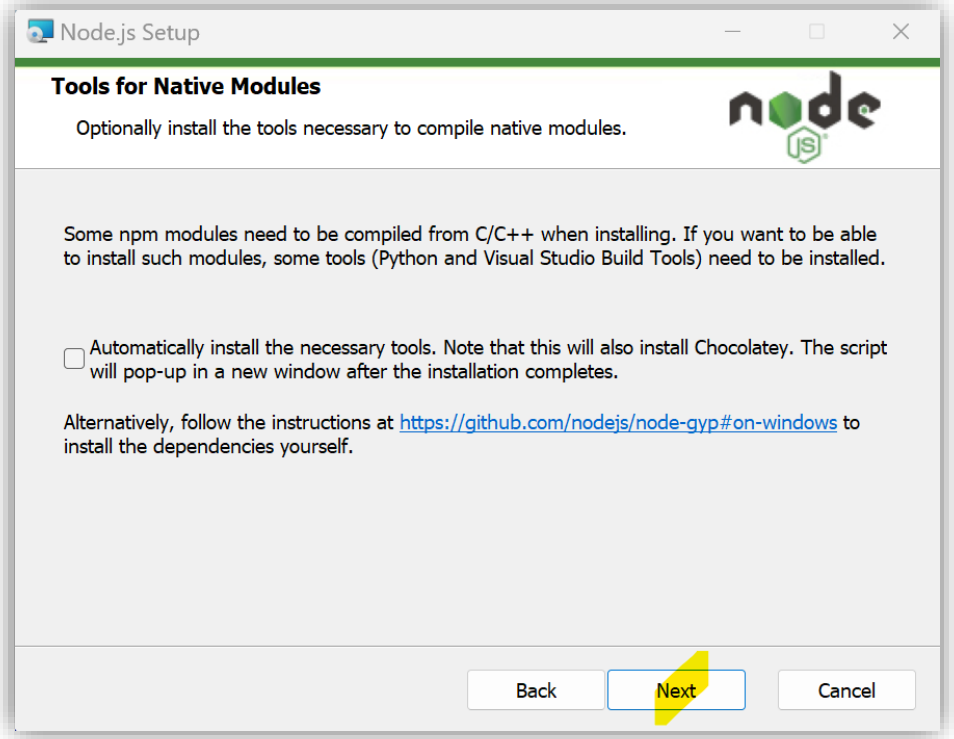


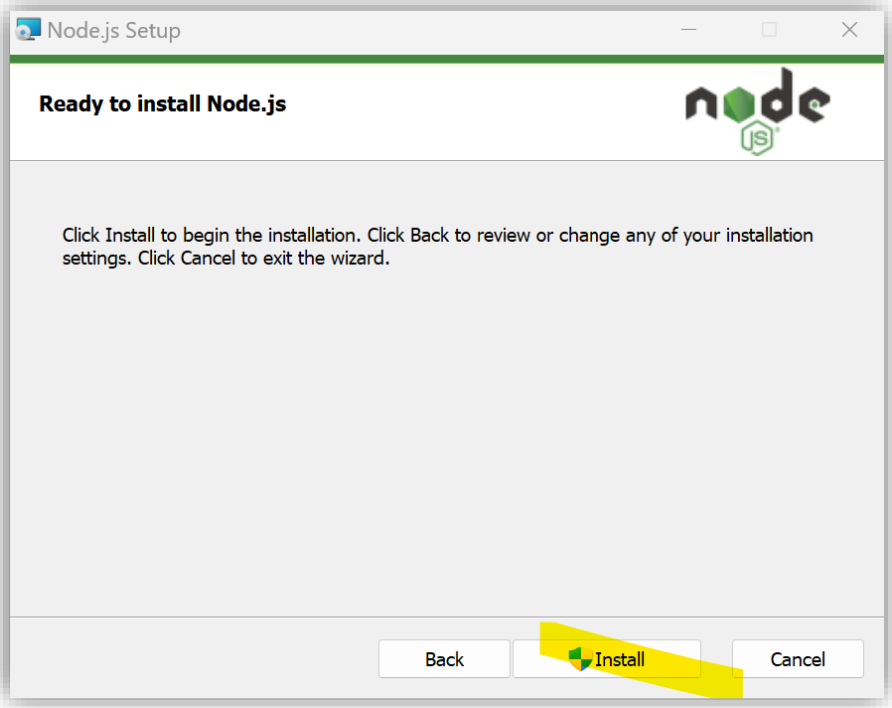

Node.js Setup
Welcome to the Node.js Setup Wizard
The Setup Wizard will install Node.js on your computer.
Back
Next
Cancel

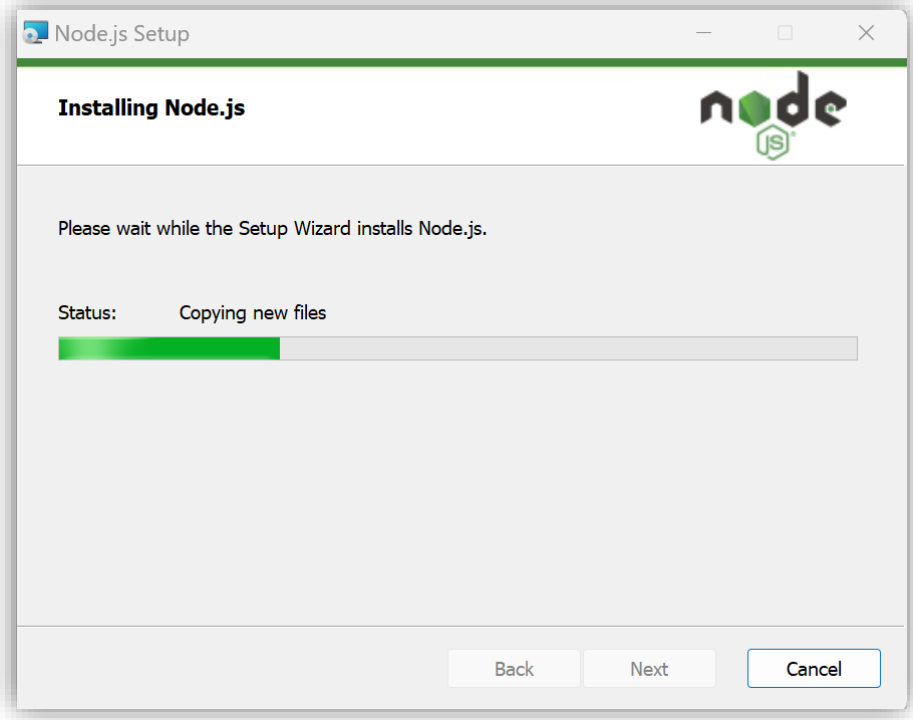

Node.js Setup
End-User License Agreement
Please read the following license agreement carefully
Node.js is licensed for use as follows:
Copyright Node.js contributors. All rights reserved.
Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the
I accept the terms in the License Agreement
Print
Back
Next
Cancel

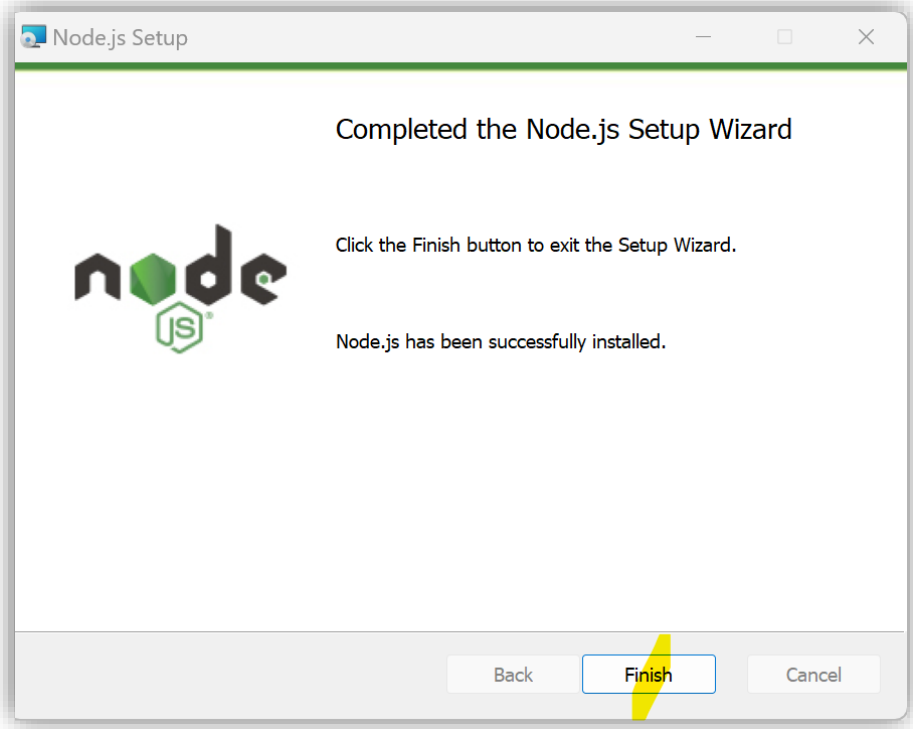

Node.js Setup
Destination Folder
Choose a custom location or click Next to install.
Install Node.js to:
C:\Program Files\nodejs\
Change...
Back
Next
Cancel


Node.js Setup
Custom Setup
Select the way you want features to be installed.
Click the icons in the tree below to change the way features will be installed.
Node.js runtime
corepack manager
npm package manager
Online documentation shortcuts
Add to PATH
Install the core Node.js runtime (node.exe).
This feature requires 58MB on your hard drive. It has 1 of 1 subfeatures selected. The subfeatures require 12KB on your hard drive.
Browse...
Reset
Disk Usage
Back
Next
Cancel

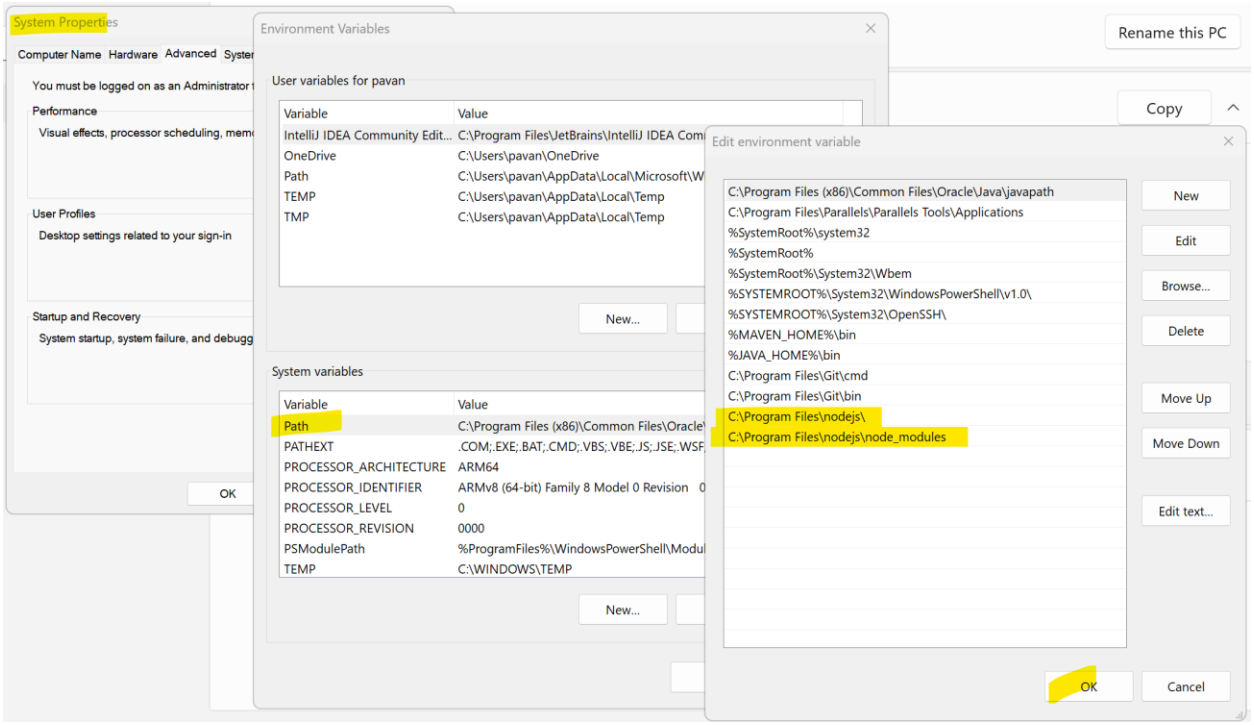

Node.js Setup
Tools for Native Modules
Optionally install the tools necessary to compile native modules.
Some npm modules need to be compiled from C/C++ when installing. If you want to be able to install such modules, some tools (Python and Visual Studio Build Tools) need to be installed.
Automatically install the necessary tools. Note that this will also install Chocolatey. The script will pop-up in a new window after the installation completes.
Alternatively, follow the instructions at https://github.com/nodejs/node-gyp#on-windows to install the dependencies yourself.
Back
Next
Cancel


Node.js Setup
Ready to install Node.js
Click Install to begin the installation. Click Back to review or change any of your installation settings. Click Cancel to exit the wizard.
Back
Install
Cancel

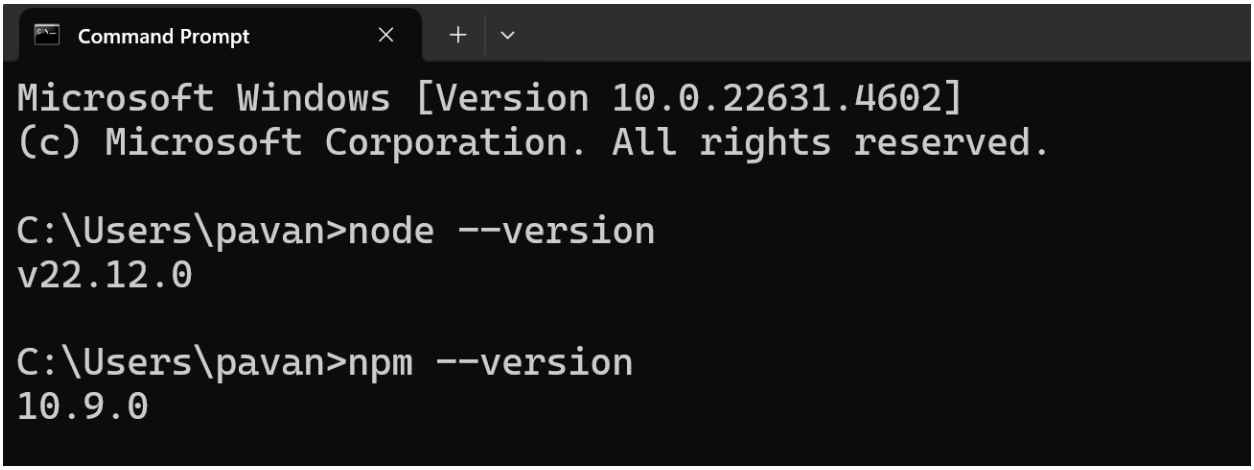

Node.js Setup
Installing Node.js
Please wait while the Setup Wizard installs Node.js.
Status: Copying new files
Back
Next
Cancel


Node.js Setup
Completed the Node.js Setup Wizard
Click the Finish button to exit the Setup Wizard.
Node.js has been successfully installed.
Back
Finish
Cancel

Set Environment variable:



Check node and npm versions (npm comes along with node.js)

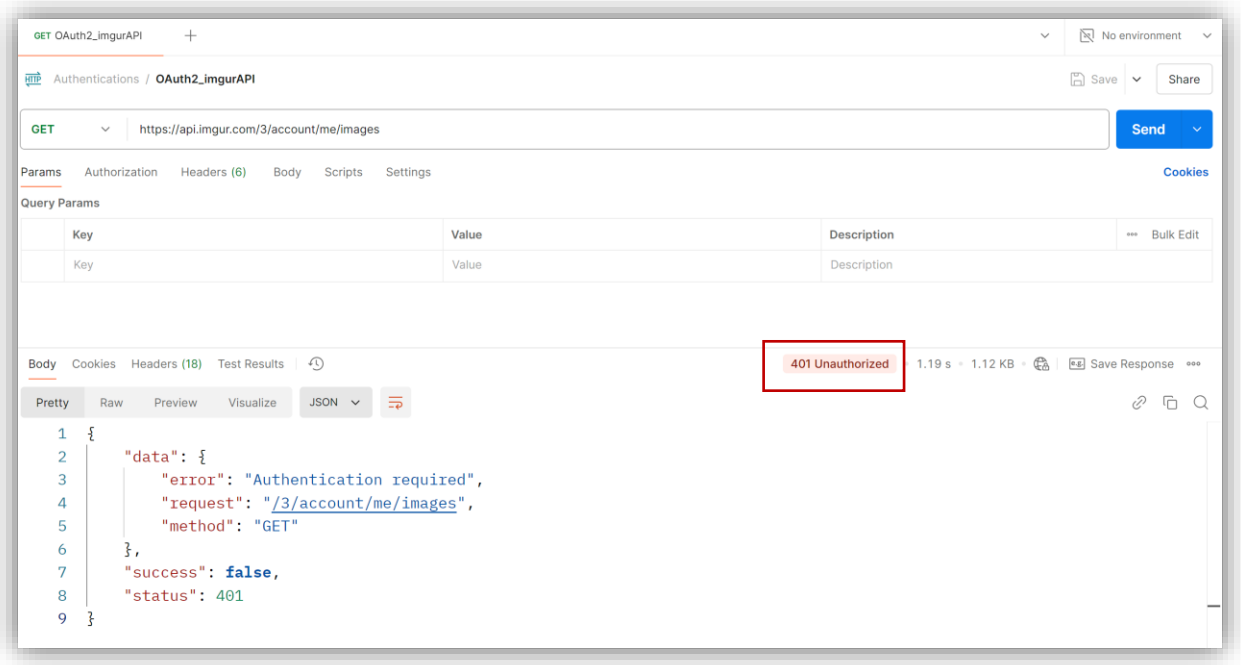


OAuth 2.0 Authorization with Postman

Create a new Get Request in Postman.

This is the endpoint for fetching your images from your account in imgur.

GET https://api.imgur.com/3/account/me/images

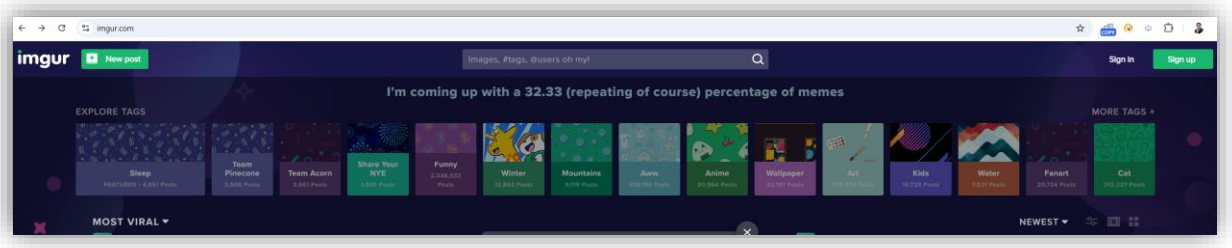


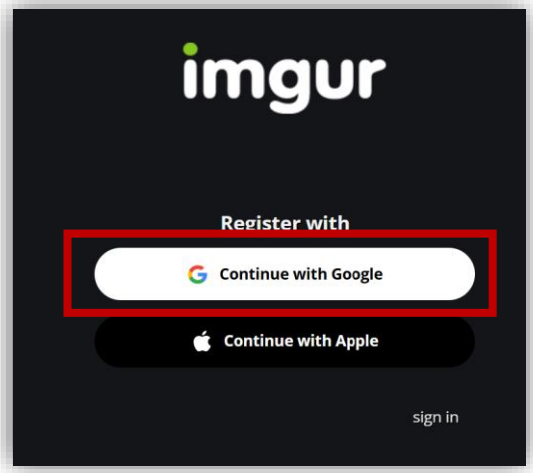
Authentication error will occur and ***401 Unauthorized*** status will be returned. This means that this endpoint requires Authentication to access the resource.

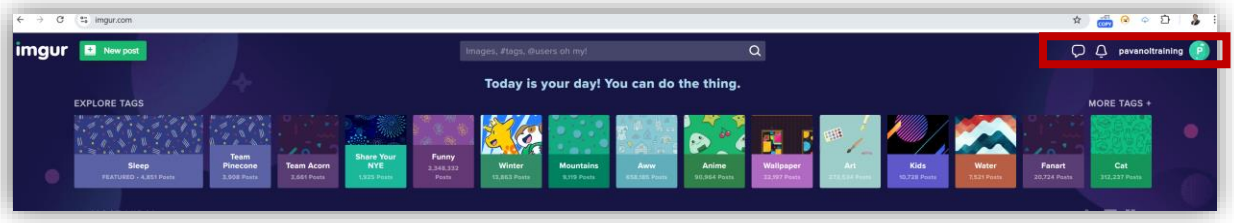
Generate Access Token using OAuth 2 in Postman

Step 1: Sign Up on Imgur

1. Open a web browser and navigate to the url https://imgur.com/
2. Sign up for an account using your email and create a username and password.
3. Complete any additional verification steps if prompted.

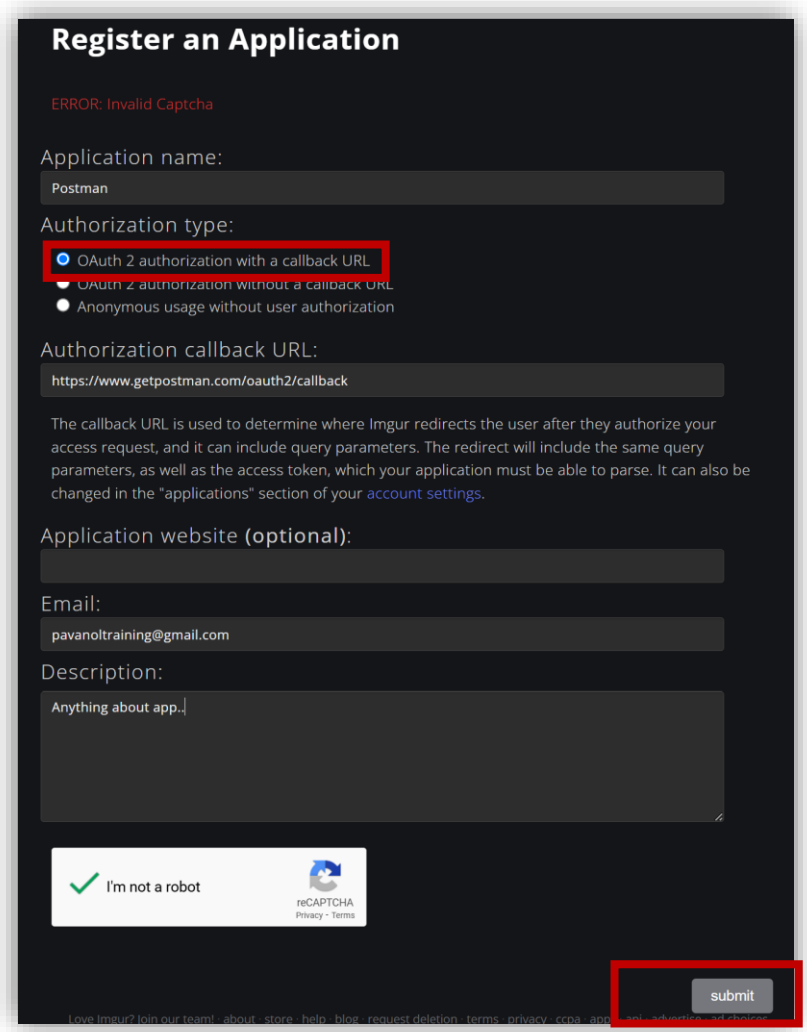


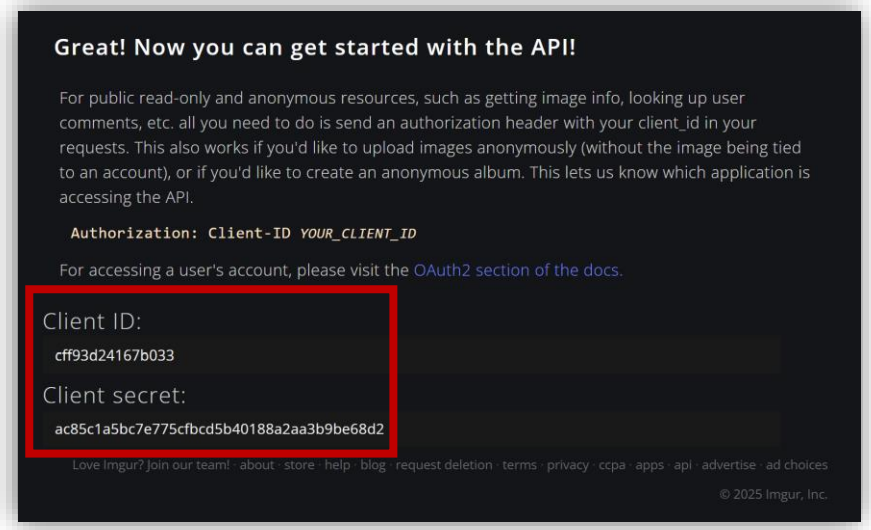




Step 2: Register Your Application

1. Log in to your Imgur account using your username and password.
2. Open the following URL in your browser to register your application: https://api.imgur.com/oauth2/addclient
3. Fill in the required details in the registration form. Use the following values:
    - **Application Name**: Postman (Any name)
    - **Authorization Type**: OAuth 2
    - **Authorization Callback URL**: https://www.getpostman.com/oauth2/callback
    - **Email**: xyz@gmail.com(Any email)
4. Click **Submit** after filling in the form.
5. After submission, you will receive a **Client ID** and **Client Secret**. Save these details securely for the next steps.



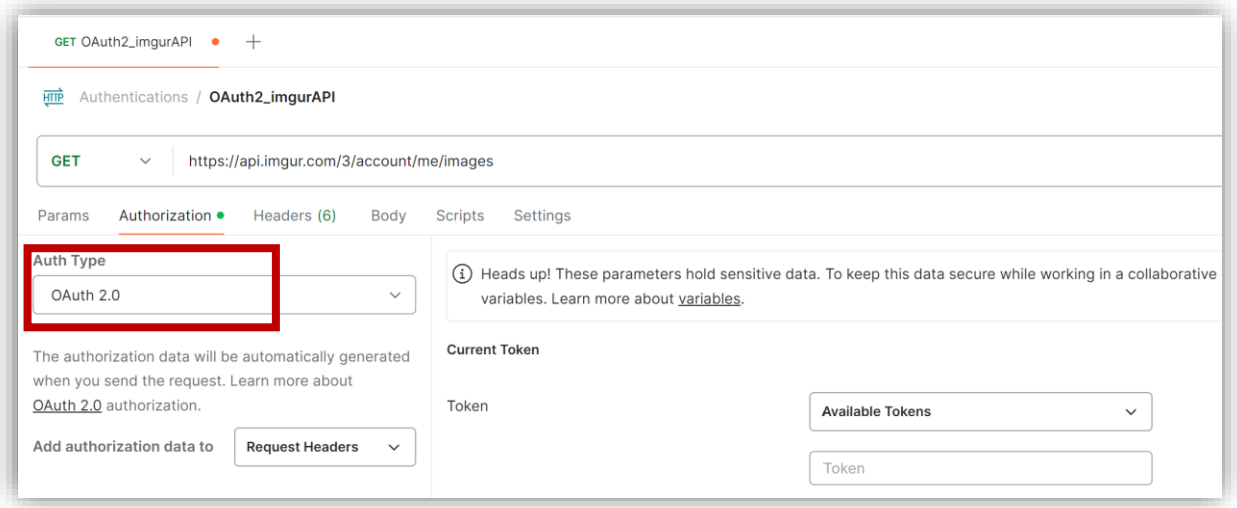


Client ID: cff93d[redacted]33

Client secret: ac[redacted]bcd5b40188a2aa3b9be68d2

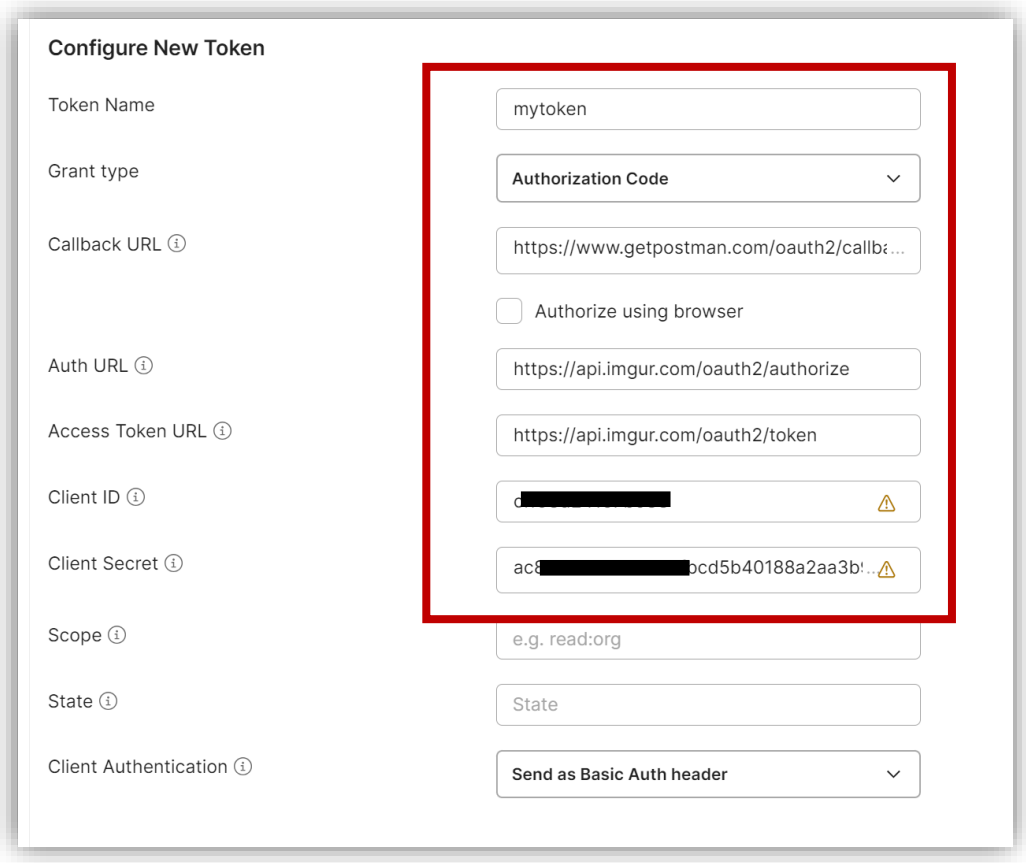
Step 3: Configure Authorization in Postman

1. Open the **Postman** application on your computer.
2. Navigate to the **Authorization** tab of your request.
3. In the **Type** dropdown, select **OAuth 2.0**.



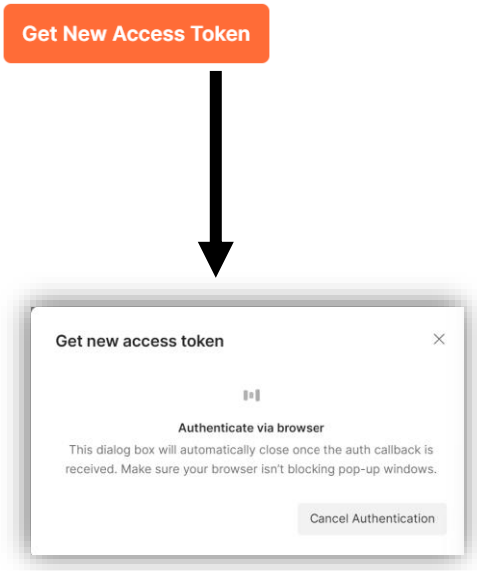
Step 4: Fill in OAuth 2.0 Details

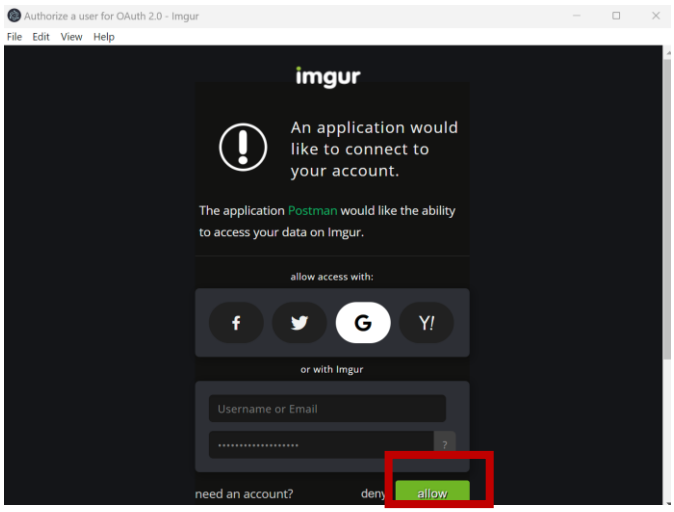
1. In the OAuth 2.0 configuration form, provide the following details:
    - **Token Name**: Provide a descriptive name (e.g., Imgur Token).
    - **Callback URL**: https://www.getpostman.com/oauth2/callback
    - **Auth URL**: https://api.imgur.com/oauth2/authorize
    - **Access Token URL**: https://api.imgur.com/oauth2/token
    - **Client ID**: Enter the Client ID you received during registration.
    - **Client Secret**: Enter the Client Secret you received during registration.
    - **Scope**: Leave this field empty or add specific scopes if required (check Imgur documentation for scope options).
    - **Grant Type**: Select the grant type supported by Imgur (usually **Authorization Code**).

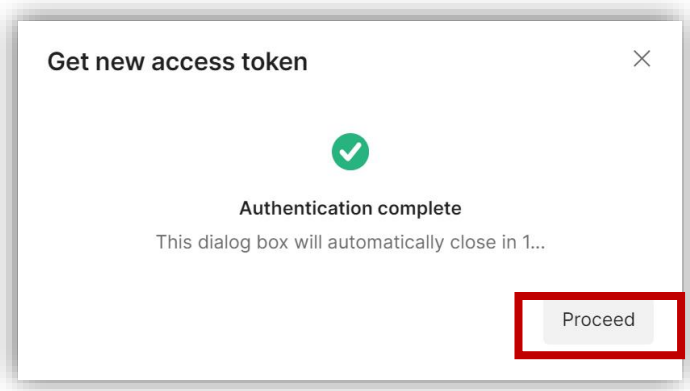


Step 5: Generate a New Token

1. Press **Get New Access Token**.
2. A new window will open, redirecting you to Imgur's login page.
3. Enter your Imgur credentials (username and password) and log in.
4. You will be prompted to allow access for the application. Click **Allow**.

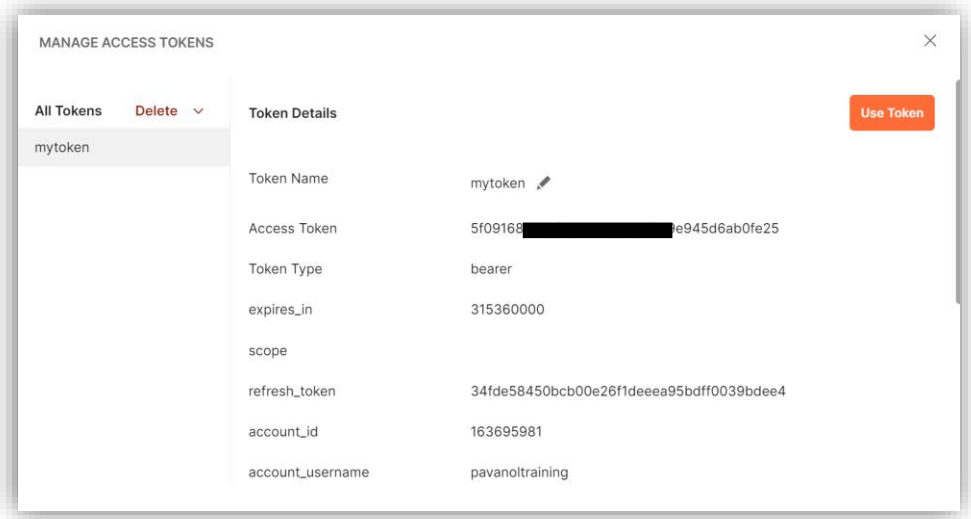


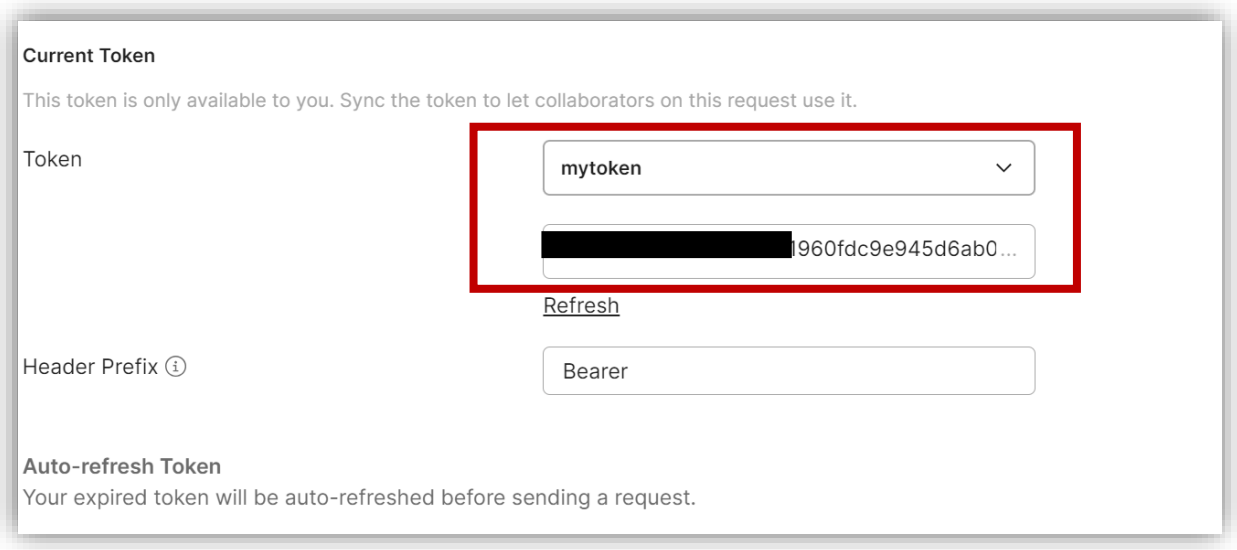




Step 6: Use the Access Token

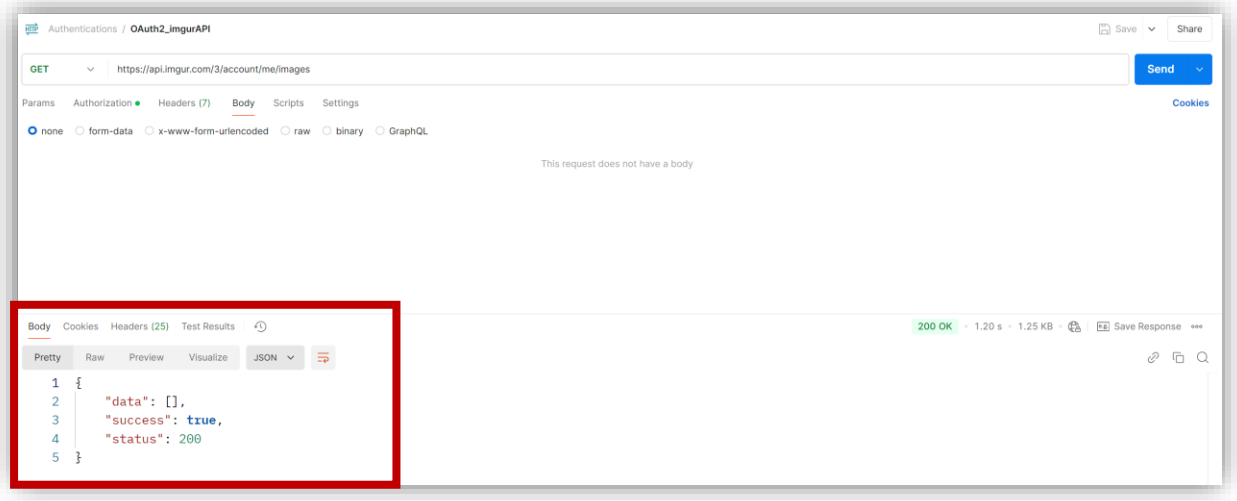
1. Once the token is successfully generated, you will see a confirmation in Postman.
2. Click **Use Token** to apply the token to your request.
3. The token will be automatically added to the request headers.





Step 7: Test Your API Requests

1. Return to the **Request** tab in Postman.
2. Use the configured access token to test authorized requests to Imgur's API.



Opencart Build Deployment/Installation

1) **Opencart download location:**
   https://www.opencart.com/index.php?route=cms/download

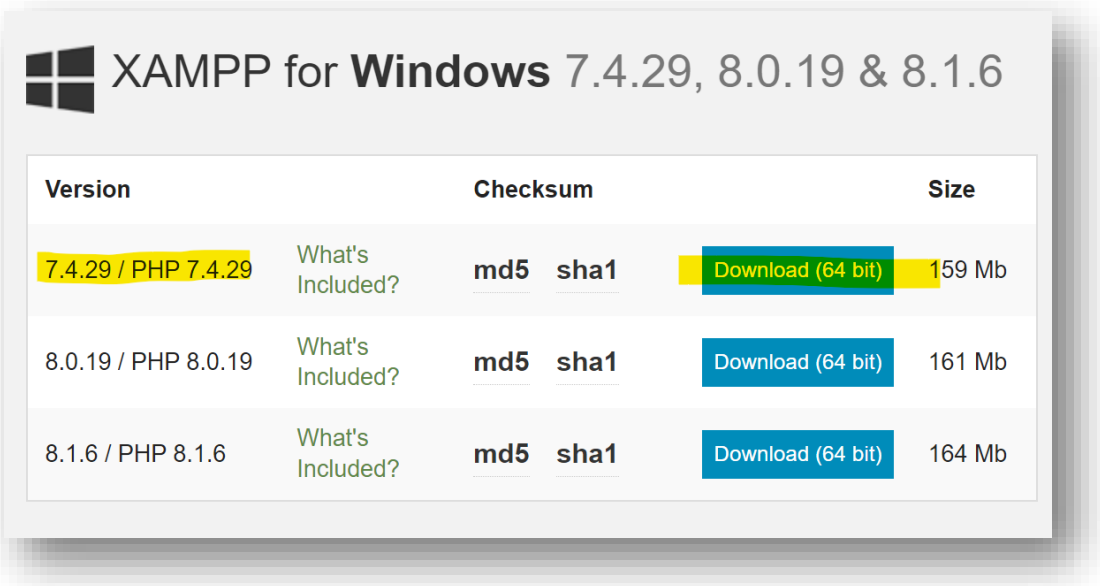
Click on previous releases

Recommended version: opencart-3.0.3.8

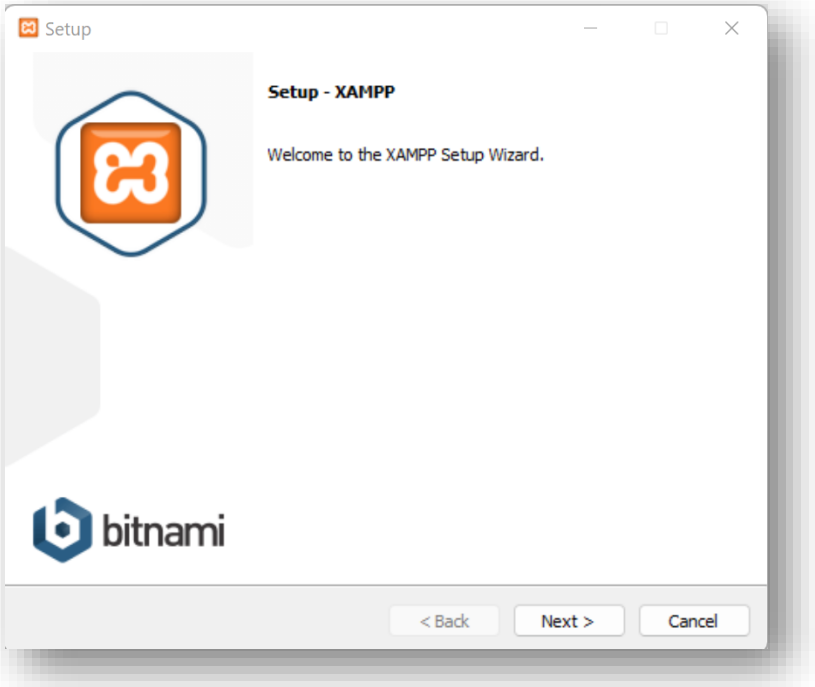
2) **XAMPP for apache, mysql, and php installation.**

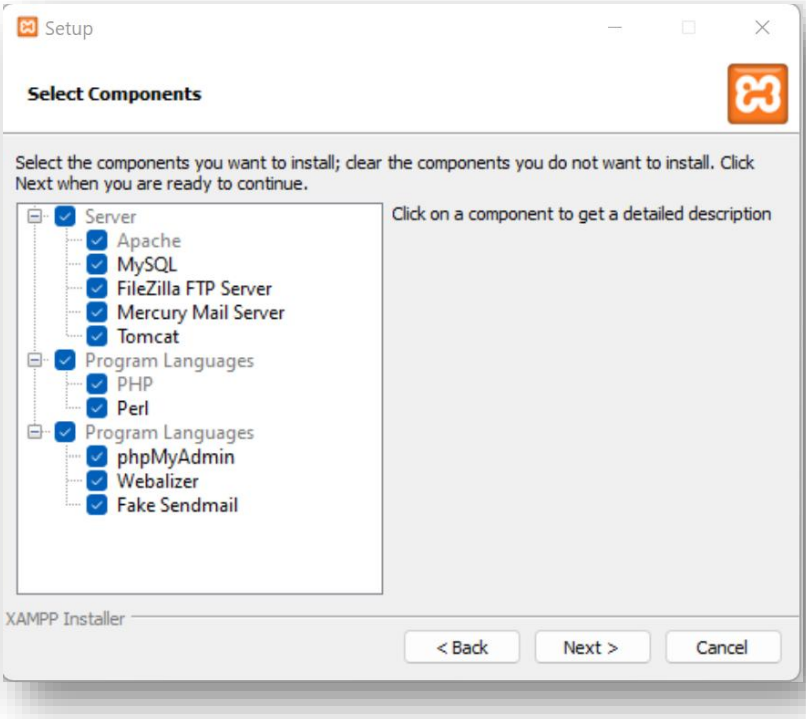
Download Link: https://www.apachefriends.org/download.html

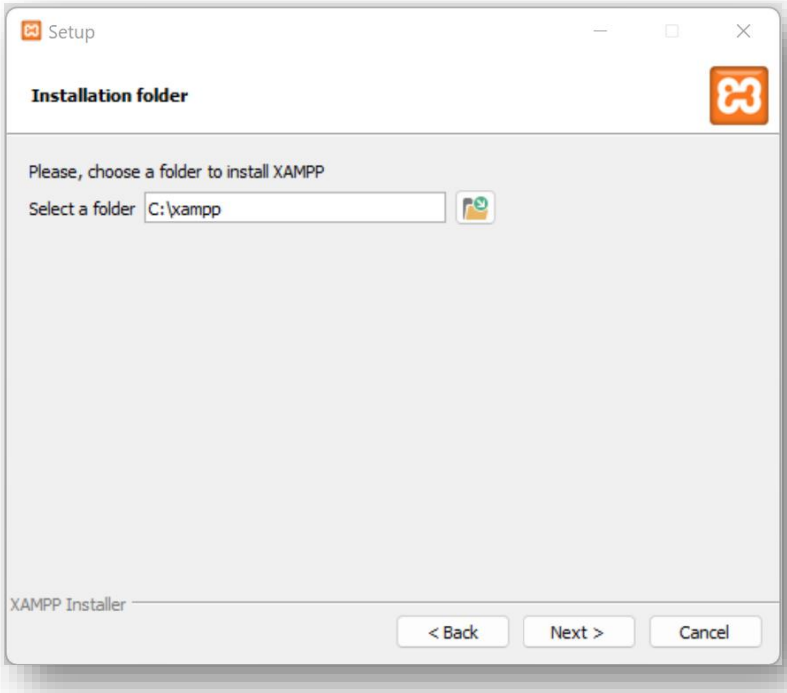
Recommended version: 7.4.29/PHP 7.4.29

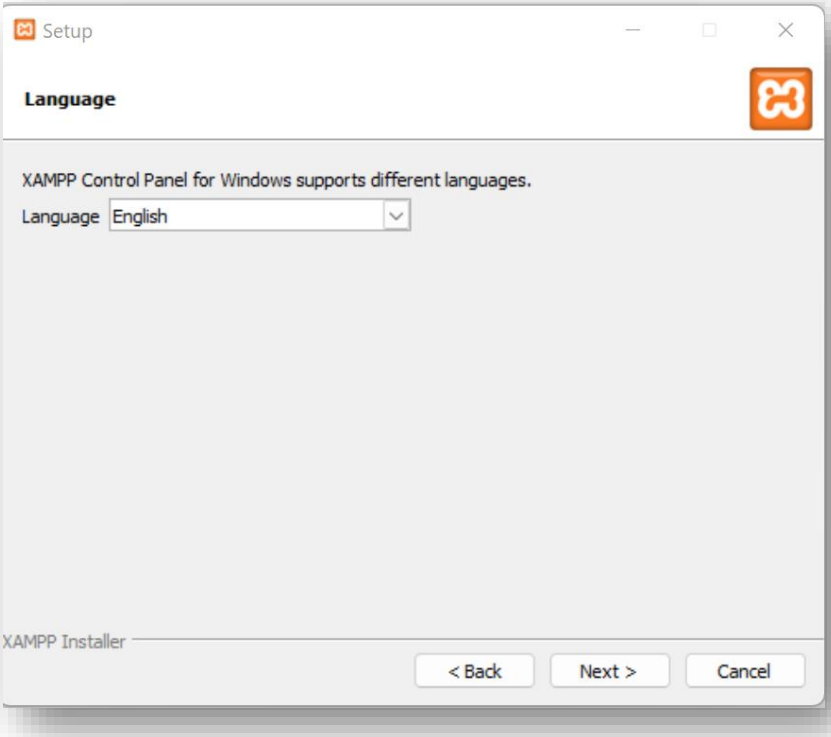


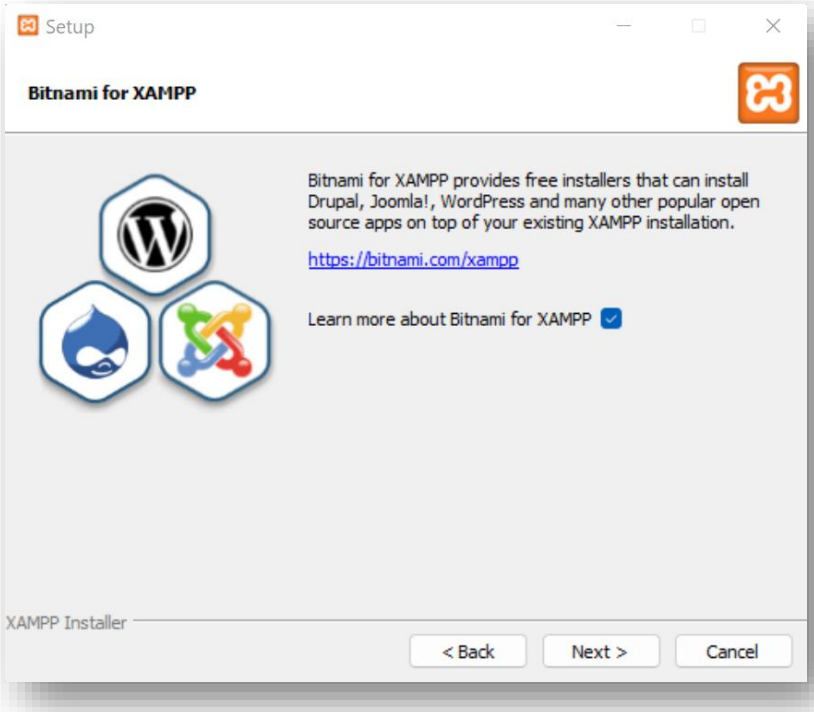
Step by step XAMPP Installation

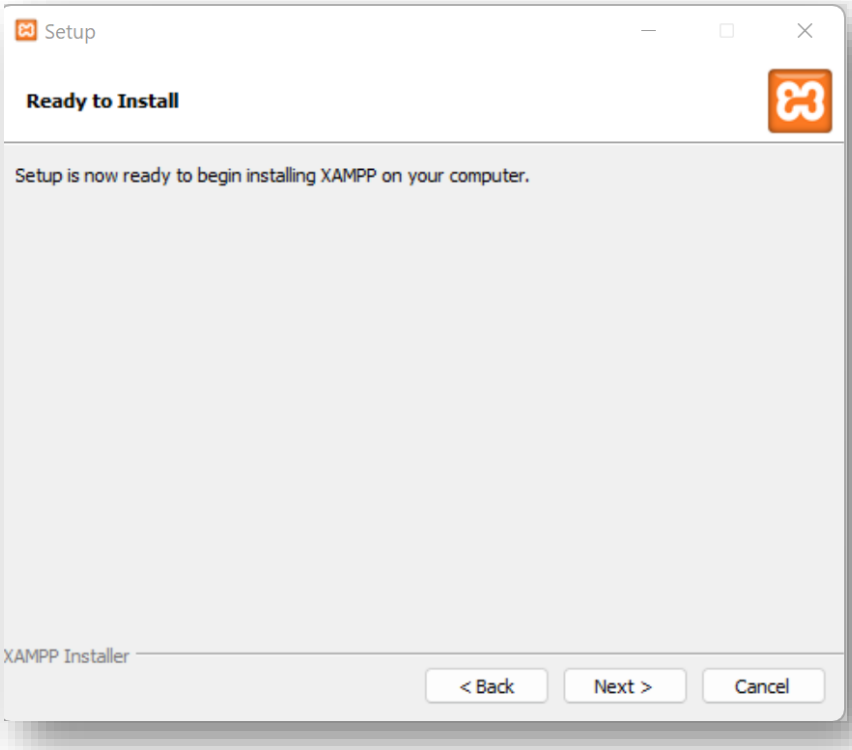


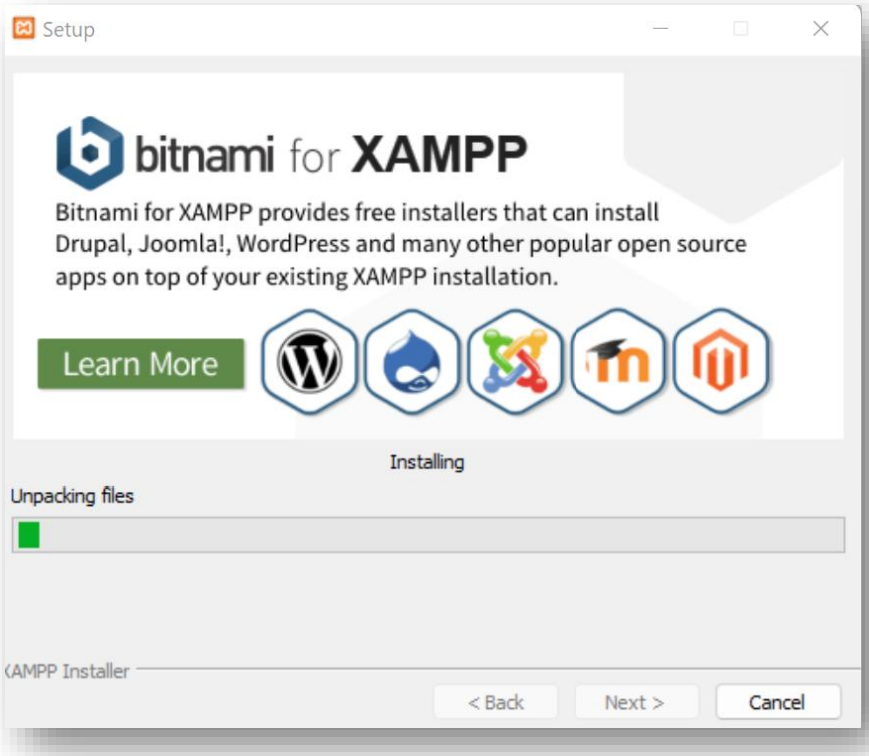


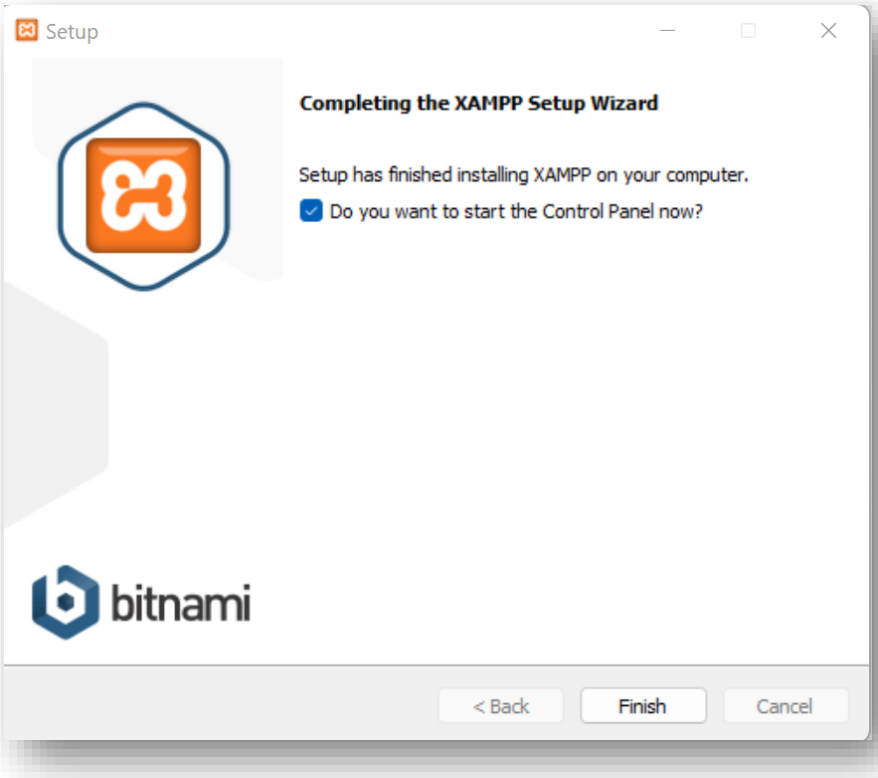

Setup
Installation folder
Please, choose a folder to install XAMPP
Select a folder
C:\xampp
XAMPP Installer
< Back
Next >
Cancel

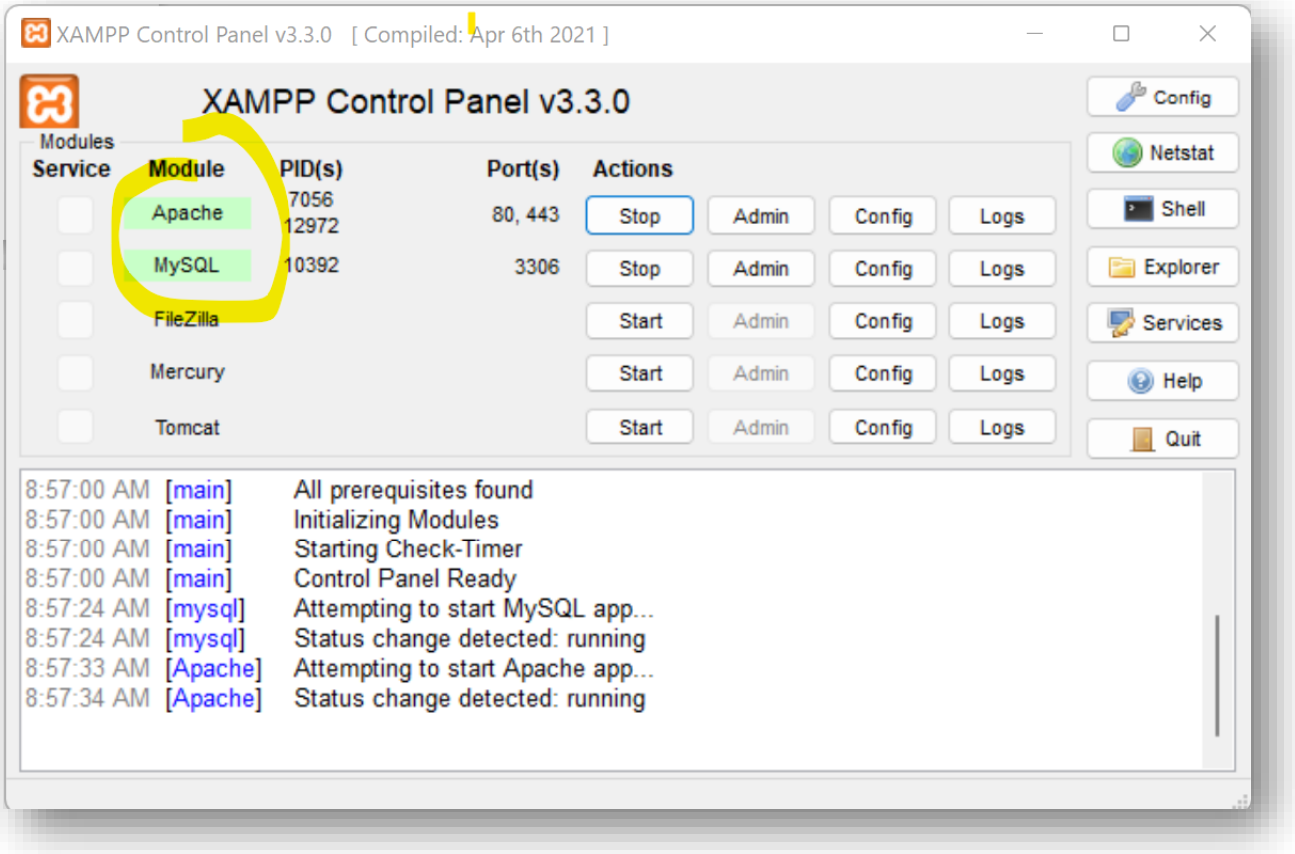

Setup
Language
XAMPP Control Panel for Windows supports different languages.
Language
English
XAMPP Installer
< Back
Next >
Cancel


Setup
Bitnami for XAMPP
Bitnami for XAMPP provides free installers that can install Drupal, Joomla!, WordPress and many other popular open source apps on top of your existing XAMPP installation.
https://bitnami.com/xampp
Learn more about Bitnami for XAMPP
XAMPP Installer
< Back
Next >
Cancel


Setup
Ready to Install
Setup is now ready to begin installing XAMPP on your computer.
XAMPP Installer
< Back
Next >
Cancel

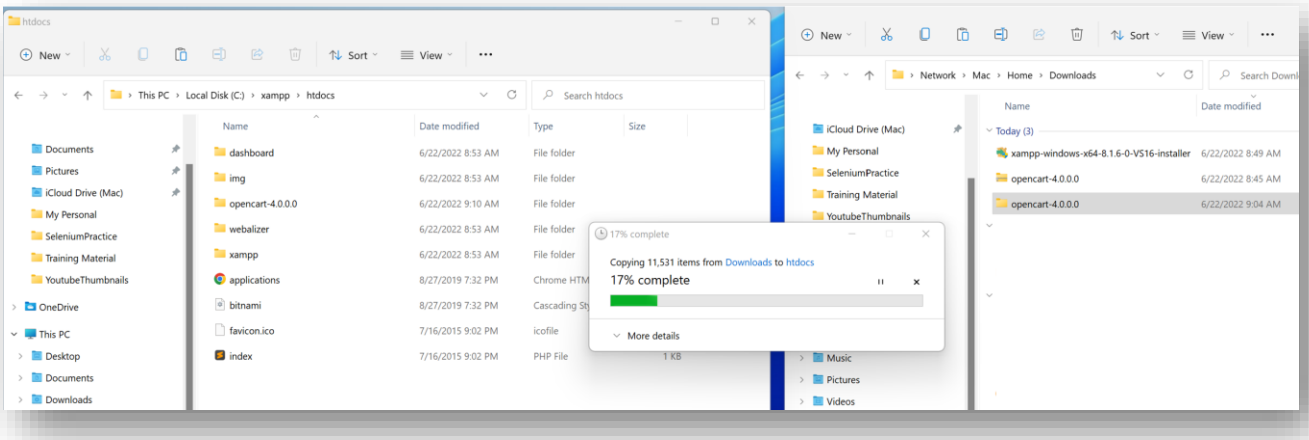

Setup
bitnami for XAMPP
Bitnami for XAMPP provides free installers that can install Drupal, Joomla!, WordPress and many other popular open source apps on top of your existing XAMPP installation.
Learn More
Installing
Unpacking files
(AMPP Installer
< Back
Next >
Cancel


Setup
Completing the XAMPP Setup Wizard
Setup has finished installing XAMPP on your computer.
Do you want to start the Control Panel now?
bitnami
< Back
Finish
Cancel

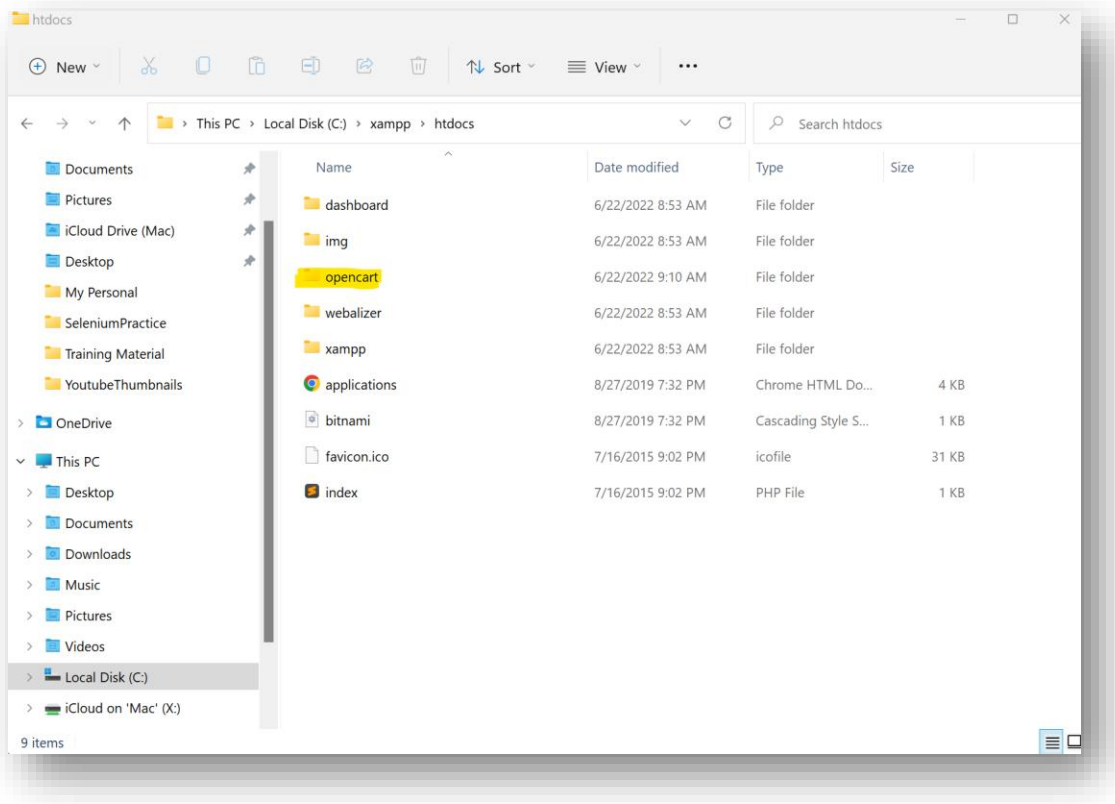


Step by step OpenCart Installation

3) Copy opencart folder(Step1) in to below location. C:\xampp\htdocs



4) **After copying rename the folder " opencart "**

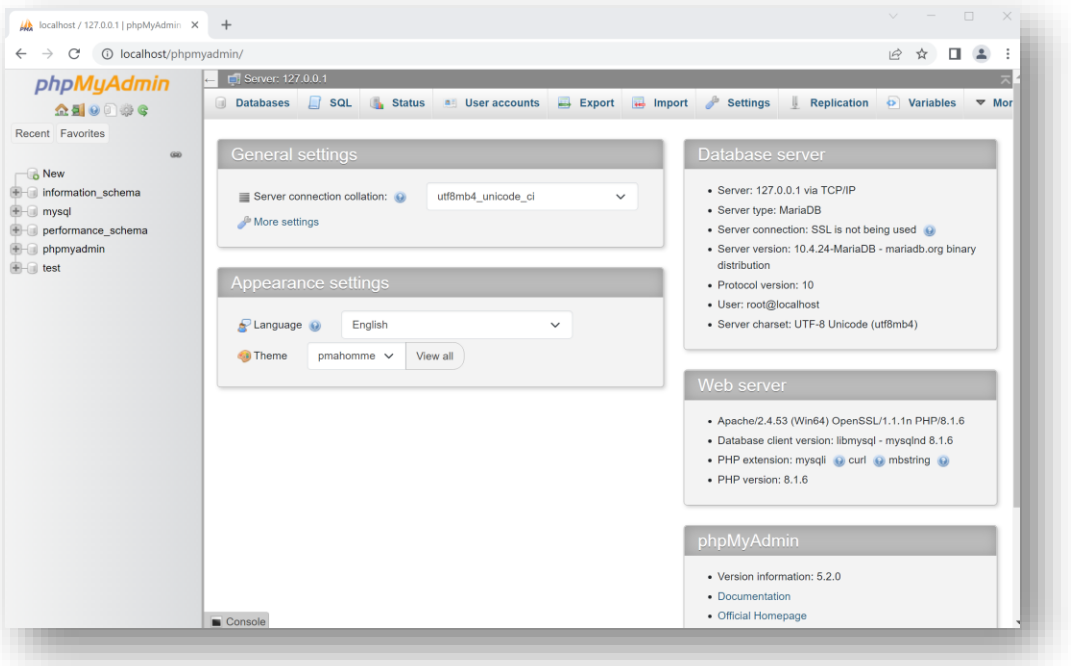


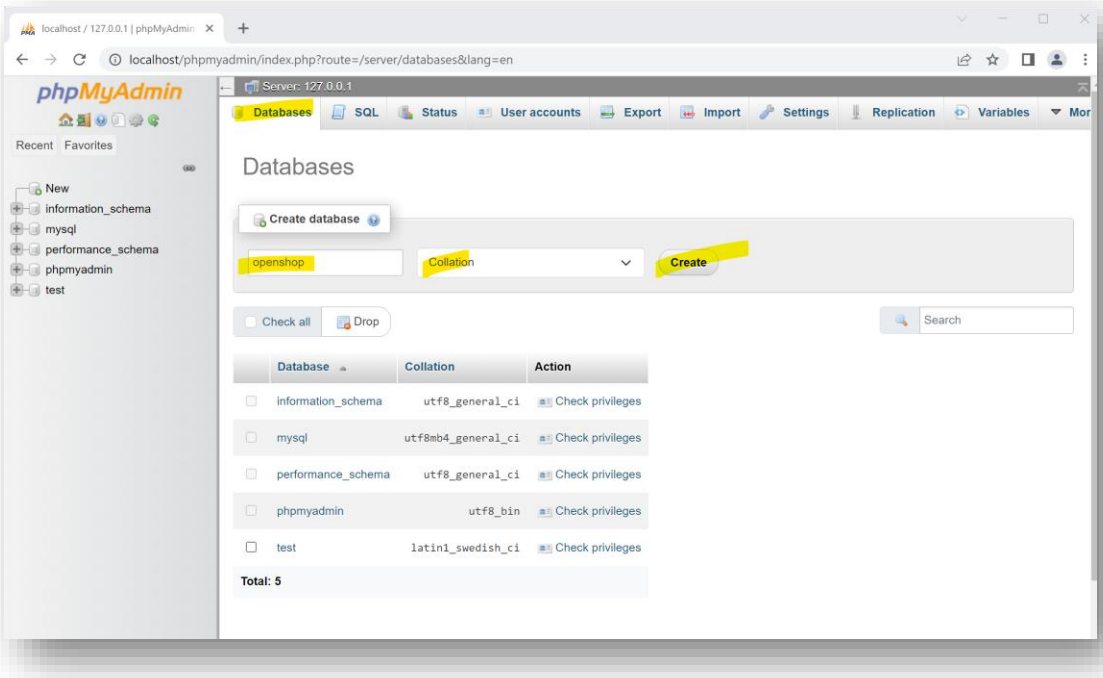
5) **Rename files**
   Go to C:\xampp\htdocs\opencart\upload
   Rename file 'config-dist.php' to 'config.php'

   Go to C:\xampp\htdocs\opencart\upload\admin
   Rename file 'config-dist.php' to 'config.php'

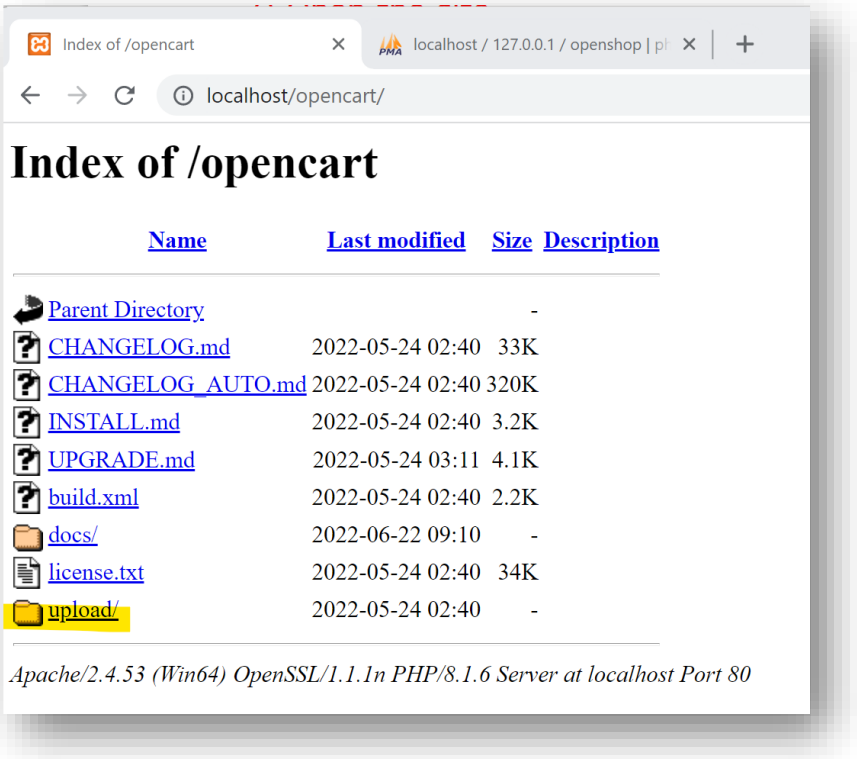
6) **Connect to the database and create DB.**
   DB Access URL: http://localhost/phpmyadmin/

   Create new database 'openshop' (Refer the screenshots below to create new Database in MySQL)



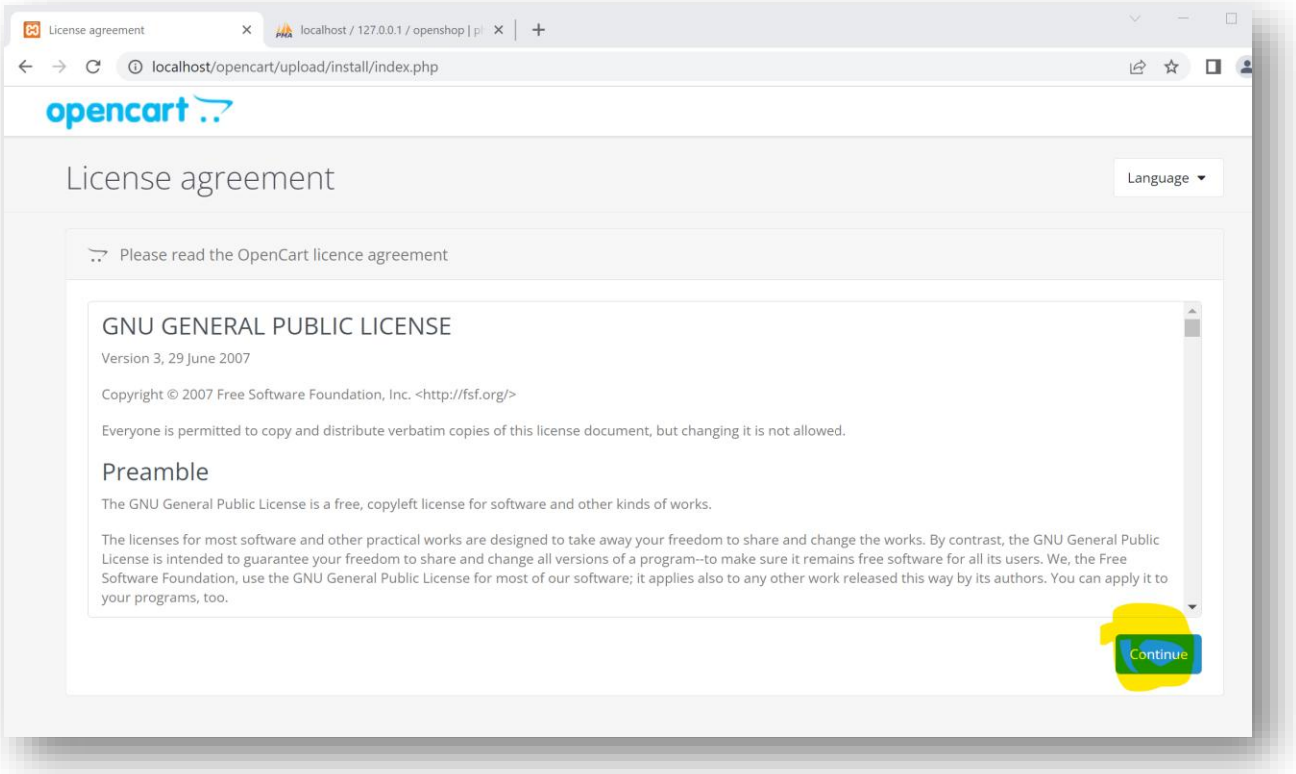


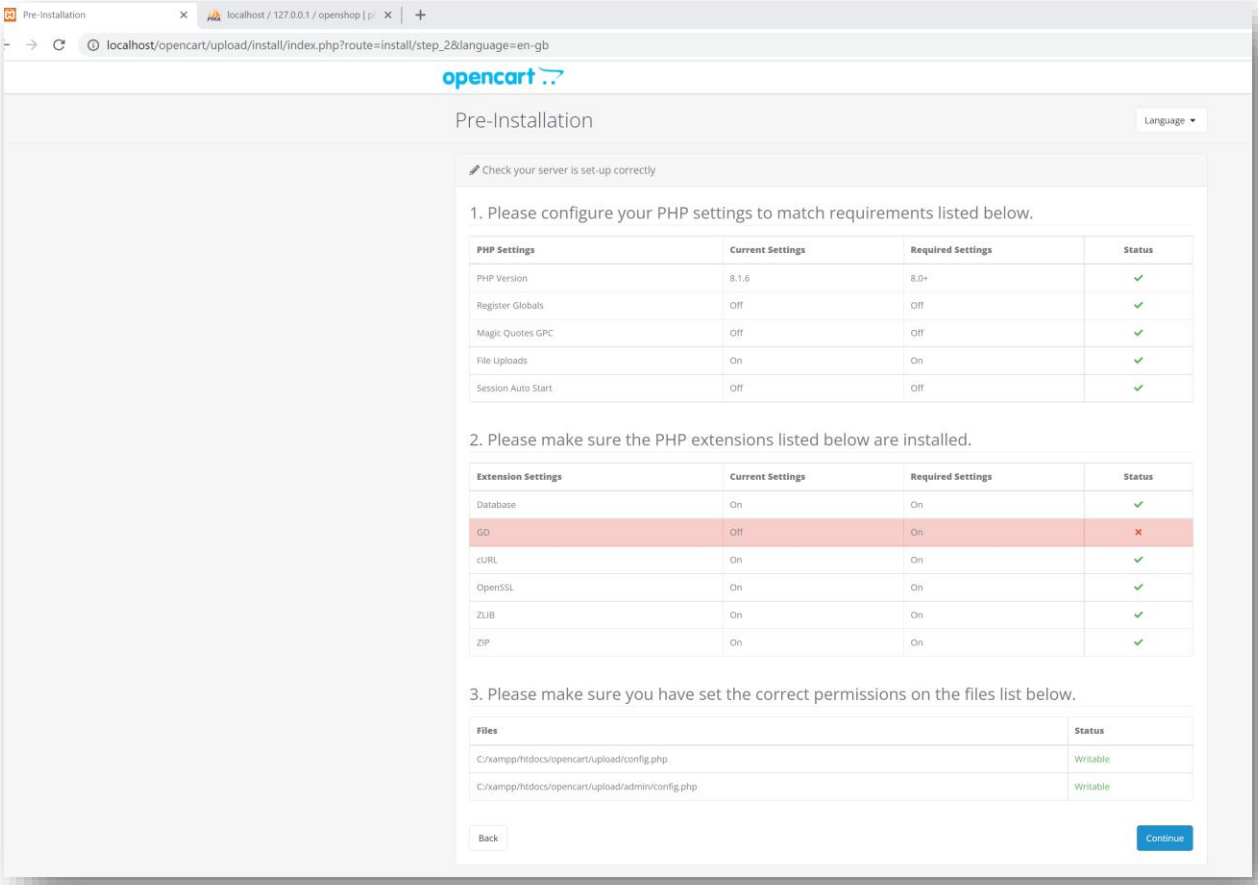
7) **Open the site** http://localhost/opencart/
   Click on upload



Navigate to http://localhost/opencart/upload/install/index.php

Click on CONTINUE Button....

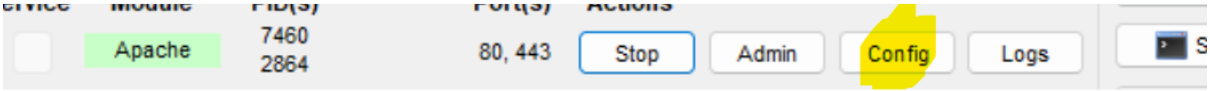




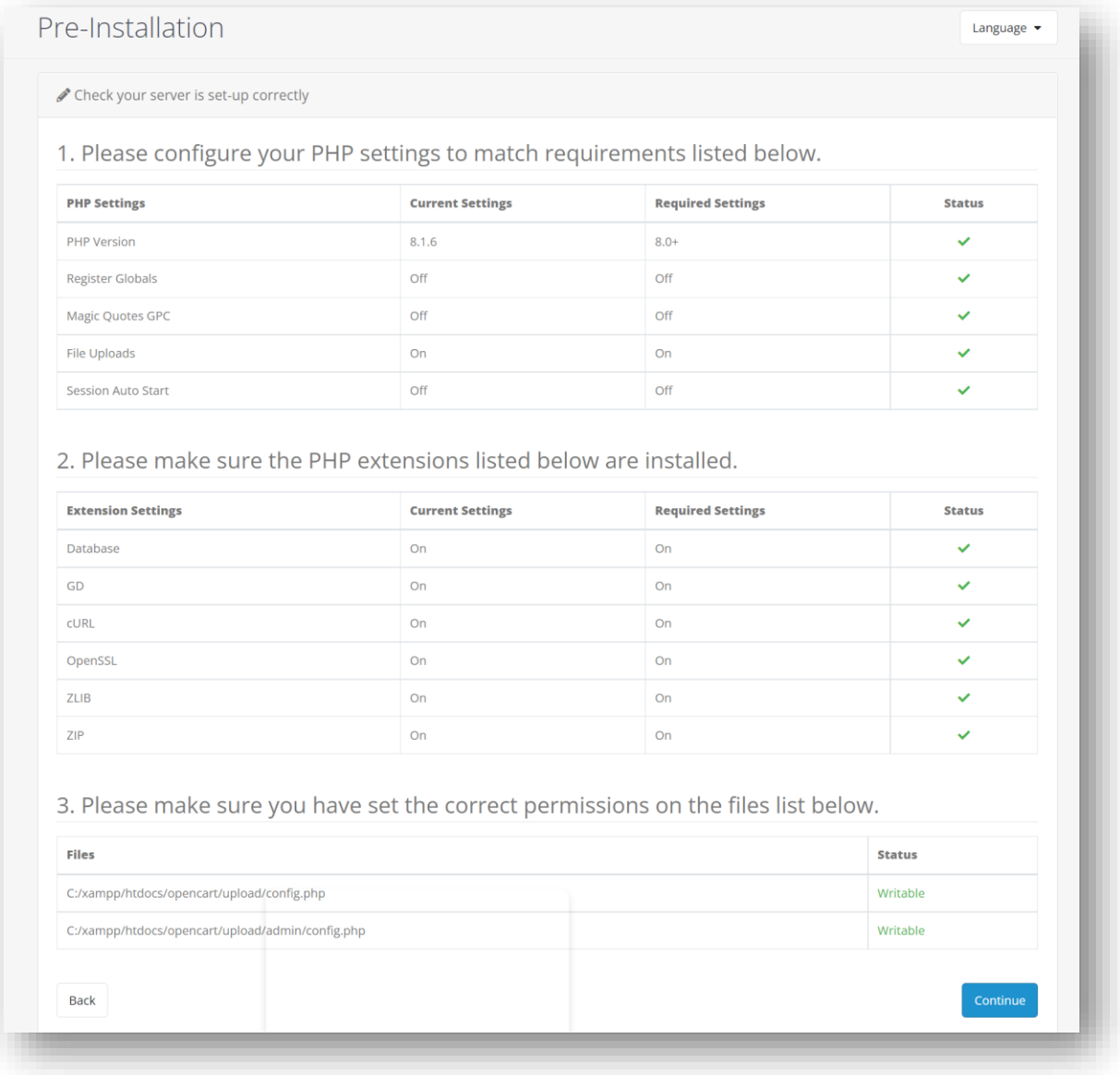
If you are getting above error GD is off then you have to do below steps.

Open XAMPP Control panel, click on config button for Apache then open PHP(php.ini) file

Add new entry in the php.ini file `extension=gd`

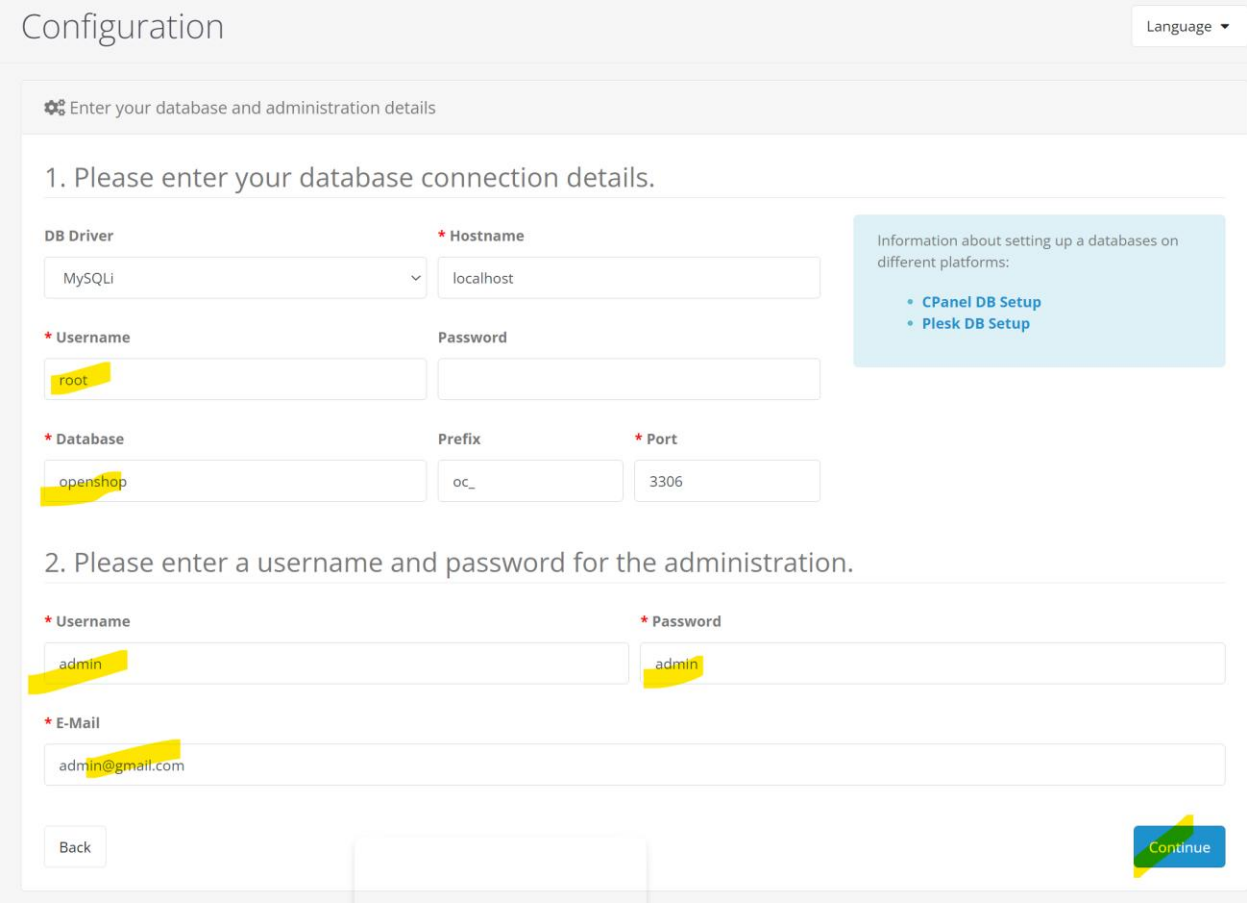


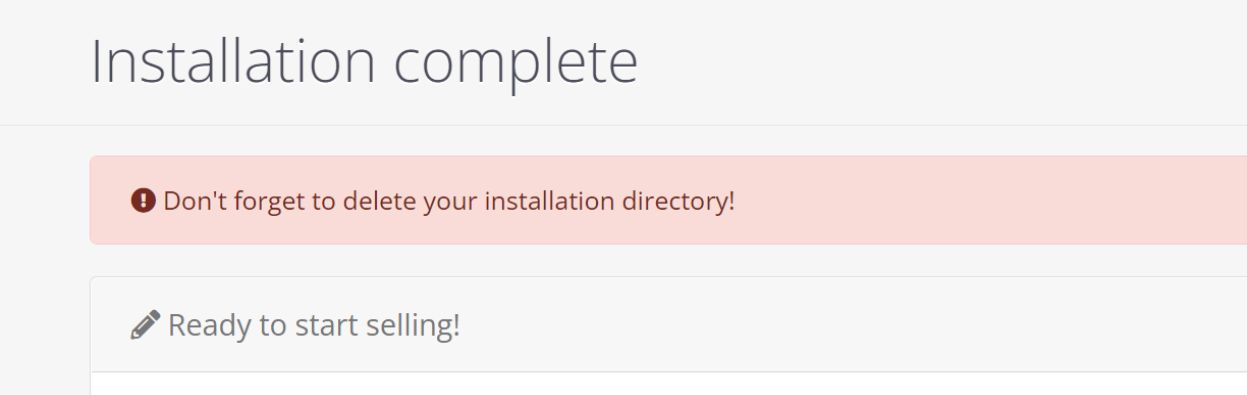
Then stop and start Apache server. Then you will see below screen.



Click on continue…..

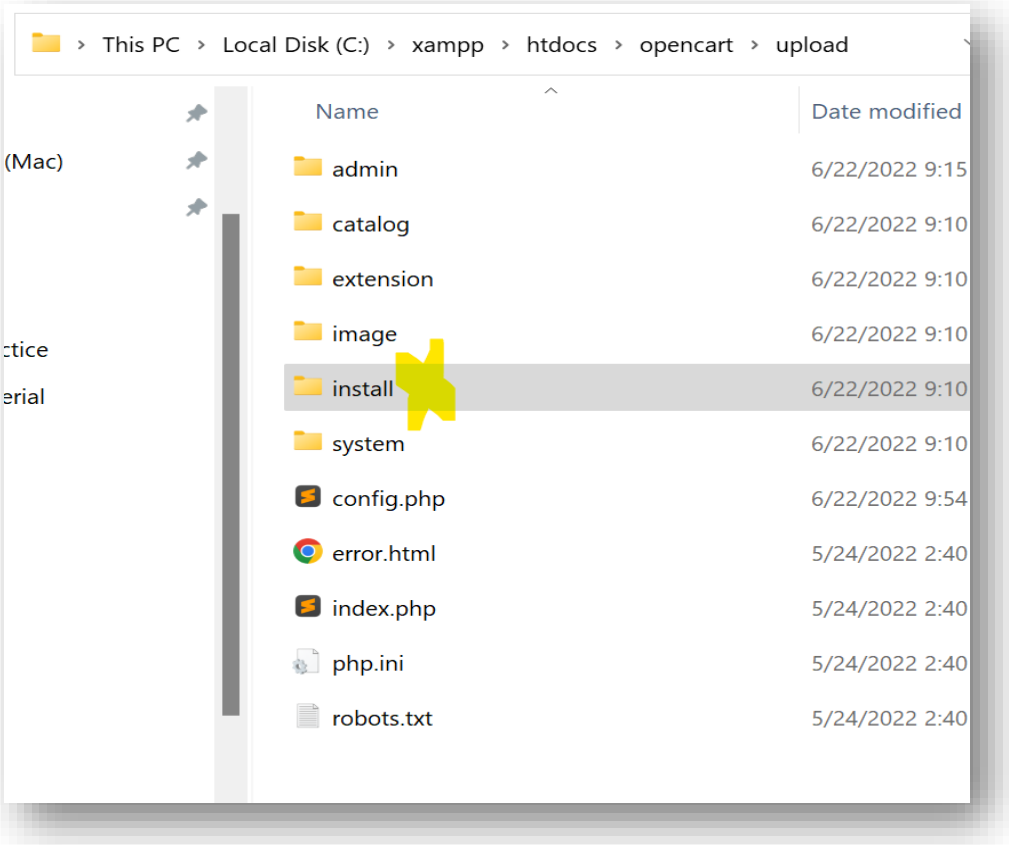
8) Provide Database connection details





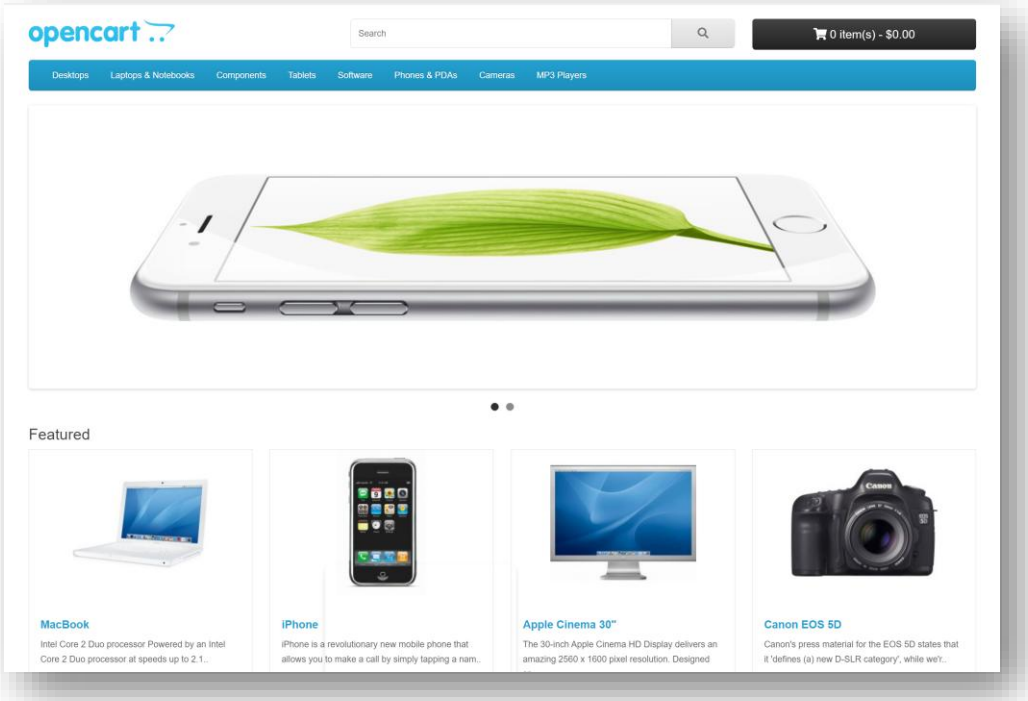
9) Go to location C:\xampp\htdocs\opencart\upload

Delete install folder

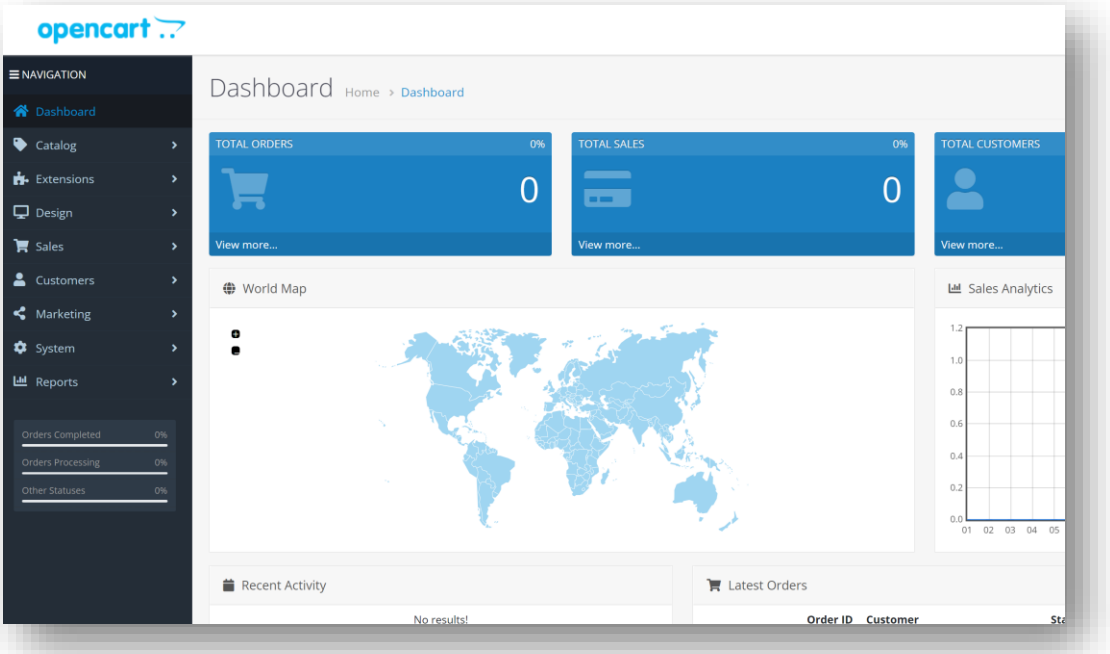


10) Go to Application URL

Frontend application URL: http://localhost/opencart/upload/



Backend Application url: http://localhost/opencart/upload/admin/

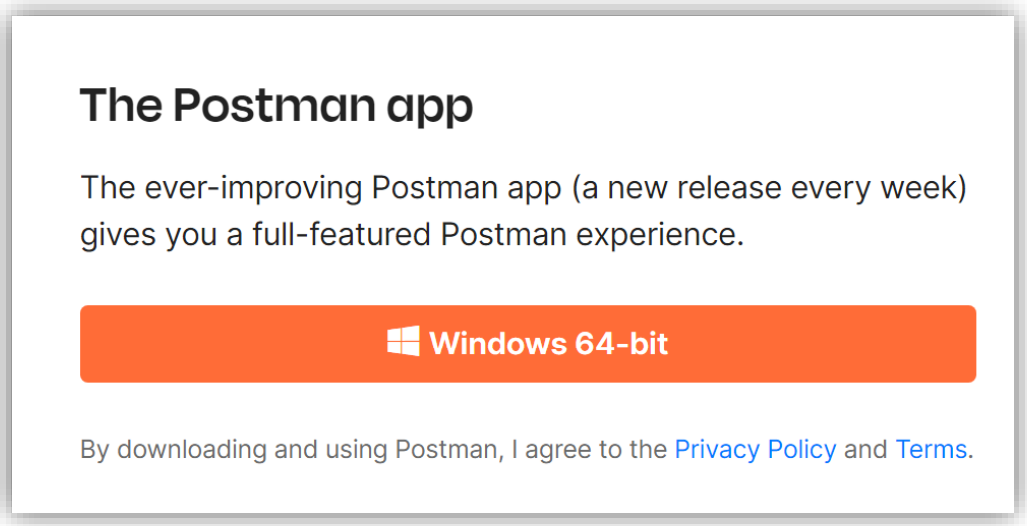


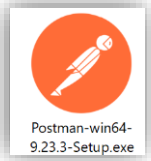
Postman

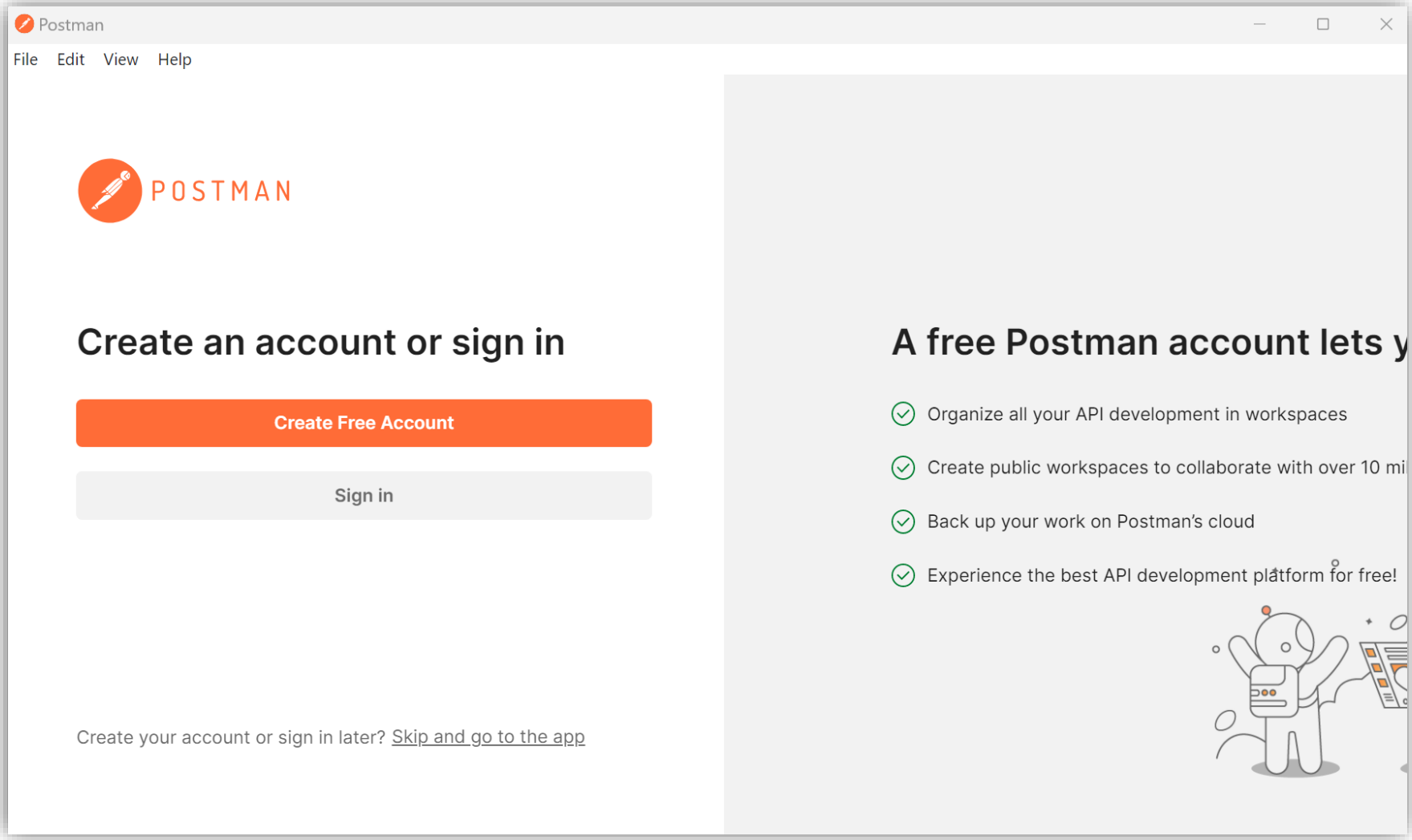
Postman Tool Installation

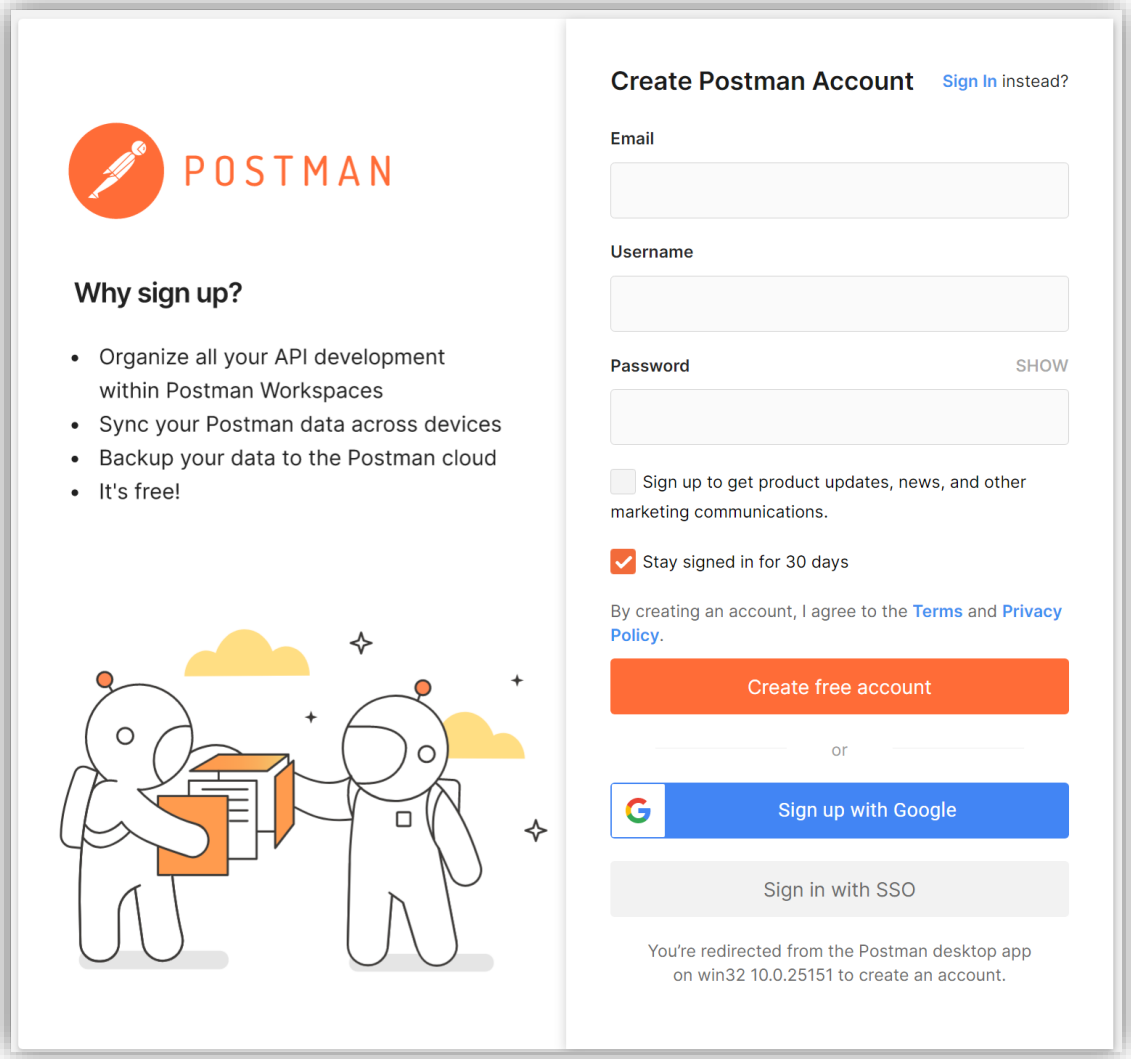
Download Postman Tool

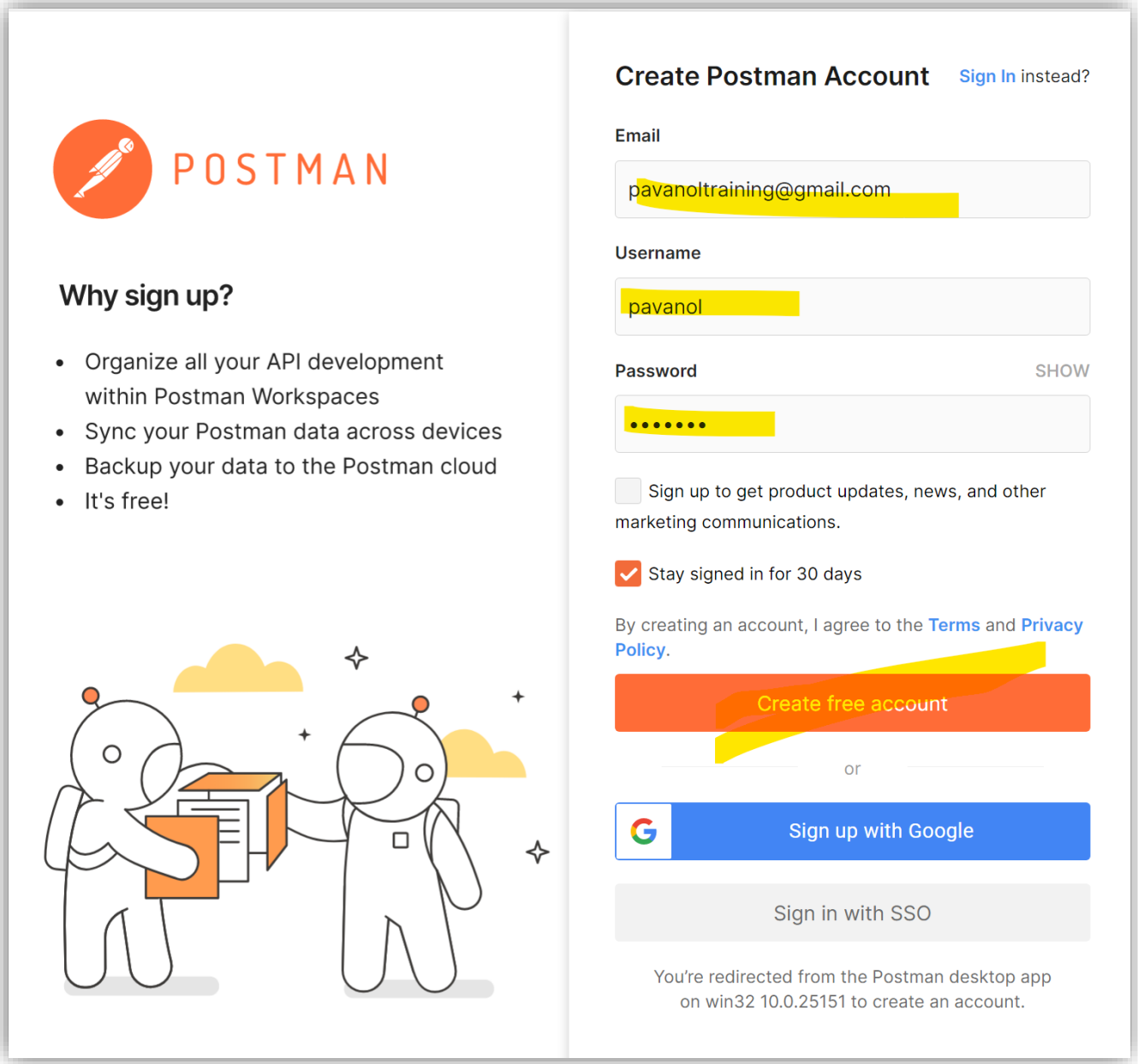
https://www.postman.com/downloads/

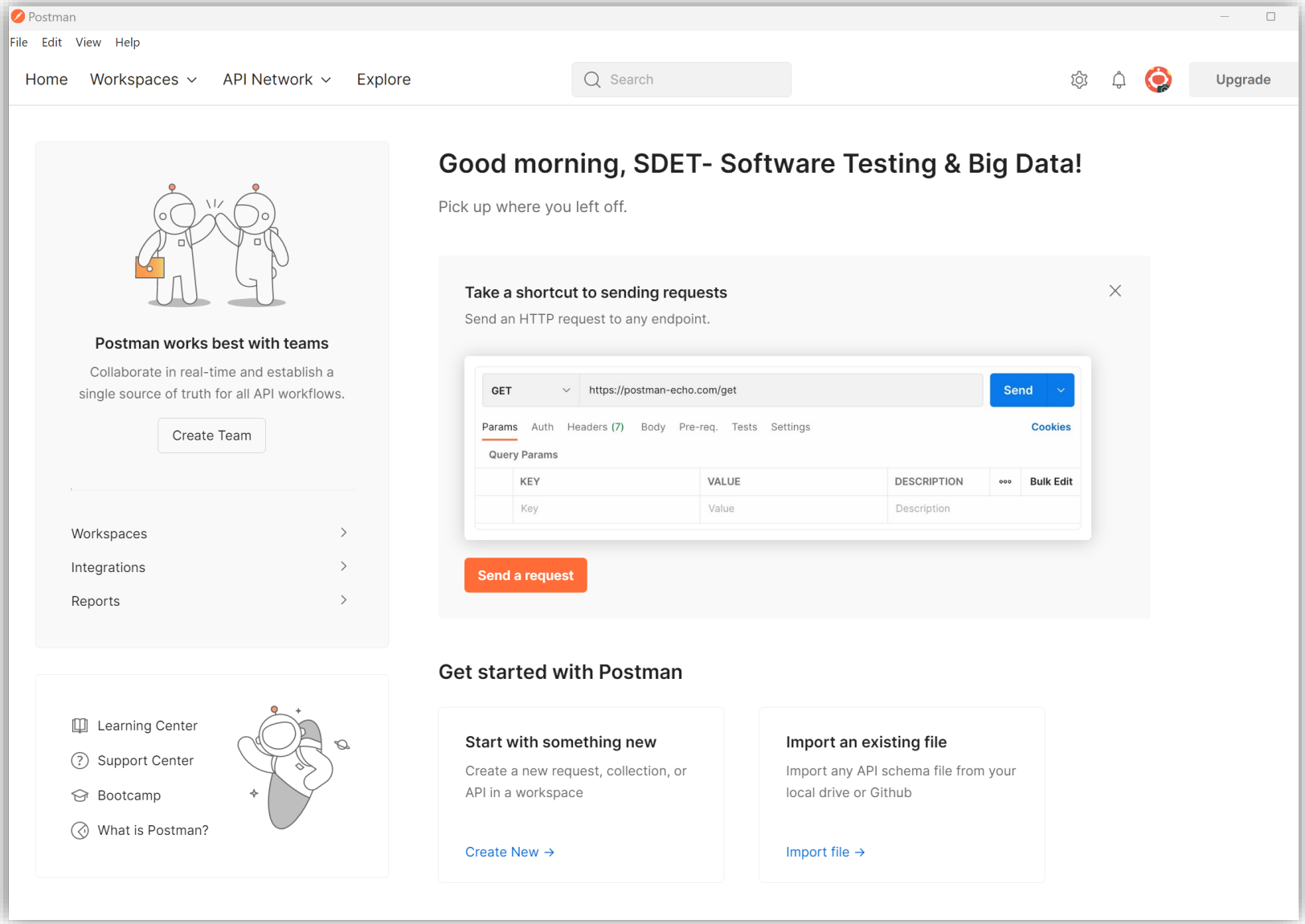





Postman
File Edit View Help
POSTMAN
Create an account or sign in
Create Free Account
Sign in
Create your account or sign in later? Skip and go to the app
A free Postman account lets y
Organize all your API development in workspaces
Create public workspaces to collaborate with over 10 mi
Back up your work on Postman's cloud
Experience the best API development platform for free!

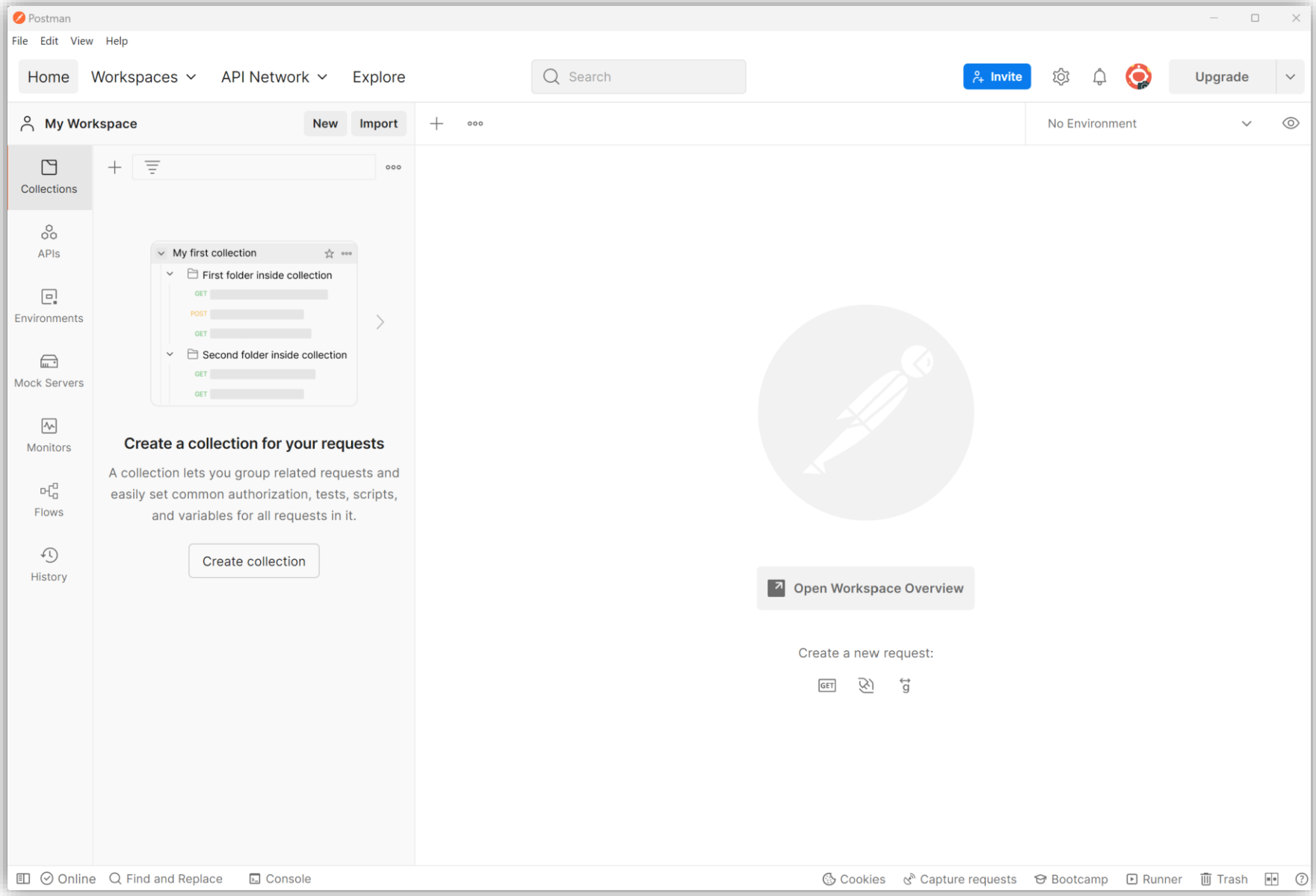

POSTMAN
Why sign up?
Organize all your API development within Postman Workspaces
Sync your Postman data across devices
Backup your data to the Postman cloud
It's free!
Create Postman Account
Sign In instead?
Email
Username
Password
SHOW
Sign up to get product updates, news, and other marketing communications.
Stay signed in for 30 days
By creating an account, I agree to the Terms and Privacy Policy.
Create free account
or
Sign up with Google
Sign in with SSO
You're redirected from the Postman desktop app on win32 10.0.25151 to create an account.


POSTMAN
Why sign up?
Organize all your API development within Postman Workspaces
Sync your Postman data across devices
Backup your data to the Postman cloud
It's free!
Create Postman Account
Sign In instead?
Email
pavanoltraining@gmail.com
Username
pavanol
Password
SHOW
Sign up to get product updates, news, and other marketing communications.
Stay signed in for 30 days
By creating an account, I agree to the Terms and Privacy Policy.
Create free account
or
Sign up with Google
Sign in with SSO
You're redirected from the Postman desktop app on win32 10.0.25151 to create an account.



https://www.pavanonlinetrainings.com

https://www.youtube.com/@sdetpavan

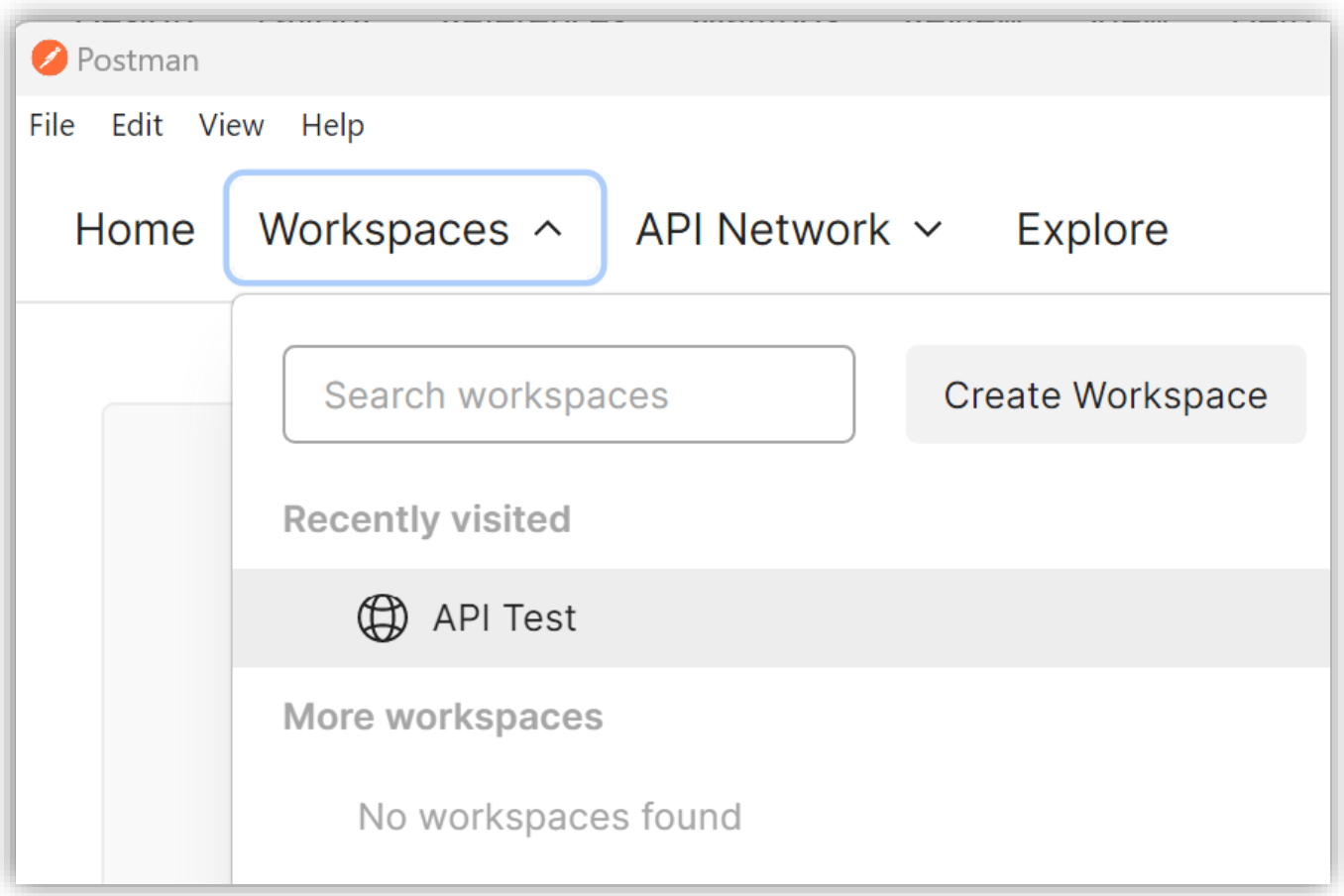


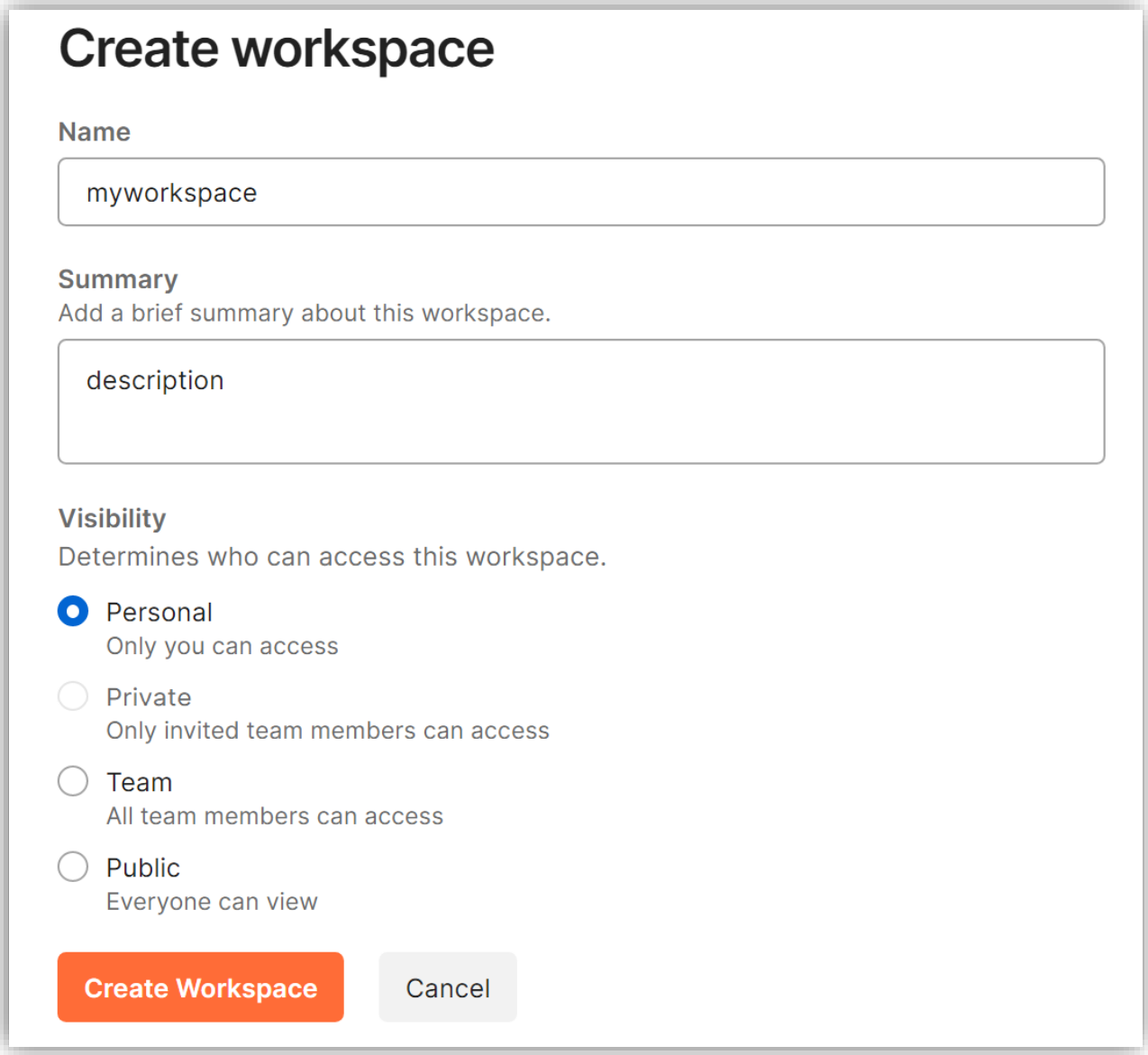
https://www.pavanonlinetrainings.com

https://www.youtube.com/@sdetpavan

Creating Work Space in Postman

Workspaces→Create Workspace




Create workspace
Name
myworkspace
Summary
Add a brief summary about this workspace.
description
Visibility
Determines who can access this workspace.
Personal
Only you can access
Private
Only invited team members can access
Team
All team members can access
Public
Everyone can view
Create Workspace
Cancel

Creating your team workspace

Hang on while we set up your team and invite people.

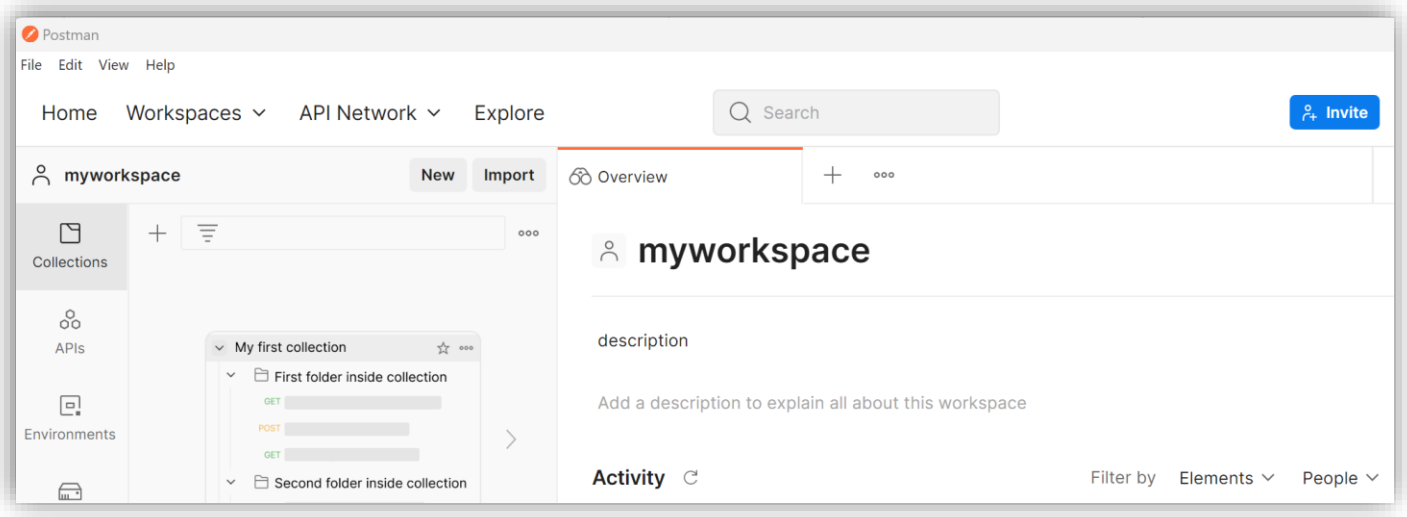
Please don’t close or refresh this tab.

Team workspace created

Customize your team’s profile to make it easy for your future team members to find.

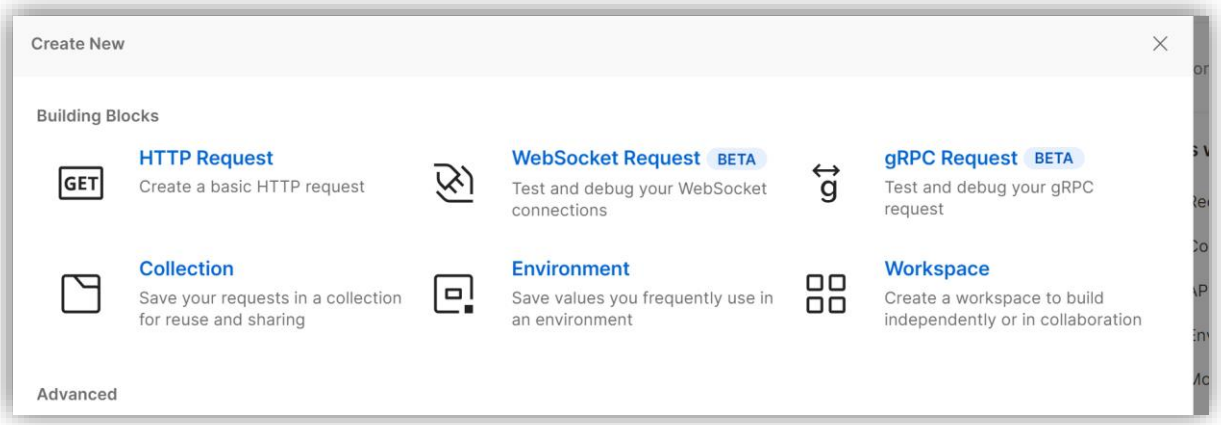
Edit Team profile

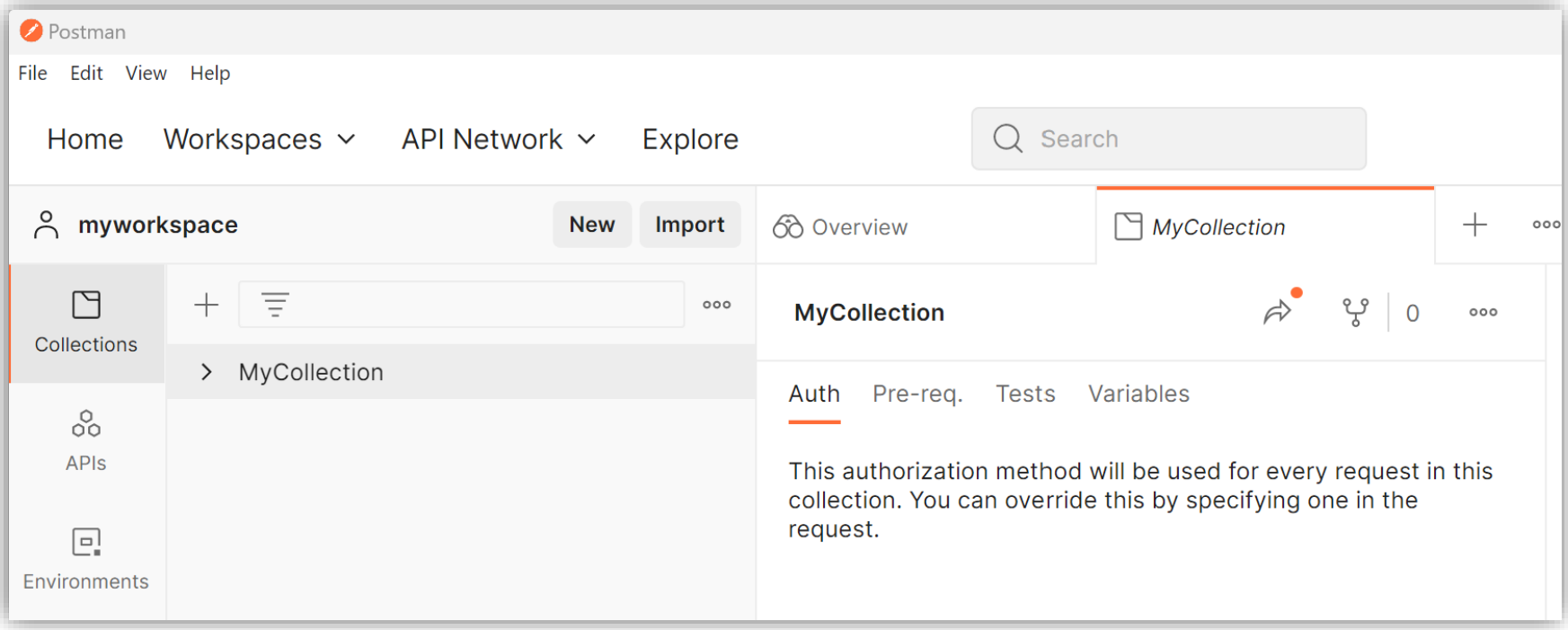
Go to Workspace



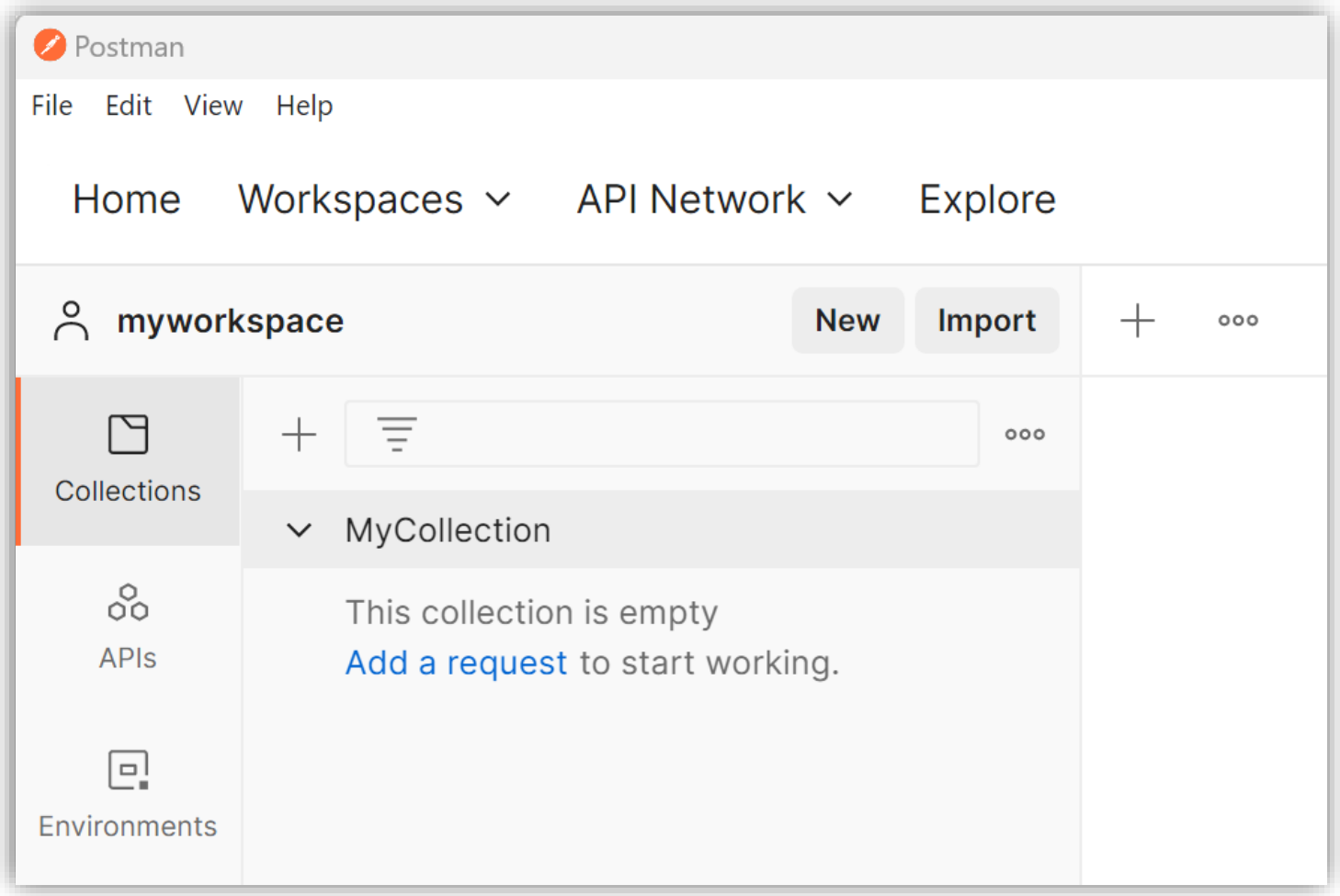
Creating Collection

File → New→Collection



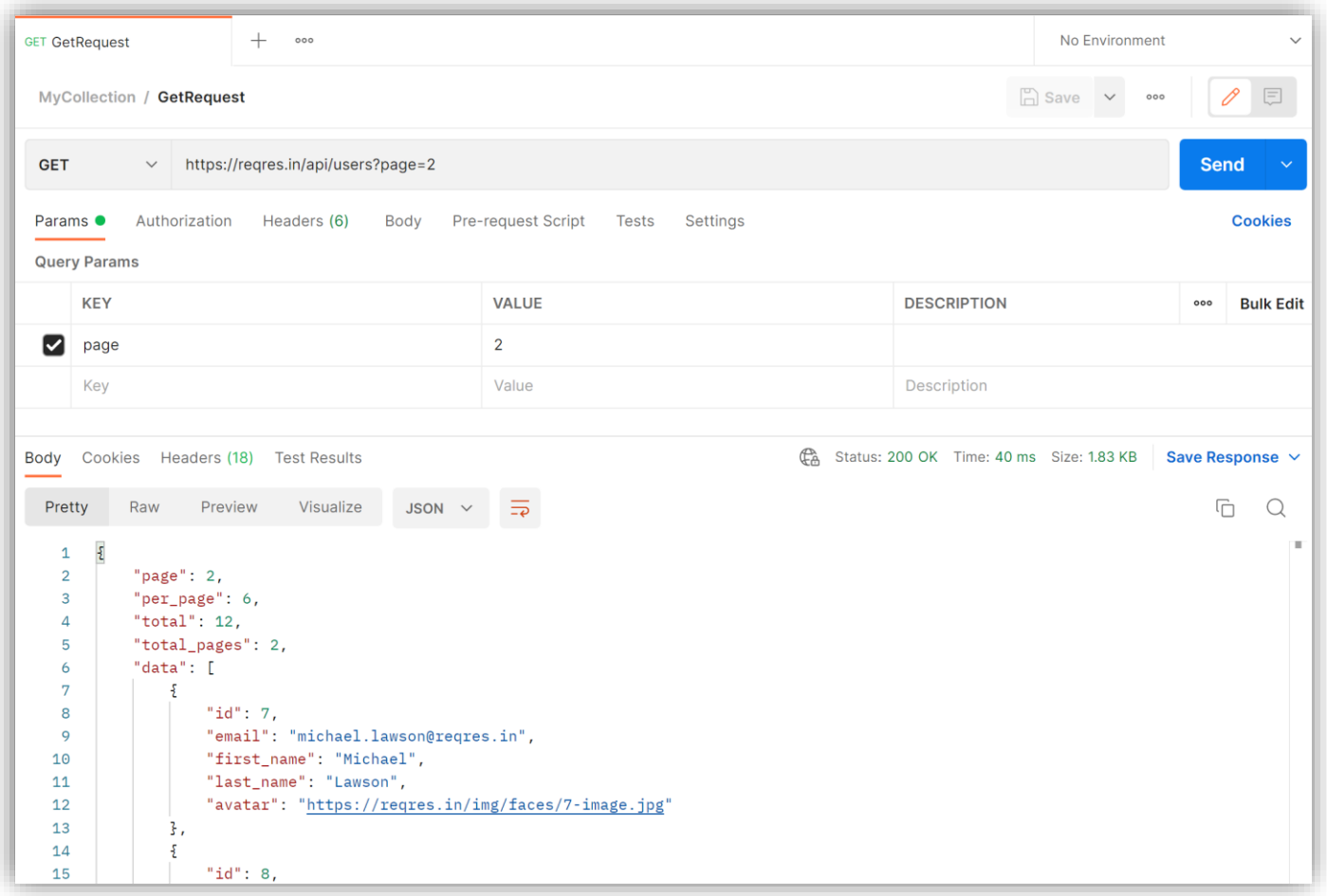

Postman
File Edit View Help
Home Workspaces API Network Explore
Search
myworkspace
New
Import
Overview
MyCollection
Collections
APIs
Environments
MyCollection
MyCollection
0
Auth Pre-req. Tests Variables
This authorization method will be used for every request in this collection. You can override this by specifying one in the request.

Add Requests



GET Request

https://reqres.in/api/users?page=2

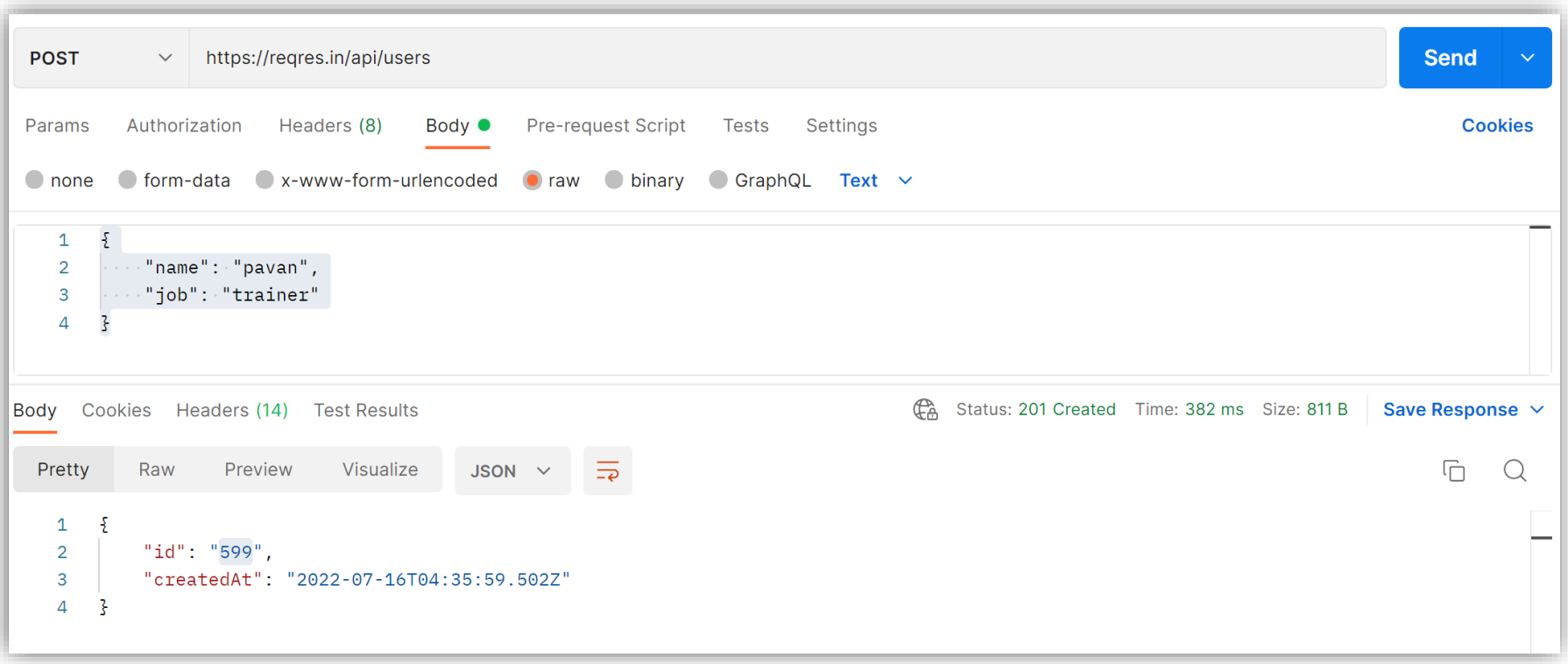


POST Request

https://reqres.in/api/users

Request Payload:

```
{
   "name": "pavan",
   "job": "trainer"
}
```



PUT Request

https://reqres.in/api/users/599

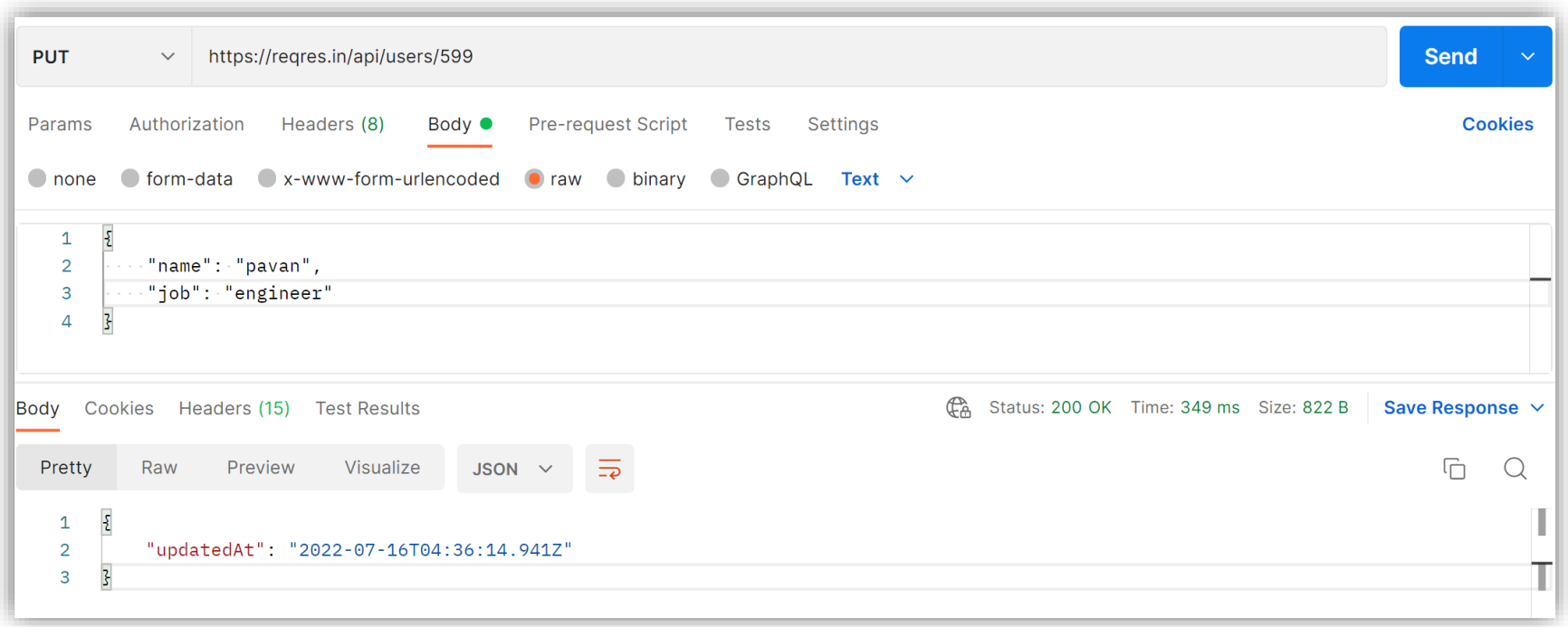
Request Payload:

{

"name": "pavan",

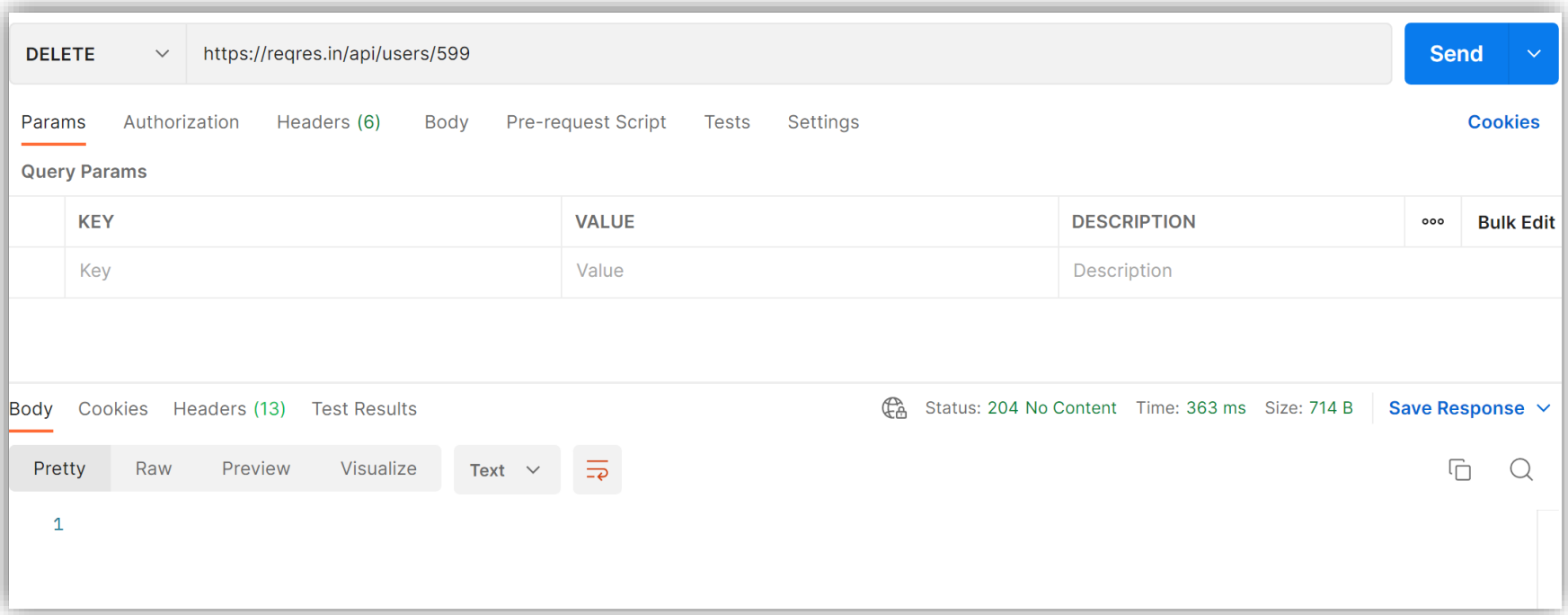
"job": "engineer"

}

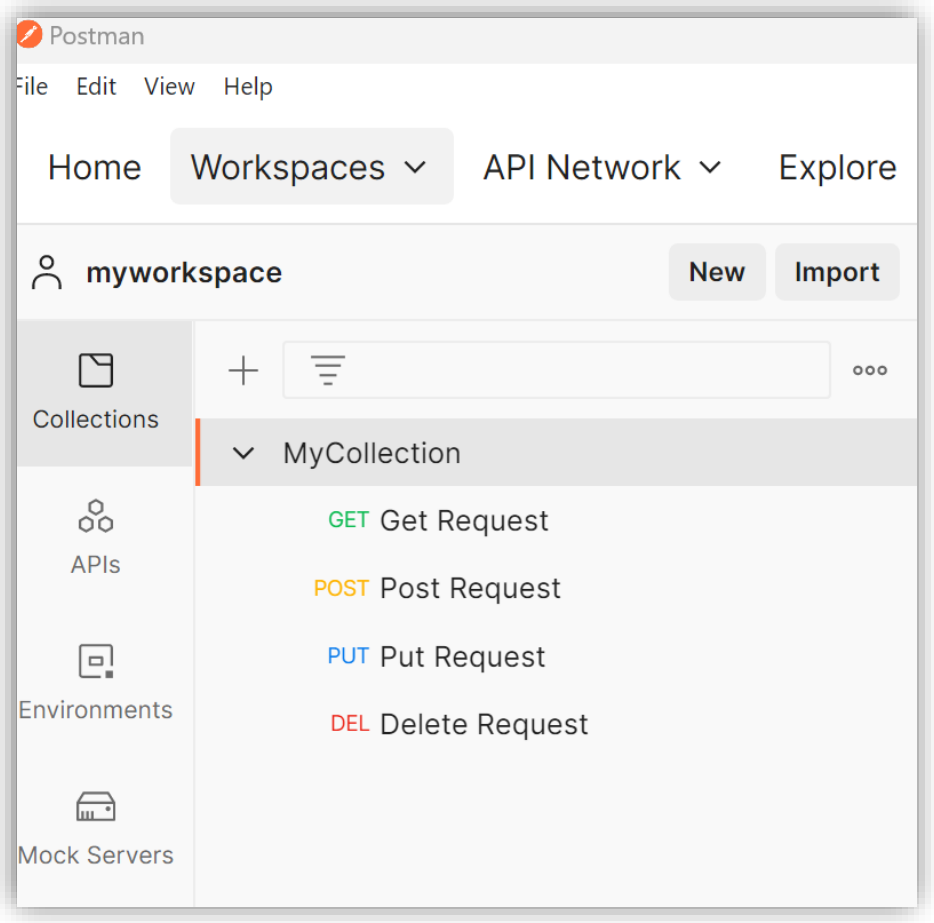


DELETE Request

https://reqres.in/api/users/599



Requests created under Collection



RESTful Booker API

The **RESTful Booker API** is a sample API used to practice REST API testing with operations like creating, retrieving, updating, and deleting hotel bookings.

✅ 1. Authentication (Auth)

🔐 Create Token

Purpose: Get a token to access protected endpoints like **PUT** and **DELETE** booking.

- **Endpoint:** POST https://restful-booker.herokuapp.com/auth
- **Headers:**
  Content-Type: application/json
- **Body:**

```
{
 "username": "admin",
 "password": "password123"
}
```

- **Success Response:**

```
{
 "token": "abc123"
}
```

Use this token in a cookie/header for secured operations like update or delete.

📘 2. Booking Operations

🆔 Get All Booking IDs

Purpose: Get a list of all existing booking IDs.

- **Endpoint:** GET https://restful-booker.herokuapp.com/booking
- **Optional Filters:**
  - firstname
  - lastname
  - checkin (format: YYYY-MM-DD)
  - checkout (format: YYYY-MM-DD)

- **Response Example:**

```
[
 { "bookingid": 1 },
 { "bookingid": 2 }
]
```

🔍 Get Booking by ID

Purpose: Fetch full details of a booking by ID.

- **Endpoint:** GET https://restful-booker.herokuapp.com/booking/{id}
- **Headers:**
  Accept: application/json
- **Response Example:**

```
{
 "firstname": "Sally",
 "lastname": "Brown",
 "totalprice": 111,
 "depositpaid": true,
 "bookingdates": {
  "checkin": "2013-02-23",
  "checkout": "2014-10-23"
 },
 "additionalneeds": "Breakfast"
}
```

🆕 Create Booking

Purpose: Add a new booking.

- **Endpoint:** POST https://restful-booker.herokuapp.com/booking
- **Headers:**
  - Content-Type: application/json
  - Accept: application/json

- **Body Example:**

```
{
 "firstname": "Jim",
 "lastname": "Brown",
 "totalprice": 111,
 "depositpaid": true,
 "bookingdates": {
  "checkin": "2018-01-01",
  "checkout": "2019-01-01"
 },
 "additionalneeds": "Breakfast"
}
```

- **Response Example:**

```
{
 "bookingid": 1,
 "booking": {
  ...
 }
}
```

🔁 **Update Booking (Full)**

Purpose: Replace the full booking details.

- **Endpoint:** PUT https://restful-booker.herokuapp.com/booking/{id}
- **Headers:**
  - Content-Type: application/json
  - Accept: application/json
  - Cookie: token=abc123 *(or use Authorization header)*
- **Body:** Same structure as **Create Booking**
- **Success Response:** Updated booking JSON

🖍️ Partial Update Booking

Purpose: Update only some fields of a booking.

- **Endpoint:** PATCH https://restful-booker.herokuapp.com/booking/{id}
- **Headers:** Same as PUT
- **Body Example:**

```
{
 "firstname": "James",
 "lastname": "Brown"
}
```

- **Response:** Updated fields + existing booking data

❌ Delete Booking

Purpose: Delete a booking by ID.

- **Endpoint:** DELETE https://restful-booker.herokuapp.com/booking/{id}
- **Headers:**
  - Cookie: token=abc123 or
  - Authorization: Basic YWRtaW46cGFzc3dvcmQxMjM=
- **Response:**

  HTTP/1.1 201 Created

💡 Quick Tips

- Always authenticate before **updating** or **deleting** bookings.
- Use tools like **Postman** to test endpoints.
- Format dates as YYYY-MM-DD.

RESTful Booker API

The **RESTful Booker API** is a sample API used to practice REST API testing with operations like creating, retrieving, updating, and deleting hotel bookings.

✅ 1. Authentication (Auth)

🔐 Create Token

Purpose: Get a token to access protected endpoints like **PUT** and **DELETE** booking.

- **Endpoint:** POST https://restful-booker.herokuapp.com/auth
- **Headers:**
  Content-Type: application/json
- **Body:**

```
{
 "username": "admin",
 "password": "password123"
}
```

- **Success Response:**

```
{
 "token": "abc123"
}
```

Use this token in a cookie/header for secured operations like update or delete.

📘 2. Booking Operations

🆔 Get All Booking IDs

Purpose: Get a list of all existing booking IDs.

- **Endpoint:** GET https://restful-booker.herokuapp.com/booking
- **Optional Filters:**
  - firstname
  - lastname
  - checkin (format: YYYY-MM-DD)
  - checkout (format: YYYY-MM-DD)

- **Response Example:**

```
[
 { "bookingid": 1 },
 { "bookingid": 2 }
]
```

🔍 Get Booking by ID

Purpose: Fetch full details of a booking by ID.

- **Endpoint:** GET https://restful-booker.herokuapp.com/booking/{id}
- **Headers:**
  Accept: application/json
- **Response Example:**

```
{
 "firstname": "Sally",
 "lastname": "Brown",
 "totalprice": 111,
 "depositpaid": true,
 "bookingdates": {
  "checkin": "2013-02-23",
  "checkout": "2014-10-23"
 },
 "additionalneeds": "Breakfast"
}
```

🆕 Create Booking

Purpose: Add a new booking.

- **Endpoint:** POST https://restful-booker.herokuapp.com/booking
- **Headers:**
  - Content-Type: application/json
  - Accept: application/json

- **Body Example:**

```
{
 "firstname": "Jim",
 "lastname": "Brown",
 "totalprice": 111,
 "depositpaid": true,
 "bookingdates": {
  "checkin": "2018-01-01",
  "checkout": "2019-01-01"
 },
 "additionalneeds": "Breakfast"
}
```

- **Response Example:**

```
{
 "bookingid": 1,
 "booking": {
  ...
 }
}
```

🔁 **Update Booking (Full)**

Purpose: Replace the full booking details.

- **Endpoint:** PUT https://restful-booker.herokuapp.com/booking/{id}
- **Headers:**
  - Content-Type: application/json
  - Accept: application/json
  - Cookie: token=abc123 *(or use Authorization header)*
- **Body:** Same structure as **Create Booking**
- **Success Response:** Updated booking JSON

🖍️ Partial Update Booking

Purpose: Update only some fields of a booking.

- **Endpoint:** PATCH https://restful-booker.herokuapp.com/booking/{id}
- **Headers:** Same as PUT
- **Body Example:**

```
{
 "firstname": "James",
 "lastname": "Brown"
}
```

- **Response:** Updated fields + existing booking data

❌ Delete Booking

Purpose: Delete a booking by ID.

- **Endpoint:** DELETE https://restful-booker.herokuapp.com/booking/{id}
- **Headers:**
  - Cookie: token=abc123 or
  - Authorization: Basic YWRtaW46cGFzc3dvcmQxMjM=
- **Response:**

  HTTP/1.1 201 Created

💡 Quick Tips

- Always authenticate before **updating** or **deleting** bookings.
- Use tools like **Postman** to test endpoints.
- Format dates as YYYY-MM-DD.